SyRA: Sybil-Resilient Anonymous Signatures with Applications to Decentralized Identity

Elizabeth Crites[1], Aggelos Kiayias[2], Markulf Kohlweiss[2], and Amirreza Sarencheh[2⋆]

[1] Web3 Foundation, Zug, Switzerland
elizabeth@web3.foundation
[2] The University of Edinburgh and IOG, Edinburgh, UK
firstname.lastname@ed.ac.uk

Abstract. We study Sybil-Resilient Anonymous (SyRA) signatures, a cryptographic primitive that enables credentialed users to generate, on demand, unlinkable pseudonyms tied to any given context, and issue signatures on behalf of these pseudonyms. Concretely, SyRA allows a distributed issuer to turn any legacy identity or personhood identifier, possibly of low entropy, into a unique associated cryptographic key of high pseudoentropy, for use in generating signatures for any given context. Sybil-resilient anonymous signatures achieve three main objectives: 1) *Sybil resilience*: every user is entitled to *at most one* digital identity, 2) *anonymity*: no information about the user's real identity is leaked, and 3) *non-interactive context switching*: users can create on their own at most one credential for any given context in a manner that is unlinkable across contexts.

We conceptualize the SyRA primitive as an ideal functionality in the Universal Composition (UC) setting and put forth SASSI, an efficient, pairing-based construction that realizes it by utilizing two levels of verifiable random functions (VRFs), a design which may be of independent interest. The first level consists of threshold VRF issuance of a user's unique secret key tied to their real-world identifier. The second level allows a user to create signatures for each context, under a unique pseudonym per context. Compared to prior cryptographic tools capable of realizing SyRA, SASSI has the unique feature that issuers are *stateless* and hence do not need to retain any information about past user interactions, a relevant property for a decentralized implementation.

We overview various applications of SASSI in multiparty systems, such as cryptocurrency account management and airdrops, e-voting (e.g., for decentralized governance), and privacy-preserving regulatory compliance (e.g., AML/CFT checks). In the context of creating addresses for digital assets, SyRA signatures enable users to embed their legacy identity into their address in a manner that protects their privacy for each application with which they interact. We demonstrate the practicality of SASSI by providing an implementation and performance evaluation of our construction.

Keywords: Anonymous Signatures · Sybil Resilience · Decentralized Identity · Decentralized Governance · Universal Composition

⋆ **Corresponding author:** Amirreza Sarencheh

1 Introduction

A recent development in identity systems is the concept of decentralized identity (DID), which comes with the promise of empowering users by enabling them to be "self sovereign" in terms of identity management.[3] This means that users have the ability to self-manage the secret key(s) behind their identities and access services that require identification without a trusted third party acting as an intermediary, e.g., in the case of single sign-on systems, cf. [Wang et al.(2012), Fett et al.(2016)]. From a cryptographic perspective, two seemingly contradictory requirements are relevant for identity systems.

- *Sybil resilience.* With the help of the issuing authority, each user can translate their real-world identity[4] into *at most one* credential.
- *Anonymity.* Credentials reveal nothing about the user's identity, even allowing them to be unlinkable across applications.

Given that users may be operating across a multitude of applications, or *contexts*, it is important for them to be able to easily generate "context-specific" credentials that, for privacy considerations, must be unlinkable.[5] This highlights the following property.

- *Non-interactive context switching.* Users can create (at most one) credential, on demand and non-interactively, for a given context, in a manner that is unlinkable across contexts. [6]

Achieving the above properties is feasible by employing some substantial cryptographic machinery (see below in related work) but an important consideration still remains: in all existing approaches capable of meeting the above considerations, the connection to the real-world identity of the signer is tenuous – it fundamentally relies on the ability of the credential issuer to maintain a mapping between credentials and real-world identities. This raises scalability, security, and privacy considerations, namely: how is it possible to manage such an ever-growing mapping of real-world identities to credentials, distribute such a mapping across many servers with Byzantine fault tolerance, and, at the same time, keep it private in order to prevent adversaries from performing deanonymization attacks via correlation? We refer to these considerations as the *(credential) issuer state problem.*

[3] The W3C Decentralized Identifier working group [Consortium(2022)] and Decentralized Identity Foundation [Foundation(2023)] have been developing standards for DIDs.

[4] Examples of real-world identities include government-issued identifiers such as a Social Security Number (SSN) in the United States, or national ID numbers.

[5] See Sec. 4.2 of the Peer-DID standard that covers-context specific DIDs for users `https://identity.foundation/peer-did-method-spec/index.html`.

[6] An interactive context-specific credential generation introduces a dependency on issuers, which can have negative effects both for latency and privacy, since the user will have to interact with the issuers each time they engage with a new app.

Solving the issuer state problem poses a conundrum that begs cryptographic treatment: Sybil resilience seems to mandate that the issuer maintain a mapping of real-world identities to credentials; indeed, failure to do so would permit a trivial attack against Sybil resilience: the user would exploit the confusion of the issuer to dispense more than one credential for itself and thus become a Sybil. On the other hand, scaling and distributing the issuer would benefit from obfuscating, or even eliminating (if possible!) such state altogether.

Motivated by the connection of all of the above to the DID problem, we focus in this work on the following question: *What is a natural cryptographic primitive that reconciles Sybil resilience with anonymity and non-interactive context switching, and how can we realize it in a way that solves the issuer state problem?*

Our results. We tackle the above question by putting forth a new primitive, **Sy**bil-**R**esilient **A**nonymous (SyRA) signatures, and realizing it via an efficient construction that solves the issuer state problem, SASSI (**Sy**bil-resilient **A**nonymous **S**ignatures with **S**tateless **I**ssuing), and facilitates a *stateless* distributed issuer entity.

In a SyRA signature scheme, a set of issuers is tasked with translating a real-world identity s into a signing key for the prospective user.[7] By "real-world identity" (cf. Section 3) we refer to a unique identifier associated with a user through a verifiable *personhood relation*. This includes government-issued credentials (e.g., an X.509 attribute certificate tied to a national ID), a biometric identifier as in systems like Worldcoin [Worldcoin(2023)] (cf. Section 6), or other identity assertions from legacy systems, such as those provided via DECO [Zhang et al.(2020)] or OAuth/OpenID [Sakimura et al.(2014)]. Crucially, we abstract the notion of identity through a public personhood relation $\mathcal{R}_s$ that is verifiable by the issuers, so that we enable compatibility with a broad range of identity instantiations without prescribing any specific realization. Given this, the issuers introduce a user into the system whose identity string satisfies the personhood relation $\mathcal{R}_s$ and must ensure that there is a one-to-one correspondence between issued keys and distinct, real-world identities. Once the issuing protocol succeeds, a SyRA signature scheme enables a user to sign messages for any given context ctx; signatures can be verified against the public key of the issuing authorities and the context string. The signing procedure generates a context-specific pseudonym, or tag T, that is attached to each signature. This enables the user to create separate and unlinkable pseudonyms to issue signatures across different contexts. Nevertheless, within a specific context, the signatures issued by the user can be linked to one another, see Figure 1.

To capture formally the properties of this new primitive, we provide an ideal functionality for SyRA signatures in Section 3. Some key characteristics of this

[7] Note that for SyRA signatures with centralized malicious issuer, we cannot expect to have privacy, since in that case, the issuer could always impersonate a real-world user with identity s and use the tracing operation to find all of the user's signatures. Note also that s is assumed to be low entropy, such as a government identification number.

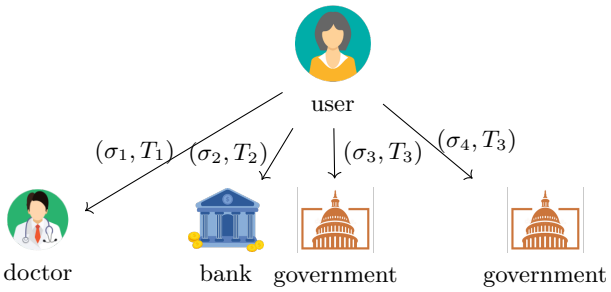

Fig. 1: Illustration of SyRA signatures' pseudonyms T enabling anonymous, unlinkable engagements with various contexts. Each signature carries a pseudonym T_i derived from the user's identity s and the application context. When the same user engages with the same app more than once, the pseudonym remains the same enabling the app to service the user by maintaining user specific state. Nevertheless, even against collusion of parties, the pseudonyms reveals nothing about s or whether the same user is behind two separate interactions between two different apps.

functionality are as follows. First, it verifies that the prospective user possesses a valid witness for their identity s with respect to the personhood relation $\mathcal{R}_s$. Second, upon each signature generation step, it creates a unique user-context identifier that is leaked to the adversary. This ensures that as long as unique user-context combinations are being exercised, there is no loss of anonymity. On the other hand, once the same user signs with the same context twice, the adversary will be notified by receiving the same user-context identifier. Note that this does not allow correlation with user actions in other contexts. Finally, the verification interface enables the verifier to publicly extract the pseudonym of each signature.

We then set out to present our construction, SASSI, which realizes SyRA via a distributed and stateless set of issuers, cf. Section 4. From a design perspective, the challenge is to embed the identity of the user into their signing key so that signatures are unlinkable as long as distinct contexts are signed by different users, and at the same time enable linking for user signatures on the same context. Crucially, we cannot rely on any auxiliary cryptographic structure in the membership credential – this is because even if the user repeats the whole issuing protocol again, as long as their real-world identity s remains the same, the user should not be able to sign the same context more than once without being linked to the previous instantiation. In this way, our issuers can be stateless.

Our construction overcomes this difficulty via the composition of two verifiable random functions (VRF). The first VRF is keyed by the issuing authorities. During issuance, the user obtains the VRF value on their real-world identity string s while proving to the issuing authorities that the personhood relation $\mathcal{R}_s$ is satisfied. In this manner, the user obtains a value that is deterministically tied to their real-world identity s. The second challenge is to ensure that this value can be safely used as the cryptographic key to a second VRF. The reason we

aim for a second VRF here is to ensure unlinkability on the one hand, while on the other hand still linking by a common pseudonym all signatures created by the same user in the same context.

In our construction, we employ the Dodis-Yampolskiy verifiable random function (VRF) [Dodis and Yampolskiy(2005)], which we extend to Type-III bilinear groups. Our asymmetric DY VRF produces the same classical DY VRF output in the target group with a proof of correctness in one source group; however, it additionally outputs the same proof in the other source group. Intuitively, asymmetric groups are needed to enable the use of ElGamal encryption which relies on Decisional-Diffie Hellman in the zero-knowledge proof that assures the correct composition of the two VRFs. [8]

These two values, which have the form $(g^{1/(s+\mathsf{isk})}, \hat{g}^{1/(s+\mathsf{isk})})$, serve as the user secret key in SASSI, generated distributively by the issuers who share the secret key isk and receive a sharing of the user's identity string s.

The second primitive in our construction is inspired by the verifiable unpredictable function (VUF) of [Gurkan et al.(2021)], which has the unique property that it employs a *group* element secret key. This is a key ingredient in our scheme, as the output of the first VRF becomes the secret key of the second. The VUF of [Gurkan et al.(2021)] leverages the determinism of BLS signatures [Boneh et al.(2004b)] and a Groth-Escala-style NIZK [Escala and Groth(2014)] proof of correctness with respect to the group element key. Indeed, the output of the VUF is the pairing $T = e(\mathsf{h}(\mathsf{ctx}), \hat{\mathsf{usk}})$, where $\hat{\mathsf{usk}} = \hat{g}^{1/(s+\mathsf{isk})}$ is part of the user secret key, ctx is the context, and h is a hash function modeled as a random oracle. Our scheme has the same output, but we swap the Groth-Escala NIZK with a Sigma protocol, as Groth-Escala proofs do not support the zero-knowledge property for statements that involve the target group. Furthermore, we show in the random oracle model that $T = e(\mathsf{h}(\mathsf{ctx}), \hat{\mathsf{usk}})$ is not only unpredictable, but actually pseudorandom. It follows that the second primitive is also a VRF (essential for our privacy property).

In order to protect the user's identity string s, the user chooses a set of issuers and performs a verifiable secret-sharing of s. Subsequently, the issuers engage in a distributed VRF computation. As claimed in the original Dodis-Yampolskiy paper, it is easy to construct a distributed computation of the function $F_{\mathsf{isk}}(s) = g^{1/(s+\mathsf{isk})}$ when the issuers have shares of the secret isk, see Figure 2. Indeed, we realize it using standard techniques for multiparty computation via oblivious transfer (OT) [Even et al.(1985)] multiplication [Haitner et al.(2022)].

We provide a performance evaluation for our construction in Section 5.1; our results, cf. Figure 11 and Table 1, are highly competitive for real-world settings, with signature generation (inclusive of context switching if needed) requiring 10.42 ms, signature verification 11.24 ms, signature size 5.703 KB, and issuance with 150 out of 300 issuers at 73 sec. (Issuance needs to be executed for each

[8] We leave as open the question whether a provably secure alternative construction relying solely on symmetric pairings exists. This might be feasible using DLIN based encryption [Boneh et al.(2004a)]. In any case, both symmetric pairing groups and DLIN based encryption affect performance negatively in practice.

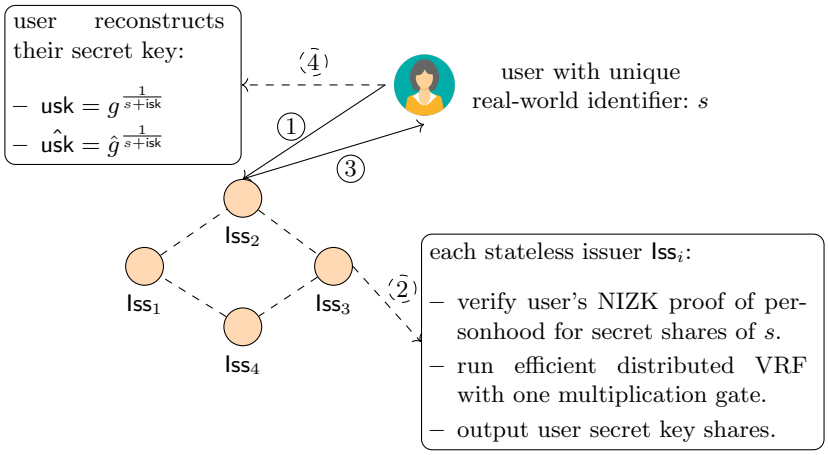

Fig. 2: Threshold issuance protocol. A user P with a unique real-world identifier s interacts with t (e.g., $t = 4$) issuers Iss_i, each holding a secret key isk_i, to generate a secret key pair $(\mathsf{usk}, \hat{\mathsf{usk}})$. The protocol begins with P verifiably secret-sharing s and proving its correctness via a zero-knowledge proof ①. The issuers after a bespoke multiparty computation ②, return the output to the user ③ who reconstructs the secret-key ④. No issuer learns s.

user only once.) We conclude our exposition with an overview of deployment considerations and applications for SyRA signatures. In particular, we describe how the personhood relation may be realized in an actual deployment using a variety of methods such as DECO [Zhang et al.(2020)], biometric authentication as in [Worldcoin(2023)], and OAuth as in [Baldimtsi et al.(2024)]. Finally, we overview how SyRA signatures apply to various use cases, such as e-voting, privacy-preserving regulatory compliance, cryptocurrency airdrops, and solving the "peer DID" objective.

Related work. Below we overview prior cryptographic primitives and their relation to SyRA signatures and our construction.

Group signatures. Group signatures [Chaum and van Heyst(1991)] allow a member of a group to anonymously sign messages on behalf of the group so that the group manager can determine the identity of any signer using a special trapdoor. Similar to group signatures, in *traceable* signatures, each group member has a tracing token that, if revealed, can link signatures from the same signer [Kiayias et al.(2004)]. In SyRA signatures, there is no opening (or tracing) authority; however, each signature incorporates an obfuscation of the user's identity string s in such a way that the user is unlinkable across different contexts, but linkable (while still being anonymous) within the same context.

Ring signatures. Ring signatures [Rivest et al.(2001)] allow a member of a group, called a ring, to anonymously sign messages on behalf of the group. Ring signatures are similar to group signatures; however, there is no group manager who creates keys and manages the group. Instead, the formation is ad hoc. Ring

signatures provide a strong notion of privacy: there is no mechanism for de-anonymizing users from the signatures they produce. A related concept of ring verifiable random functions has been explored in [Burdges et al.(2023)].

Identity-based signatures. Identity-based signatures (IBS) [Shamir(1984)] allow a user's signatures to be associated with a real-world identity string, such as an email address or phone number. User secret keys are generated by a master entity who ties them to identities using secret information. A *deterministic* identity-based signature scheme was proposed in [Herranz(2006)], i.e., signing the same message twice results in the same signature. This is achieved by employing a Schnorr signature for the key issuance phase, followed by a BLS signature for the signing phase, where the field element component of the Schnorr signature is used as the secret key for BLS. Closer to our aims, an IBS scheme where the key issuance phase is deterministic was proposed in [Hess(2003)]. Thus, a user cannot obtain multiple secret keys associated with their identity string. Here, a BLS signature is used for key issuance, followed by a custom signature algorithm for the signing phase that takes as input a group-element secret key. A key technical challenge in our work is combining deterministic signing at both levels, which we achieve via the Dodis-Yampolskiy VRF followed by a custom-built VRF with a group-element secret key. In a *hidden* identity-based signature scheme [Kiayias and Zhou(2007)], the user's identity string is not public information. SyRA signatures also facilitate hiding identities as in [Kiayias and Zhou(2007)], but offer, by design, linkability for multiple signatures in the same context.

Anonymous credentials. The concept of context-specific pseudonyms appears first in the anonymous credentials literature, beginning with the work of Chaum [Chaum(1985), Chaum and Evertse(1986)] (who also is the first to define, to the best of our knowledge, the objective of Sybil resilience), and followed by subsequent works, cf. [Camenisch and Lysyanskaya(2001)]. In this setting, users have distinct and unlinkable pseudonyms across organizations and have to engage with with each organization individually in order to have their pseudonym authorized—in principle multiple pseudonyms per user could be authorized even if this might be costly for the user. In SyRA signatures, on the other hand, users can generate their organizational (context-specific) credentials on their own, pseudorandomly extracting them from their real world identity. The Coconut framework [Sonnino et al.(2019)] introduces an anonymous credential construction that supports distributed threshold issuance. Within this framework, it is possible to achieve unlinkable selective attribute disclosures and manage both public and private attributes, even in scenarios where some issuing authorities may behave maliciously or be offline. However, Coconut does not provide guarantees for Sybil resilience.

Unclonable group identification. This line of work, beginning with [Damgård et al.(2006), Camenisch et al.(2006b)], covers a setting in which users are unlinkable across discrete time periods – a more limited setting compared to arbitrary context strings. In these constructions, the issuer verifies the real-world identity of the user and maps it to a credential that has the desired Sybil-resilient, context-specific property across periods. [Camenisch et al.(2006b)] builds upon

prior work on e-cash [Camenisch et al.(2005), Camenisch et al.(2006a)], with the addition of this type of context.

Domain-specific pseudonymous signatures (DSPS) and direct anonymous attestation (DAA), cf. [Bender et al.(2012), Bringer et al.(2014), Kutyłowski et al.(2016), Kluczniak et al.(2016), Błaśkiewicz et al.(2019), Brickell and Li(2010), Yang et al.(2021)]. These primitives allow users to generate pseudonyms for different contexts and create signatures on behalf of these pseudonyms. While it is possible to adapt and thresholdize these techniques as well as those of unclonable group identification to facilitate a SyRA construction, the resulting schemes would suffer from the credential issuer state problem: there is no immediately effective way to tie a secret key to a real-world identity (other than generic MPC that, e.g., evaluates a PRF "in the exponent"). A similar consideration applies to UTT [Tomescu et al.(2022)], a decentralized payment system based on anonymous credentials. In contrast, our construction SASSI only requires a special-purpose MPC with a single multiplication gate.

Linkable and traceable ring signatures. Linkable ring signatures [Liu et al.(2004)] allow for two signatures issued by the same signer with respect to the same ring of keys to be linked. *Traceable* ring signatures [Fujisaki and Suzuki(2007), Au et al.(2013)] generalize this notion to arbitrary contexts and allow for the recovery of the public key of the signer in case of signing twice in the same context. *One-time traceable* ring signatures [Scafuro and Zhang(2021)] facilitate the recovery of the public key of the signer in the linkable ring signature scenario. A similar type of functionality is achieved in the context of e-petitions in [Diaz et al.(2008)], where users can only sign a petition once. As mentioned earlier, in all these primitives, the connection between the user's secret key and their real-world identity is via an explicit mapping maintained by the issuer. While, in principle, such techniques could be used to produce a SyRA construction, they fundamentally rely on an issuer-maintained mapping of real-world identities to secret keys.

Threshold DY VRF. As mentioned, Dodis and Yampolskiy [Dodis and Yampolskiy(2005)] suggests a strategy for threshold implementation, which is explored further in [Dodis et al.(2006)] with security in the semi-honest setting. [Miao et al.(2020)] focuses on the n-out-of-n setting, while the general malicious setting is studied later in [Doerner et al.(2023)]. Our issuance protocol for SASSI suitably adapts these techniques to realize SyRA.

Legacy-compatible[9] *identification.* In this setting, put forth in [Zhang et al.(2020)] and utilized in the context of DIDs in CanDID [Maram et al.(2021)], users can prove a piece of information, which can be their real-world identity, from a "legacy" web service to a third-party provider. In the context of CanDID, such a provider (which is in fact a distributed entity) can issue a credential that the user can, on demand (albeit interactively with the issuer), transpose to any

[9] Legacy refers to a scheme that is not purpose-built for the task at hand but is preexisting. See, for example, [Maram et al.(2021)], where the same terminology is used for this purpose.

context he wishes, while remaining unlinkable; furthermore, Sybil resilience is ensured with respect to the user's real-world identity. The CanDID construction makes the only attempt (to the best of our knowledge) towards solving the issuer state problem: the real-world identity of the user is obfuscated in the form of its pseudorandom function (PRF) evaluation via a key that is shared between the issuing authorities. Each issuing authority keeps a copy of this obfuscated database and, hence, any attempt of the user to subvert Sybil resilience can be blocked by checking the PRF evaluation of their real-world identity against the obfuscated database. It is worth noting that this approach still has the downsides that each issuer needs to maintain a state linear in the number of credentials, and that users who lose their keys need additional infrastructure to help them recover them, as the issuers, in their effort to protect against Sybils, will deny re-issuance to any user who is unfortunate enough to lose their key. Our construction SASSI, on the other hand, circumvents both of these downsides by requiring no (updatable) state for the issuers and by requiring no interaction to introduce users to new contexts. Furthermore, it solves two open questions from [Zhang et al.(2020)], enforcing non-transferability (due to linkability of signatures in the same context), and providing unlinkability even if the adversary controls a minority of "committee members" (issuers in our setting).

zkLogin [Baldimtsi et al.(2024)] aims to enable users to authenticate on the blockchain using legacy credentials *without* managing blockchain-specific secret keys. To achieve this, zkLogin introduces a *salt service*, which adds high-entropy salt to low-entropy user attributes, deriving a zkLogin address while preserving the privacy of user attributes. zkLogin offers an alternative mode in which users manage the salt themselves. In this mode, deviating from zkLogin's main objective of removing the need to manage blockchain-specific secret keys, users must securely record the secret salt in order to access their on-chain funds. Importantly, for our purposes, in both modes (salt service and user-managed salt), zkLogin does not provide Sybil resilience. SyRA, unlike zkLogin which relies solely on OAuth, can interoperate with a variety of identity providers (including OAuth) as elaborated in Section 6. Proving the well-formedness of user addresses in zkLogin is not ZK-friendly, whereas SyRA defines user addresses algebraically, enabling more efficient proofs.

A similar approach is employed by ZK Address Abstraction [Park et al.(2023)], which uses zero-knowledge proofs over signed statements issued by providers based on user information to authenticate blockchain transactions. While this scheme protects users' private data, it also does not address Sybil resilience.

2 Preliminaries

Let $\lambda \in \mathbb{N}$ denote the security parameter and 1^λ its unary representation. Let p be a λ-bit prime. For all positive polynomials $f(\lambda)$, a function $\mathsf{negl} : \mathbb{N} \to \mathbb{R}^+$ is called *negligible* if $\exists\ \lambda_0 \in \mathbb{N}$ such that $\forall\ \lambda > \lambda_0$ it holds that $\mathsf{negl}(\lambda) < 1/f(\lambda)$. We denote by $\mathbb{G}^*$ the set $\mathbb{G} \setminus 1_\mathbb{G}$, where $1_\mathbb{G}$ is the identity element of the group $\mathbb{G}$. We denote the group of integers mod p by $\mathbb{Z}_p = \mathbb{Z}/p\mathbb{Z}$ and its multiplicative

group of units by $\mathbb{Z}_p^*$. We denote the set of integers $\{1, \ldots, n\}$ by $[1, n]$. Let $Y \xleftarrow{\$} F(X)$ denote running probabilistic algorithm F on input X and assigning its output to Y. Let $x \xleftarrow{\$} \mathbb{Z}_p$ denote sampling an element of $\mathbb{Z}_p$ uniformly at random. All algorithms are randomized unless expressly stated otherwise. PPT refers to probabilistic polynomial time.

Definition 1 (Bilinear group). *A bilinear group generator* $\mathsf{GrGen}(1^\lambda)$ *returns a tuple* $\mathsf{bp} \leftarrow (\mathbb{G}, \hat{\mathbb{G}}, \mathbb{G}_T, p, e, g, \hat{g})$ *such that* $\mathbb{G}$, $\hat{\mathbb{G}}$ *and* $\mathbb{G}_T$ *are finite groups of the same prime order* p, $g \in \mathbb{G}$ *and* $\hat{g} \in \hat{\mathbb{G}}$ *are generators, and* $e : \mathbb{G} \times \hat{\mathbb{G}} \to \mathbb{G}_T$ *is a bilinear pairing, which is efficiently computable and satisfies (1) non-degeneracy:* $e(g, \hat{g}) \neq 1_{\mathbb{G}_T}$ *and (2) bilinearity:* $\forall\ x, y \in \mathbb{Z}_p$, $e(g^x, \hat{g}^y) = e(g, \hat{g})^{xy} = e(g^y, \hat{g}^x)$.

We rely on Type-III, or asymmetric, bilinear groups $\mathbb{G}$ and $\hat{\mathbb{G}}$, for which there is no efficiently computable isomorphism between them, and two hardness assumptions defined over them: the Decisional Diffie-Hellman (DDH) assumption and the q-Decisional Bilinear Diffie-Hellman Inversion (q-DBDHI) assumption. Our construction also relies on Shamir secret sharing (SSH), the ElGamal public key encryption (PKE) scheme, and the Pedersen commitment scheme. See Appendix A for their definitions.

2.1 Verifiable random functions

A verifiable random function (VRF) is a function whose output is pseudorandom and for which a proof of correct evaluation is provided (i.e., a PRF with verifiability).

In our construction of a Sybil-resilient anonymous (SyRA) signature scheme, SASSI, we employ a VRF to compute user secret keys. Recall the Dodis-Yampolskiy VRF in Figure 3 [Dodis and Yampolskiy(2005)]. It is defined over a *symmetric* pairing $e : \mathbb{G} \times \mathbb{G} \to \mathbb{G}_T$, where the output is a value $F_{\mathsf{sk}}(\tau) = e(g, g)^{1/(\tau + \mathsf{sk})}$ in the target group, with corresponding proof $\pi_{\mathsf{vrf}} = g^{1/(\tau+\mathsf{sk})}$ in the source group. It is proven secure (i.e., pseudorandom) under the q-Decisional Bilinear Diffie-Hellman Inversion assumption (q-DBDHI) in [Dodis and Yampolskiy(2005)]: Given $(g, g^x, \ldots, g^{x^q})$, it is hard to distinguish $e(g, g)^{1/x}$ from random. The q powers are used to simulate VRF output queries in the security reduction. In our construction, we make use of the Dodis-Yampolskiy VRF in the *asymmetric* setting. In particular, user secret keys consist of a pair of group elements, one in each of the source groups, corresponding to a VRF proof of correctness in each group. We capture this in our VRF definitions (Definitions 4 and 5) by allowing a second proof $\hat{\pi}_{\mathsf{vrf}}$ to be output.[10] Pseudorandomness plays a key role in the security proof of our construction (specifically, in preserving user anonymity) while verifiability ensures that users can prove they have interacted with the VRF secret-key holder. Our asymmetric DY VRF is secure under the

[10] Our definitions are the standard VRF definitions where the proof consists of both elements $(\pi_{\mathsf{vrf}}, \hat{\pi}_{\mathsf{vrf}})$. We simply chose to present them in a way that is syntactically closer to our scheme, for greater clarity and self-containment.

q-Decisional Diffie-Hellman inversion assumption (q-DBDHI) over asymmetric pairings (see Appendix A).

Definition 2 (Verifiable random function (VRF)). [Dodis and Yampolskiy(2005)] *Let $\ell : \mathbb{N} \to \mathbb{N} \cup \{*\}$ and $\ell' : \mathbb{N} \to \mathbb{N}$ be any functions for which $\ell(\lambda)$ and $\ell'(\lambda)$ are computable in poly(λ) time (except when ℓ takes the value $*$ for an input of any length). A function family $F_{(\cdot)}(\cdot) : \{0,1\}^{\ell(k)} \to \{0,1\}^{\ell'(k)}$ is a family of VRFs if there exist polynomial-time algorithms* $(\mathsf{VRF.Setup}, \mathsf{VRF.Gen}, \mathsf{VRF.Prove}, \mathsf{VRF.Ver})$ *as follows:*

- $\mathsf{PP} \xleftarrow{\$} \mathsf{VRF.Setup}(1^\lambda)$ *: a PPT algorithm that takes as input the security parameter and outputs public parameters* PP*, which are implicitly given as input to all other algorithms.*
- $(\mathsf{vk}, \mathsf{sk}) \xleftarrow{\$} \mathsf{VRF.Gen}()$*: a PPT algorithm that returns a public verification key* vk *and secret key* sk*.*
- $(F_{\mathsf{sk}}(\tau), \pi_{\mathsf{vrf}}) \leftarrow \mathsf{VRF.Prove}(\mathsf{sk}, \tau)$*: an algorithm that takes as input a secret key and VRF input τ and returns a VRF output $F_{\mathsf{sk}}(\tau)$ and proof of correctness π_{vrf}.*
- $0/1 \leftarrow \mathsf{VRF.Ver}(\mathsf{vk}, \tau, T, \pi_{\mathsf{vrf}})$*: a deterministic algorithm that takes as input a verification key, VRF input τ, VRF output T, and proof π_{vrf} and verifies that $T = F_{\mathsf{sk}}(\tau)$ using the proof π_{vrf}, outputting 1 to indicate acceptance or 0 to indicate rejection.*

The main security properties of a VRF are correctness, uniqueness, and pseudorandomness.

Definition 3 (VRF correctness). *For all $\lambda \in \mathbb{N}$ and input values $\tau \in \{0,1\}^\lambda$,*

$$\Pr\big[\mathsf{PP} \leftarrow \mathsf{VRF.Setup}(1^\lambda); (\mathsf{vk}, \mathsf{sk}) \xleftarrow{\$} \mathsf{VRF.Gen}(); \\ (T, \pi_{\mathsf{vrf}}) \xleftarrow{\$} \mathsf{VRF.Prove}(\mathsf{sk}, \tau) : \mathsf{VRF.Ver}(\mathsf{vk}, \tau, T, \pi_{\mathsf{vrf}}) = 1\big] = 1$$

Definition 4 (VRF uniqueness). *For all PPT adversaries $\mathcal{A}$ playing game $\mathsf{Game}^{unique}_{\mathcal{A}}$, there exists a negligible function negl such that: $\mathsf{Adv}^{unique}_{\mathcal{A}}(\lambda) = \Pr[\mathsf{Game}^{unique}_{\mathcal{A}}(\lambda) = 1] \leq \mathsf{negl}(\lambda)$.*

The game $\mathsf{Game}^{unique}_{\mathcal{A}}(\lambda)$ (with optional second proof $\hat{\pi}_{\mathsf{vrf}}$) is defined as follows:

- $\mathsf{PP} \leftarrow \mathsf{VRF.Setup}(1^\lambda)$
- $(\mathsf{vk}^*, \tau^*, T_1^*, T_2^*, \pi^*_{\mathsf{vrf},1}, \pi^*_{\mathsf{vrf},2}, \boxed{\hat{\pi}^*_{\mathsf{vrf},1}}, \boxed{\hat{\pi}^*_{\mathsf{vrf},2}}) \xleftarrow{\$} \mathcal{A}()$
- **return** $(T_1^* \neq T_2^*) \wedge \mathsf{VRF.Ver}(\mathsf{vk}^*, \tau^*, T_1^*, \pi^*_{\mathsf{vrf},1}, \boxed{\hat{\pi}^*_{\mathsf{vrf},1}}) \wedge$ $\mathsf{VRF.Ver}(\mathsf{vk}^*, \tau^*, T_2^*, \pi^*_{\mathsf{vrf},2}, \boxed{\hat{\pi}^*_{\mathsf{vrf},2}})$

Definition 5 (VRF pseudorandomness). *For all PPT adversaries $\mathcal{A}$ playing game $\mathsf{Game}^{pseudo}_{\mathcal{A}}$, there exists a negligible function negl such that: $\mathsf{Adv}^{pseudo}_{\mathcal{A}}(\lambda) = \Pr[\mathsf{Game}^{pseudo}_{\mathcal{A}}(\lambda) = 1] \leq \mathsf{negl}(\lambda)$.*

The game $\mathsf{Game}^{pseudo}_{\mathcal{A}}(\lambda)$ (with optional second proof $\hat{\pi}_{\mathsf{vrf}}$) is defined as follows:

- $\mathbb{S} \leftarrow \emptyset$ // *query set*
- $\mathsf{PP} \leftarrow \mathsf{VRF.Setup}(1^\lambda)$
- $(\mathsf{vk}, \mathsf{sk}) \leftarrow \mathsf{VRF.Gen}()$
- $b' \xleftarrow{\$} \mathcal{A}^{\mathcal{O}^{\mathsf{Sign}}, \mathcal{O}^{\mathsf{Chal}}}(\mathsf{vk})$
- **return** $(b' = b)$

where the oracle $\mathcal{O}^{\mathsf{Sign}}(\tau)$ *and the oracle* $\mathcal{O}^{\mathsf{Chal}}(\tau^*)$ *(*called once*) are defined as follows:*

$\mathcal{O}^{\mathsf{Sign}}(\tau)$*:*

- **if** $\tau = \tau^*$ **return** $\perp$
- **else**
- $(T, \pi_{\mathsf{vrf}}, \boxed{\hat{\pi}_{\mathsf{vrf}}}) \leftarrow \mathsf{VRF.Prove}(\mathsf{sk}, \tau)$
- $\mathbb{S} \leftarrow \mathbb{S} \cup \{\tau\}$
- **return** $(T, \pi_{\mathsf{vrf}}, \boxed{\hat{\pi}_{\mathsf{vrf}}})$

$\mathcal{O}^{\mathsf{Chal}}(\tau^*)$*:*

- **if** $\tau^* \in \mathbb{S}$ **return** $\perp$
- $b \xleftarrow{\$} \{0,1\}$
- **if** $b = 0$, $T_0 \leftarrow F_{\mathsf{sk}}(\tau^*)$
- **if** $b = 1$, $T_1 \xleftarrow{\$} \{0,1\}^{\ell'(\lambda)}$
- **return** T_b

Our asymmetric Dodis-Yampolskiy VRF construction. Our asymmetric Dodis–Yampolskiy VRF ($\mathsf{DY\text{-}VRF}^\dagger$) construction is specified in Figure 4. $\mathsf{DY\text{-}VRF}^\dagger$ is distinguished from DY-VRF in a number of ways, e.g., proofs reside in both asymmetric source groups, its public key exists only in the second source group, and VRF verification checks three equalities in the target group.

Theorem 1. $\mathsf{DY\text{-}VRF}^\dagger$ *(Figure 4) satisfies uniqueness (Definition 4) and pseudorandomness (Definition 5) under the q-Decisional Bilinear Diffie-Hellman Inversion assumption (see Appendix A).*

Proof. This follows from the uniqueness and pseudorandomness of the DY VRF under the symmetric q-Decisional Bilinear Diffie-Hellman Inversion assumption (q-DBDHI) [Dodis and Yampolskiy(2005)].[11]

In the original Dodis-Yampolskiy reduction, the powers $(g, g^x, \ldots, g^{x^q})$ are used to simulate the output and VRF proof. The additional powers $(\hat{g}, \hat{g}^x, \ldots, \hat{g}^{x^q})$ in our construction are only needed to simulate the VRF proof $\hat{\pi}_{\mathsf{vrf}}$ in the other source group.

[11] [Dodis and Yampolskiy(2005)] restricts inputs to be of at most super-logarithmic size (in the security parameter) in order to prove security against adaptive adversaries. We also note that real-world identifiers, in the context of our work, are small e.g., SSN in the US.

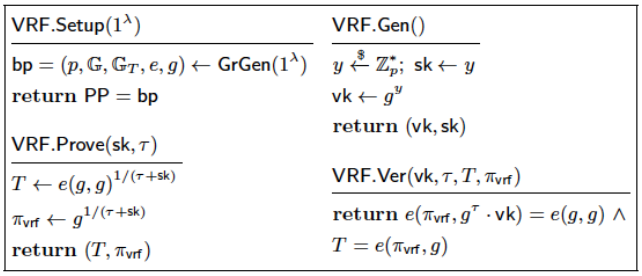

Fig. 3: The Dodis-Yampolskiy VRF [Dodis and Yampolskiy(2005)].

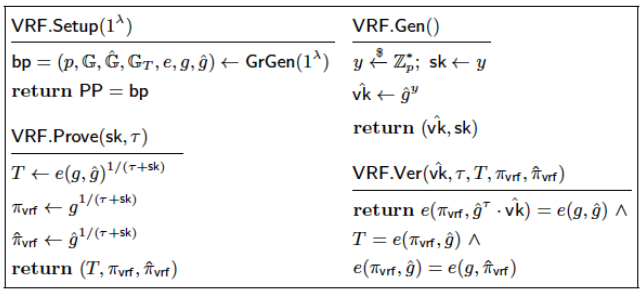

Fig. 4: Our asymmetric Dodis-Yampolskiy VRF ($\mathsf{DY\text{-}VRF}^\dagger$).

3 Formal modelling of SyRA signatures

We define Sybil-resilient anonymous (SyRA) signatures using an ideal functionality $\mathcal{F}_{\mathsf{SyRA}}$, which captures the desired security properties: threshold issuance, unforgeability, Sybil resilience, privacy-preservation, and unlinkability. The functionality $\mathcal{F}_{\mathsf{SyRA}}$ is parameterized by a threshold $t \leq n$ and n issuers denoted as $\mathcal{I} = (\mathsf{Iss}_1, \ldots, \mathsf{Iss}_n)$. The party identifiers of the issuers are incorporated into the session identifier $\mathsf{sid} = (\mathcal{I}, \mathsf{sid}')$. Initially, the state variable `Init` is set to 0. At the conclusion of the *key generation* phase, `Init` is updated to 1. For all other interfaces of the functionality (*threshold issuance*, *signature generation*, *verification*, and *leak signatures*) a check is performed at the beginning to ensure that `Init` $= 1$. If this condition is not satisfied, $\mathcal{F}_{\mathsf{SyRA}}$ disregards the received message.

Key generation. The functionality $\mathcal{F}_{\mathsf{SyRA}}$ first verifies that $\mathsf{sid} = (\mathcal{I}, \mathsf{sid}')$, ensuring that each *set* of n issuers can initialize their own instance of the functionality. Upon receiving the $(\mathtt{Init}, \mathsf{sid})$ message from all issuers specified in sid, $\mathcal{F}_{\mathsf{SyRA}}$ queries the adversary $\mathcal{A}$ for a verification key ivk, which is then recorded by $\mathcal{F}_{\mathsf{SyRA}}$.

Threshold issuance. Before a party P is granted the capability to sign messages, it must demonstrate ownership of *the personhood identity string* s. This verification is formalized through an abstract personhood relation $\mathcal{R}_s(x_s, (s, w_s))$.

The relation $\mathcal{R}_s$ operates on a public value x_s, which encapsulates publicly accessible information shared between the enrolling party P and each issuer $\mathsf{Iss}_i \in \mathbf{I}$ participating in the multiparty issuance process. The multiparty setup requires $\mathbf{I} \subset \mathcal{I}$, and $|\mathbf{I}| = t$, where t is the threshold number of issuers necessary to register a user. The enrolling party also has the private witness (s, w_s) corresponding to the identity string s to satisfy the relation $\mathcal{R}_s$.

The personhood relation $\mathcal{R}_s$ allows us to configure the functionality to support different identification processes. For instance, if users[12] are in possession of X.509 attribute certificates for their s value, the relation can be of the form $\mathcal{R}_s^{Gov}(x_s, (s, w_s))$ so that $x_s = (\mathsf{pk}_{Gov}, \mathsf{cert}_\mathsf{P}), w_s = \mathsf{sk}_\mathsf{P}$, $\mathsf{cert}_\mathsf{P} = (\mathsf{pk}_\mathsf{P}, s)$, and $\mathsf{Cert.Verify}(\mathsf{pk}_{Gov}, \mathsf{cert}_\mathsf{P}) = 1 \wedge \mathsf{pk}_\mathsf{P} = g^{\mathsf{sk}_\mathsf{P}}$. That is, the party P proves that they possess a certificate on s signed by the government for which they know their secret key. Note that it is crucial that the issuers confirm pk_{Gov} as part of their input; otherwise, the user could use certificates signed by arbitrary public keys.

It is straightforward to extend the above to a setting where various identity providers are recognized by the issuers. Another alternative is that the issuers and user run DECO [Zhang et al.(2020)], a mechanism for porting existing identities from websites (e.g., bank or government websites). Note also that x_s models the leakage of the certification process. If one were to, for example, model anonymous credentials instead of classical certificates, cert_P would become part of w_s, and x_s would not reveal any information about s. We intentionally leave any further details about $\mathcal{R}_s$ and the way it is implemented in the environment unspecified, so that our mechanism is compatible with a variety of different personhood instantiations.

The mapping $\mathbb{I}$ associates parties P with their corresponding identity strings s. Each honest party P is uniquely associated with a single s value, whereas malicious parties may be associated with multiple s values, provided that for each s, the relation $\mathcal{R}_s(x_s, (s, w_s))$ holds. If a malicious party submits an s value (for which $\mathcal{R}_s$ holds) that has already been recorded for an honest party, or if an honest party submits an s value that is already associated with a malicious party, the functionality $\mathcal{F}_{\mathsf{SyRA}}$ records s using the mapping $\hat{\mathcal{P}}(s)$. Here, $\hat{\mathcal{P}}(s)$ maps the identity string s to its associated party identifiers. In such cases, the party P is deemed a *compromised party*. When a party P invokes the *threshold issuance* interface, if $\mathsf{P} \in \hat{\mathcal{P}}(s)$ holds for the provided s, the functionality $\mathcal{F}_{\mathsf{SyRA}}$ does not offer anonymity guarantees for P.

Signature generation. Both honest and malicious parties can request signatures σ for a given context ctx and message m. Upon receiving a signature request, a new signature is generated by interacting with the adversary $\mathcal{A}$. During this process, only the identities of compromised parties are revealed, along with a value ucid, which serves as a unique label for each user-context pair. Recall that a party is classified as compromised, for instance, if its identity string s has been submitted to the issuance protocol of a malicious party. This scenario models the use of multiple devices belonging to the same user.

[12] The terms "user" and "party" are used interchangeably.

Upon receiving a response from the adversary, the functionality updates the set Sig to facilitate signature verification. The adversary submits the signature σ, the pseudonym T (against which Sybil is checked), and the identity strings s^* for malicious parties to $\mathcal{F}_{\mathsf{SyRA}}$. As the adversary may possess multiple identity strings, $\mathcal{F}_{\mathsf{SyRA}}$ records s^* for malicious parties. For honest parties, s^* is ignored because $\mathcal{F}_{\mathsf{SyRA}}$ already maintains a unique identity string s for each honest party. The functionality $\mathcal{F}_{\mathsf{SyRA}}$ verifies the submitted pseudonym T e.g., whether it is consistent with an already recorded pseudonym T' for the same verification key ivk, identity string s, and context ctx. To ensure consistency, $\mathcal{F}_{\mathsf{SyRA}}$ adds new records to the set Sig as necessary.

It might be possible to strengthen the functionality by having the adversary commit to the algorithm that issues signatures and pseudonyms for honest users in advance (and monitor this algorithm to ensure the desired properties). We leave the specification and realization of such a functionality as a direction for future work.

Verification. The verification interface enables external parties to validate signatures and retrieve the associated pseudonyms T. Notably, we do not assume the existence of a public-key infrastructure for issuer verification keys or the use of authenticated channels between verifiers and issuers. As a result, verifiers may inadvertently rely on incorrect issuer verification keys. For signatures generated using the verification keys of the session's designated issuers, the verification process leverages the records in Sig that were created during signature generation. For other verification keys, the adversary determines the verification outcome.

Regarding pseudonym T extraction, the interface allows external parties to submit signatures and retrieve the associated pseudonym, enabling parties, upon obtaining the pseudonym T, to query their local key-value store table to determine whether a tuple (pseudonym T, ctx) exists. By doing so, parties can efficiently detect all signatures generated by the same identity string s holder within the same context ctx. If the submitted signature is invalid with respect to ivk, the functionality $\mathcal{F}_{\mathsf{SyRA}}$ returns $\perp$ as the pseudonym. To ensure consistency, $\mathcal{F}_{\mathsf{SyRA}}$ maintains a record of previous verification and pseudonym extraction results in the set Ver.

Leak signatures. This is an adversarial interface for retrieving the signatures of users who are corrupted during the protocol (i.e., adaptively corrupted). It is an interesting question whether some of this leakage can be avoided by employing key-evolving VRFs [David et al.(2024)] to achieve forward secrecy. This question and the question of adaptive corruption in general is left for future work.

Functionality Sybil-resilient anonymous signatures $\mathcal{F}_{\mathsf{SyRA}}$

The functionality $\mathcal{F}_{\mathsf{SyRA}}$ is parameterized by a threshold $t \leq n$ and n issuers $\mathcal{I} = (\mathsf{Iss}_1, \ldots, \mathsf{Iss}_n)$. All sets and mappings are initialized as empty: $W = \mathbb{I}(\mathsf{P}) = \hat{\mathcal{P}}(s) = T = \mathbb{M} = \mathbb{D} = Sig = Ver = \emptyset$.

Key generation. Upon receiving the input $(\texttt{Init}, \mathsf{sid})$ from a party Iss_i: Parse $\mathsf{sid} = (\mathcal{I}, \mathsf{sid}')$. If $\mathsf{Iss}_i \notin \mathcal{I}$, ignore. Else, update $W \leftarrow W \cup \{\mathsf{Iss}_i\}$. Output $(\texttt{Init}, \mathsf{sid}, \mathsf{Iss}_i)$ to $\mathcal{A}$. Once $|W| = n$, and upon receiving $(\texttt{VerKey}, \mathsf{sid}, (\hat{\mathsf{ivk}}_i, \hat{\mathsf{ivk}}))$ from $\mathcal{A}$ for each $\mathsf{Iss}_i \in \mathcal{I}$: Output $(\texttt{VerKey}, \mathsf{sid}, (\hat{\mathsf{ivk}}_i, \hat{\mathsf{ivk}}))$ to each $\mathsf{Iss}_i \in \mathcal{I}$ via public-delayed output. Record $\hat{\mathsf{ivk}}$, and set $\texttt{Init} \leftarrow 1$.

Threshold issuance. (i) Upon receiving $(\texttt{Issue}, \mathsf{sid}, s, x_s, w_s, \mathbf{I})$ from some party P, and $(\texttt{Issue.Ok}, \mathsf{sid}, x_s)$ from issuer Iss_i, where $\mathsf{sid} = (\mathcal{I}, \mathsf{sid}')$, $|\mathbf{I}| = t$, $\mathbf{I} \subset \mathcal{I}$, and $\mathsf{Iss}_i \in \mathbf{I}$ act as follows.
Upon receiving $(\texttt{Issue.Ok}, \mathsf{sid}, x_s)$ from t issuers, ignore if $\mathcal{R}_s(x_s, (s, w_s)) \neq 1$ or if P is honest and there exists s' such that $\mathbb{I}(\mathsf{P}) = s'$ and $s' \neq s$. Else, act as follows:

1. If P is malicious, retrieve $\{\mathsf{P}_i\}_i$ values of honest parties for which $\{\mathbb{I}(\mathsf{P}_i) = s\}_i$ holds. Set $\hat{\mathcal{P}}(s) \leftarrow \{\mathsf{P}_i\}_i$.
2. Else (P is honest), if there exists a malicious party P' for which $s \in \mathbb{I}(\mathsf{P}')$ holds, set $\hat{\mathcal{P}}(s) \leftarrow \hat{\mathcal{P}}(s) \cup \mathsf{P}$.

Generate a fresh id. Set $T(\mathsf{id}) \leftarrow (\mathsf{P}, s)$. If $\mathsf{P} \in \hat{\mathcal{P}}(s)$, set $\mathcal{T} \leftarrow (\mathsf{P}, s, \hat{\mathcal{P}}(s), x_s, \mathbf{I})$. Else, set $\mathcal{T} \leftarrow (x_s, \mathbf{I})$. Hand $(\texttt{Issue}, \mathsf{sid}, \mathsf{id}, \mathcal{T})$ to $\mathcal{A}$. *// $\mathcal{F}_{\mathsf{SyRA}}$ guarantees privacy: the adversary learns neither s nor P.*
(ii) Upon receiving $(\texttt{Issue}, b, \mathsf{sid}, \mathsf{id})$ from $\mathcal{A}$: Ignore if $T(\mathsf{id}) = \perp$. Else, retrieve $T(\mathsf{id}) = (\mathsf{P}, s)$. If $b = 1$: set $\mathbb{I}(\mathsf{P}) \leftarrow \mathbb{I}(\mathsf{P}) \cup \{s\}$; and output $(\texttt{Issued}, \mathsf{sid})$ to P. Else ($b = 0$), output $(\texttt{Issue.Failed}, \mathsf{sid})$ to P.

Signature generation. (i) Upon receiving a value $(\texttt{Sign}, \mathsf{sid}, \mathsf{ctx}, m)$ from some party P: Ignore if $\mathbb{I}(\mathsf{P}) = \emptyset$. Else, if there exists a ucid where $\mathbb{M}(\mathsf{ucid}) = (\mathsf{P}, \mathsf{ctx})$, retrieve ucid. Else, generate a fresh ucid and set $\mathbb{M}(\mathsf{ucid}) \leftarrow (\mathsf{P}, \mathsf{ctx})$. If there exists an s value such that $\mathsf{P} \in \hat{\mathcal{P}}(s)$, set $\psi \leftarrow (\mathsf{ctx}, m, \mathsf{P})$. Else, set $\psi \leftarrow (\mathsf{ctx}, m)$. Select a fresh tid and set $\mathbb{D}(\mathsf{tid}) \leftarrow (\mathsf{P}, \mathsf{ctx}, m)$. Hand $(\texttt{Sign}, \mathsf{sid}, \mathsf{ucid}, \psi, \mathsf{tid})$ to $\mathcal{A}$. *// $\mathcal{F}_{\mathsf{SyRA}}$ guarantees privacy: the adversary learns neither s nor P.*
(ii) Upon receiving $(\texttt{Signature}, \mathsf{sid}, \sigma, T, s^*, \mathsf{tid})$ from $\mathcal{A}$: Ignore if $\mathbb{D}(\mathsf{tid}) = \perp$, or there exists an entry $(\cdot, \cdot, \cdot, \cdot, \sigma, \cdot, \cdot)$ in the set Sig. Else, retrieve $\mathbb{D}(\mathsf{tid}) = (\mathsf{P}, \mathsf{ctx}, m)$.

1. If $s^* \in \mathbb{I}(\mathsf{P})$ and P is malicious, set $s \leftarrow s^*$.
2. Else, if P is honest, retrieve $\mathbb{I}(\mathsf{P}) = \{s'\}$ and set $s \leftarrow s'$.
3. Else, ignore.

Ignore if:

1. there exists $(\mathsf{ivk}, s, \mathsf{ctx}, \cdot, \cdot, T', \cdot) \in Sig$ where $T' \neq T$ or, *// $\mathcal{F}_{\mathsf{SyRA}}$ guarantees Sybil-resilience: it enforces one T per s and ctx*
2. there exists $(\mathsf{ivk}, s, \mathsf{ctx}', \cdot, \cdot, T, \cdot) \in Sig$ where $\mathsf{ctx}' \neq \mathsf{ctx}$ or, *// $\mathcal{F}_{\mathsf{SyRA}}$ guarantees unlinkability: it enforces two pseudonyms per fixed s but different contexts*

3. there exists $(\mathsf{ivk}, s', \mathsf{ctx}, \cdot, \cdot, T, \cdot) \in \mathit{Sig}$ where $s' \neq s$. *// $\mathcal{F}_{\mathsf{SyRA}}$ enforces two distinct pseudonyms per fixed* ctx *but different identities*

Else, add $(\mathsf{ivk}, s, \mathsf{ctx}, m, \sigma, T, 1)$ to the set Sig. Output $(\texttt{Signature}, \mathsf{sid}, \mathsf{ctx}, m, \sigma)$ to P.

Verification. (i) Upon receiving a value $(\texttt{Ver}, \mathsf{sid}, \mathsf{ivk}', \mathsf{ctx}, m, \sigma)$ from some party V: Hand $(\texttt{Ver}, \mathsf{sid}, \mathsf{ivk}', \mathsf{ctx}, m, \sigma)$ to $\mathcal{A}$.
(ii) Upon receiving $(\texttt{Ver.Ok}, \mathsf{sid}, \mathsf{ivk}', \mathsf{ctx}, m, \sigma, T, \theta)$ from $\mathcal{A}$:

1. If $(\mathsf{ivk}', \cdot, \mathsf{ctx}, m, \sigma, T', \Theta') \in \mathit{Ver}$, set $\mathcal{T} \leftarrow T'$ and $\Theta \leftarrow \Theta'$.
2. Else, if $(\mathsf{ivk}, s, \mathsf{ctx}, m, \sigma, T', 1) \in \mathit{Sig}$ such that $\mathsf{ivk}' = \mathsf{ivk}$, record $(\mathsf{ivk}, s, \mathsf{ctx}, m, \sigma, T', 1)$ in the set Ver and set $\mathcal{T} \leftarrow T'$ and $\Theta \leftarrow 1$.
3. Else, if $(\mathsf{ivk}, \cdot, \mathsf{ctx}, m, \sigma, \cdot, 1) \notin \mathit{Sig}$ such that $\mathsf{ivk}' = \mathsf{ivk}$, record $(\mathsf{ivk}, \cdot, \mathsf{ctx}, m, \sigma, \perp, 0)$ in the set Ver and set $\mathcal{T} \leftarrow \perp$ and $\Theta \leftarrow 0$. *// $\mathcal{F}_{\mathsf{SyRA}}$ guarantees unforgeability: if* $\mathsf{ivk}' = \mathsf{ivk}$ *and the signer has never signed* (ctx, m)*, then the verification fails.*
4. Else, record $(\mathsf{ivk}', \cdot, \mathsf{ctx}, m, \sigma, T, \theta)$ in the set Ver and set $\mathcal{T} \leftarrow T$ and $\Theta \leftarrow \theta$. *// If none of the conditions above are met, the functionality allows the adversary to determine the values of $\mathcal{T}$ and Θ.*

Output $(\texttt{Ver.End}, \mathsf{sid}, \mathsf{ivk}', \mathsf{ctx}, m, \sigma, \mathcal{T}, \Theta)$ to V. *// $\mathcal{F}_{\mathsf{SyRA}}$ guarantees privacy: the adversary learns neither s nor* P.

Leak signatures. Upon receiving $(\texttt{Leak.Sig}, \mathsf{sid}, \mathsf{ivk}, s)$ from $\mathcal{A}$: If $\hat{\mathcal{P}}(s) = \emptyset$, set $\eta \leftarrow \perp$. Else, retrieve $\{(\mathsf{ivk}, s, \cdot, \cdot, \sigma_i, T_i, 1)\}_i$ from set Sig and set $\eta \leftarrow \{\sigma_i\}_i$. Hand $(\texttt{Leaked.Sig}, \mathsf{sid}, \mathsf{ivk}, \eta)$ to $\mathcal{A}$.

4 Our construction: SASSI

We describe our Sybil-resilient anonymous signature construction Π_{SASSI} and prove that Π_{SASSI} securely realizes $\mathcal{F}_{\mathsf{SyRA}}$. Our construction consists of two main parts: (i) issuance of each user's secret key as a deterministic function of their unique identity string s and the joint secret key of the issuers, and (ii) generation of Sybil-resilient anonymous signatures by users under the obtained secret keys.

Our threshold issuance enables multiple issuers, each holding a share of the signing key isk, to issue a key pair $(\mathsf{usk}, \hat{\mathsf{usk}})$ to each user P. Specifically, we consider a set of n issuers $\mathcal{I} = (\mathsf{Iss}_1, \ldots, \mathsf{Iss}_n)$, where each issuer Iss_i possesses a secret key isk_i received from the distributed key generation (DKG) functionality $\mathcal{F}_{\mathsf{DKG}}^{n,t}$. Each issuer's verification key $\hat{\mathsf{ivk}}_i$ is an element of the bilinear pairing's second source group $\hat{\mathbb{G}}$.

A user's secret key $(\mathsf{usk}, \hat{\mathsf{usk}})$ is computed as $(g^{1/(s+\mathsf{isk})}, \hat{g}^{1/(s+\mathsf{isk})})$. This representation consists of two Dodis-Yampolskiy VRF proofs [Dodis and Yampolskiy(2005)], one in each source group (Figure 4). As noted in the original paper, constructing a distributed computation of the function $f_{\mathsf{isk}}(s) = g^{1/(s+\mathsf{isk})}$ is

straightforward when the issuers possess shares of the secret isk. Our *threshold issuance* protocol, is a *distributed* evaluation of Dodis-Yampolskiy VRF proofs where no issuer sees the user's identity string s in plaintext, and collusion among t issuers would be required to compromise privacy. In this protocol, t out of n issuers, each holding isk_i and s_i, collaborate with the user to generate the user's secret key $(\mathsf{usk}, \hat{\mathsf{usk}})$.

Our construction uses three cryptographic primitives: Shamir secret sharing (Definition 6), ElGamal encryption (Definition 12), and our asymmetric Dodis-Yampolskiy verifiable random function $\mathsf{DY\text{-}VRF}^{\dagger}$ (Figure 4). Additionally, it relies on the following ideal sub-functionalities: non-interactive zero knowledge functionality $\mathcal{F}_{\mathsf{NIZK}}$ (Appendix B.1), sender-anonymous and secure communication channel functionality $\mathcal{F}_{\mathsf{Ch}}^{\mathsf{sas}}$ (Appendix B.2) [13], broadcast functionality $\mathcal{F}_{\mathsf{B}}$ (Appendix B.3), distributed key generation functionality $\mathcal{F}_{\mathsf{DKG}}^{n,t}$ (Appendix B.4), multiplication functionality $\mathcal{F}_{\mathsf{mul}}$ (Appendix B.5), and random oracle functionality $\mathcal{F}_{\mathsf{RO}}$ (Appendix B.6). The construction Π_{SASSI} is presented in four phases: *key generation, threshold issuance, signature generation*, and *verification* as follows.

Key generation. In this phase, each issuer Iss_i is activated by $\mathcal{Z}$, and whenever all n issuers are activated, the protocol generates secret signing key shares isk_i for each issuer, a set of n issuer public keys $\{\hat{\mathsf{ivk}}_k\}_{k=1}^{n}$, and a public verification key $\hat{\mathsf{ivk}} \in \hat{\mathbb{G}}$ as outputs of the DKG functionality $\mathcal{F}_{\mathsf{DKG}}^{n,t}$. (Note that Iss_i does not have a verification key in the first source group $\mathbb{G}$.) The protocol is parameterized by:

1. the description of a bilinear pairing group $\mathsf{bp} = (p, \mathbb{G}, \hat{\mathbb{G}}, \mathbb{G}_T, e, g, \hat{g})$,
2. the set of generators $(h, W, \hat{W})$, where:
   - $g, W, h \in \mathbb{G}$ are generators of $\mathbb{G}$ such that the discrete logarithm between any pair is unknown;
   - $\hat{g}, \hat{W} \in \hat{\mathbb{G}}$ are generators of $\hat{\mathbb{G}}$ such that the discrete logarithm between them is unknown,
3. public parameters $\mathsf{ivk} = (\mathsf{bp}, \hat{\mathsf{ivk}}, h, W, \hat{W})$.

The algorithm run by each issuer is provided in Figure 5.

1. **Input:** Upon receiving $(\texttt{Init}, \mathsf{sid})$ from $\mathcal{Z}$, act as follows.
2. Call $\mathcal{F}_{\mathsf{DKG}}^{n,t}$ [a] with $(\texttt{DKeyGen}, \mathsf{sid})$.
3. Upon receiving $(\texttt{Point}, \mathsf{sid}, \hat{\mathsf{ivk}}, r_i, \{\hat{\mathsf{ivk}}_k\}_{k=1}^{n})$ from $\mathcal{F}_{\mathsf{DKG}}^{n,t}$, set: $\mathsf{isk}_i \leftarrow r_i$, and $\hat{\mathsf{ivk}}_i \leftarrow \hat{g}^{\mathsf{isk}_i}$.
4. Record $(\mathsf{isk}_i, \hat{\mathsf{ivk}})$, and $\{\hat{\mathsf{ivk}}_i\}_{i=1}^{n}$.

[13] Note that a degree of sender anonymity is essential for maintaining privacy. Without this anonymity, "network leakage" could potentially expose the parties involved in a transaction, regardless of the robustness of cryptographic safeguards at the transactional level. Furthermore, in real-world deployment, some level of network leakage might be deemed acceptable. In such a scenario, our analysis would still be applicable, acknowledging that the adversary could compromise privacy through traffic analysis to some extent.

5. **Output:** Output $(\mathtt{VerKey}, \mathsf{sid}, (\hat{\mathsf{ivk}}_i, \hat{\mathsf{ivk}}))$ to $\mathcal{Z}$.

[a] Note that the functionality $\mathcal{F}_{\mathsf{DKG}}^{n,t}$ is parameterized by the second source group $\hat{\mathbb{G}}$ of the bilinear pairing.

Fig. 5: Key generation – Iss_i

Threshold issuance.
Threshold issuance is an interactive protocol between the party P, which has a unique real-world identifier s, and t issuers $\mathsf{Iss}_i \in \mathbf{I}$, $|\mathbf{I}| = t$, each holding a secret key isk_i. The protocol enables party P to generate its secret key pair $(\mathsf{usk}, \hat{\mathsf{usk}})$, corresponding to two VRF proofs, π_{vrf} and $\hat{\pi}_{\mathsf{vrf}}$, for $\mathsf{DY\text{-}VRF}^{\dagger}$. The protocol produces no output for the issuers. Party P is activated by the environment $\mathcal{Z}$, which provides an instruction specifying a set of t issuers $\mathbf{I}$ with whom P is to engage during the issuance protocol. If the environment has previously provided a different s to P in an earlier session, P disregards the current instruction.
To initiate the protocol, P verifiably secret-shares the value s received from $\mathcal{Z}$. This is accomplished by committing to the shares and broadcasting a vector of all t commitments to the issuers. Additionally, P generates a zero-knowledge proof showing that the committed shares are consistent with the original secret s, for which the personhood relation $\mathcal{R}_s(x_s, (s, w_s)) = 1$ holds. Once this proof is constructed, P transmits it along with the associated statement and the opening of the commitment to each issuer $\mathsf{Iss}_i \in \mathbf{I}$ via the channel $\mathcal{F}_{\mathsf{Ch}}^{\mathsf{sas}}$.
The algorithm executed by P is detailed in Figure 6.

1. **Input:** Upon receiving $(\mathtt{Issue}, \mathsf{sid}, s, x_s, w_s, \mathbf{I})$ from $\mathcal{Z}$, act as follows.
2. Parse $\mathsf{sid} = (\mathcal{I}, \mathsf{sid}')$, and ignore if: (i) $\mathbf{I} \not\subset \mathcal{I}$, or (ii) $|\mathbf{I}| \neq t$, or (iii) there exists a recorded s' such that $s' \neq s$, or (iv) $\mathcal{R}_s(x_s, (s, w_s)) \neq 1$. Else, record s as s'.
3. Call $\{s_i\}_{i=1}^{n} \xleftarrow{\$} \mathsf{SSH.Share}^{n,t}(s)$.
4. Choose a fresh identifier id.
5. $\forall i \in \mathrm{index}(\mathbf{I})$: set $\mathsf{com}_i \leftarrow g^{s_i} \cdot h^{z_i}$ for $z_i \xleftarrow{\$} \mathbb{Z}_p^*$.
6. Set $\mathsf{com}_{\mathbb{I}} \leftarrow \{\mathsf{com}_i\}_{\forall i \in \mathrm{index}(\mathbf{I})}$.
7. Call $\mathcal{F}_{\mathsf{BC}}$ with $(\mathtt{Broadcast}, \mathsf{sid} \,||\, \mathsf{id} \,||\, \mathbf{I}, \mathsf{com}_{\mathbb{I}})$.
8. Set NIZK statement $x \leftarrow (\mathsf{com}_{\mathbb{I}}, x_s)$.
9. Set NIZK witness $w \leftarrow (s, w_s, \{s_j\}_{j=1}^{n}, \{z_i\}_{\forall i \in \mathrm{index}(\mathbf{I})})$.
10. Call $\mathcal{F}_{\mathsf{NIZK}}$ with $(\mathtt{Prove}, \mathsf{sid}, x, w)$ for the relation $\mathcal{R}(x, w)$:
    (a) $\mathsf{com}_{\mathbb{I}} = \left\{\mathsf{com}_i = g^{s_i} \cdot h^{z_i}\right\}_{\forall i \in \mathrm{index}(\mathbf{I})}$
    (b) $\mathsf{SSH.Agg}^{n,t}\{s_i\}_{\forall i \in \mathrm{index}(\mathbf{I})} = s$
    (c) $\mathcal{R}_s(x_s, (s, w_s)) = 1$
11. Receive $(\mathsf{Proof}, \mathsf{sid}, \pi)$ from $\mathcal{F}_{\mathsf{NIZK}}$.
12. **Output:** Call $\mathcal{F}_{\mathsf{Ch}}^{\mathsf{sas}}$ with $(\mathtt{Send}, \mathsf{sid} \,||\, \mathsf{id} \,||\, \mathbf{I}, \mathsf{Iss}_i, (s_i, z_i, x, \pi))$ for each $\mathsf{Iss}_i \in \mathbf{I}$.

Fig. 6: Threshold issuance – P (1^{st} phase)

Each issuer Iss_i awaits activation through three distinct messages: from the network and from the environment $\mathcal{Z}$. The network signals activation by delivering the user's messages sent via $\mathcal{F}_{\mathsf{BC}}$ and $\mathcal{F}_{\mathsf{Ch}}^{\mathsf{sas}}$. The environment $\mathcal{Z}$ confirms the validity of the statement associated with the user's personhood proof x_s. Upon receiving these messages, the issuer Iss_i checks the correctness of the commitment and verifies the zero-knowledge proof π_i to ensure the well-formedness of s_i. Subsequently, each issuer Iss_i independently samples a random value r_i and provides it, together with the tuple (isk_i, s_i), and auxiliary information $\mathsf{aux_i}$ to $\mathcal{F}_{\mathsf{mul}}$.

The value $\mathsf{aux_i}$ contains the randomness of commitment z_i, and two vectors of size t: $(\{\hat{\mathsf{ivk}}_k\}_{k=1}^t, \mathsf{com}_{\mathbb{I}})$. $\mathcal{F}_{\mathsf{mul}}$ first checks the well-formedness of the input of each issuer. Specifically, it checks if $\hat{g}^{\mathsf{isk}_i}$ results in $\hat{\mathsf{ivk}}_i$ and if $g^{s_i} \cdot h^{z_i}$ results in com_i (see Section 5, *Implementing Multiplication* for details). If not, ignores the input. Else, ignores if not all $(\{\hat{\mathsf{ivk}}_k\}_{k=1}^t, \mathsf{com}_{\mathbb{I}})$ values provided by t issuers are the same. Else, computes and outputs K and $\hat{K}$. These are group elements whose exponents are the inverse of multiplication of r and $(\mathsf{isk} + s)$. Finally, the issuer Iss_i computes the values $\omega_i = K^{r_i}$ and $\hat{\omega}_i = \hat{K}^{r_i}$, which are then transmitted back to the user via the same communication channel from which the user's original message was received.

The algorithm run by each issuer Iss_i is provided in Figure 7.

1. **Input:** (i) Receive $(\texttt{Broadcasted}, \mathsf{sid} \;||\; \mathsf{id} \;||\; \mathbf{I}, \mathsf{com}_{\mathbb{I}})$ from $\mathcal{F}_{\mathsf{BC}}$, and (ii) $(\texttt{Received}, \mathsf{sid} \;||\; \mathsf{id} \;||\; \mathbf{I}, \mathsf{mid}, (s_i, z_i, x, \pi))$ from $\mathcal{F}_{\mathsf{Ch}}^{\mathsf{sas}}$, and (iii) $(\texttt{Issue.Ok}, \mathsf{sid}, x_s, \mathbf{I})$ from $\mathcal{Z}$ where $x = (\mathsf{com}_{\mathbb{I}}, x_s)$.
2. Ignore if: (i) $\mathsf{com}_i \neq g^{s_i} \cdot h^{z_i}$ for $\mathsf{com}_i \in \mathsf{com}_{\mathbb{I}}$ or (ii) $(\mathsf{isk}_i, \hat{\mathsf{ivk}})$ has not been recorded.
3. Else, call $\mathcal{F}_{\mathsf{NIZK}}$ with $(\texttt{Verify}, \mathsf{sid}, x, \pi)$ and receive $(\texttt{Verification}, \mathsf{sid}, b)$. Ignore if $b = 0$.
4. Else, randomly sample $r_i \xleftarrow{\$} \mathbb{Z}_p$.
5. Set $\mathsf{aux}_i \leftarrow (z_i, \{\hat{\mathsf{ivk}}_k\}_{k=1}^t, \mathsf{com}_{\mathbb{I}})$.
6. Call $\mathcal{F}_{\mathsf{mul}}$ with $(\texttt{Compute}, \mathsf{sid} \;||\; \mathsf{id} \;||\; \mathbf{I}, r_i, (\mathsf{isk}_i, s_i), \mathsf{aux}_i)$, and receive $(\texttt{Computed}, \mathsf{sid} \;||\; \mathsf{id} \;||\; \mathbf{I}, K, \hat{K})$ from $\mathcal{F}_{\mathsf{mul}}$ where $K = g^{(r \cdot (\mathsf{isk}+s))^{-1}}$ and $\hat{K} = \hat{g}^{(r \cdot (\mathsf{isk}+s))^{-1}}$.
7. Set: $\omega_i \leftarrow K^{r_i}$ and $\hat{\omega}_i \leftarrow \hat{K}^{r_i}$.
8. **Output:** Call $\mathcal{F}_{\mathsf{Ch}}^{\mathsf{sas}}$ with $(\texttt{SendBack}, \mathsf{sid}, \mathsf{mid}, (\omega_i, \hat{\omega}_i))$.

Fig. 7: Threshold issuance – Iss_i

Upon receiving t messages from t issuers, the user computes their secret keys by reconstructing shares. Next, the user runs two secret key well-formedness equality checks. If both equalities hold, the user records their secret keys and

outputs the issued message to the environment. Otherwise, it outputs failure. The algorithm run by P is provided in Figure 8.

1. Upon receiving $(\texttt{ReceivedBack}, \mathsf{sid}, \mathsf{lss}_i, (\omega_i, \hat{\omega}_i))$ from $\mathcal{F}_{\mathsf{Ch}}^{\mathsf{sas}}$ and from t issuers, call:
   - $\omega \leftarrow \mathsf{SSH.Agg}^{n,t}\{\omega_i\}_{\forall i \in \mathrm{index}(\mathbf{I})}$
   - $\hat{\omega} \leftarrow \mathsf{SSH.Agg}^{n,t}\{\hat{\omega}_i\}_{\forall i \in \mathrm{index}(\mathbf{I})}$.
2. Set: $\mathsf{usk} = g^{1/(s+\mathsf{isk})} \leftarrow \omega$ and $\hat{\mathsf{usk}} = \hat{g}^{1/(s+\mathsf{isk})} \leftarrow \hat{\omega}$
3. Ignore and send $(\texttt{Issue.Failed}, \mathsf{sid})$ to $\mathcal{Z}$ if: $e(\mathsf{usk}, \hat{g}) \neq e(g, \hat{\mathsf{usk}})$ or $e(\mathsf{usk}, \hat{\mathsf{ivk}} \cdot \hat{g}^s) \neq e(g, \hat{g})$.
4. Else, record $(\mathsf{usk}, \hat{\mathsf{usk}})$.
5. Output $(\texttt{Issued}, \mathsf{sid})$ to $\mathcal{Z}$.

Fig. 8: Threshold issuance – P (2^{nd} phase)

Signature generation. To sign messages, party P is activated by $\mathcal{Z}$ with a message $(\texttt{Sign}, \mathsf{sid}, \mathsf{ctx}, m)$. P, who has an identity string s, a secret key $(\mathsf{usk}, \hat{\mathsf{usk}})$ (derived from s), a context-message pair (ctx, m), and issuer verification key ivk, generates a signature σ on (ctx, m). The signer must prove the well-formedness of the pseudonym $T = e(Z, \hat{\mathsf{usk}})$ without revealing their secret key $\hat{\mathsf{usk}}$. This is accomplished by encrypting $\hat{\mathsf{usk}}$ under a public group element $\hat{W}$. Hence, instead of proving $T = e(Z, \hat{\mathsf{usk}})$, they prove $e(Z, \hat{W})^{\alpha} = e(Z, \hat{C}_2)/T$. Additionally, the signer needs to prove that the encrypted $\hat{\mathsf{usk}}$ is well-formed with respect to the issuer verification key ivk. To do so, the signer first proves that $\hat{\mathsf{usk}}$ is well-formed with respect to usk (by $e(C_2, \hat{g})e(g^{-1}, \hat{C}_2) = e(W, \hat{g})^{\beta} e(g^{-1}, \hat{W})^{\alpha}$, where usk is encrypted under a public group element W) and then proves the well-formedness of usk with respect to ivk (by $e(C_2, \hat{\mathsf{ivk}})e(g^{-1}, \hat{g}) = e(W, \hat{\mathsf{ivk}})^{\beta} e(W, \hat{g})^{\beta \cdot s} e(C_2, \hat{g})^{-s}$). Finally, the well-formedness of C_1 and $\hat{C}_1$ is proven. The algorithm run by P is provided in Figure 9.

1. **Input:** Upon receiving $(\texttt{Sign}, \mathsf{sid}, \mathsf{ctx}, m)$ from $\mathcal{Z}$, act as follows.
2. Ignore if $(\mathsf{ivk}, \mathsf{usk}, \hat{\mathsf{usk}})$ has not been recorded. Else, proceed as follows.
3. Call $\mathcal{F}_{\mathsf{RO}}$ with $(\texttt{Query}, \mathsf{sid}, \mathsf{ctx})$ and receive $(\texttt{Query.Re}, \mathsf{sid}, Z)$. Compute $T \leftarrow e(Z, \hat{\mathsf{usk}})$.
4. Pick $(\beta, \alpha) \xleftarrow{\$} \mathbb{Z}_p^*$. Parse $\mathsf{ivk} = (\mathsf{bp}, \hat{\mathsf{ivk}}, h, W, \hat{W})$. Compute:
   (a) $C = (C_1, C_2) \leftarrow (g^{\beta}, W^{\beta} \cdot \mathsf{usk})$
   (b) $\hat{C} = (\hat{C}_1, \hat{C}_2) \leftarrow (\hat{g}^{\alpha}, \hat{W}^{\alpha} \cdot \hat{\mathsf{usk}})$
5. Set NIZK statement $x \leftarrow (Z, C, \hat{C}, T, \mathsf{ivk}, \mathsf{ctx}, m)$.
6. Set NIZK witness $w \leftarrow (s, \alpha, \beta)$.
7. Call $\mathcal{F}_{\mathsf{NIZK}}$ with $(\texttt{Prove}, \mathsf{sid}, x, w)$ for the following relation $\mathcal{R}(x, w)$:
   (a) $e(Z, \hat{W})^{\alpha} = e(Z, \hat{C}_2)/T$
   (b) $e(C_2, \hat{g})e(g^{-1}, \hat{C}_2) = e(W, \hat{g})^{\beta} e(g^{-1}, \hat{W})^{\alpha}$

(c) $e(C_2, \hat{\mathsf{ivk}})e(g^{-1}, \hat{g}) = e(W, \hat{\mathsf{ivk}})^{\beta} e(W, \hat{g})^{\beta \cdot s} e(C_2, \hat{g})^{-s}$
(d) $C_1 = g^{\beta}$
(e) $\hat{C}_1 = \hat{g}^{\alpha}$
8. Upon receiving $(\mathsf{Proof}, \mathsf{sid}, \pi)$ from $\mathcal{F}_{\mathsf{NIZK}}$, set signature $\sigma \leftarrow (\pi, x)$.
9. **Output:** Output $(\mathtt{Signature}, \mathsf{sid}, \mathsf{ctx}, m, \sigma)$ to $\mathcal{Z}$.

Fig. 9: Signature generation – P

Verification. The verifier V is activated upon receiving a message $(\mathtt{Ver}, \mathsf{sid}, \mathsf{ivk}', \mathsf{ctx}, m, \sigma)$ from $\mathcal{Z}$ that includes the issuer verification key ivk', the context-message pair (ctx, m), and the signature σ. The signature is verified and the associated pseudonym T is extracted. The algorithm run by V is provided in Figure 10.

1. **Input:** Upon receiving $(\mathtt{Ver}, \mathsf{sid}, \mathsf{ivk}', \mathsf{ctx}, m, \sigma)$ from $\mathcal{Z}$, act as follows.
2. Parse $\sigma = (\pi, x)$ and $x = (Z, C, \hat{C}, T, \mathsf{ivk}, \mathsf{ctx}, m)$.
3. Set $b \leftarrow 0$ and $\eta \leftarrow \perp$ if:
   - $\mathsf{ivk} \neq \mathsf{ivk}'$; or
   - upon calling $\mathcal{F}_{\mathsf{RO}}$ with $(\mathtt{Query}, \mathsf{sid}, \mathsf{ctx})$, $(\mathtt{Query.Re}, \mathsf{sid}, Z')$ is returned where $Z' \neq Z$; or
   - upon calling $\mathcal{F}_{\mathsf{NIZK}}$ with $(\mathtt{Verify}, \mathsf{sid}, x, \pi)$, $(\mathtt{Verification}, \mathsf{sid}, 0)$ is returned.
4. Else, set $b \leftarrow 1$ and $\eta \leftarrow T$.
5. **Output:** Output $(\mathtt{Ver.End}, \mathsf{sid}, \mathsf{ivk}', \mathsf{ctx}, m, \sigma, \eta, b)$ to $\mathcal{Z}$.

Fig. 10: Verification – V

5 Implementation details and SyRA performance

Implementing non-interactive zero-knowledge (NIZK) proofs. In our signature framework, the relation $\mathcal{R}(x, w)$ that is proven by the signer in *signature generation* algorithm (Figure 9) is expressed as a set of algebraic discrete logarithm equations. Sigma protocols excel in prover efficiency when applied to such algebraic statements compared to zk-SNARKs [Groth(2010)]. Sigma protocols result in concise proof sizes, involve a limited number of operations, and do not necessitate the creation of a trusted common reference string [Guillou and Quisquater(1988), Schnorr(1991)]. This observation is particularly relevant in our system, where proof generation for algebraic statements could mainly depend on individuals with limited computational resources.

In the following, the prover and the verifier are denoted by Prv and Vrf, respectively. Prv proves the relation $\mathcal{R}(x, w)$ described in Figure 9. Let us define:

$$A := e(Z, \hat{W}), \quad B := \frac{e(Z, \hat{C}_2)}{T}, \quad \Gamma := e(C_2, \hat{g})e(g^{-1}, \hat{C}_2),$$
$$\Delta := e(W, \hat{g}), \quad E := e(g^{-1}, \hat{W}), \quad \Lambda := e(C_2, \hat{\mathsf{ivk}})e(g^{-1}, \hat{g}),$$
$$M := e(W, \hat{\mathsf{ivk}}), \quad \Theta := e(C_2, \hat{g}^{-1}).$$

Hence, the relation $\mathcal{R}(x, w)$ is simplified to the following one: (a) $A^{\alpha} = B$, (b) $\Gamma = \Delta^{\beta} E^{\alpha}$, (c) $\Lambda = M^{\beta} \Delta^{\beta \cdot s} \Theta^{s}$, (d) $C_1 = g^{\beta}$, (e) $\hat{C}_1 = \hat{g}^{\alpha}$ where bases $(A, B, \Gamma, \Delta, E, \Lambda, M, \Theta)$ are target group elements that can be efficiently computed by the verifier Vrf using the statement $x = (Z, C, \hat{C}, T, \mathsf{ivk})$. Moreover, all source group elements $(C_1, \hat{C}_1, g, \hat{g})$ are known to Vrf.
Therefore, the relation $\mathcal{R}(x, w)$ can be implemented through a proof of knowledge of discrete logarithms for items (a), (b), (d), and (e) above, and via a proof of a multiplicative relation for exponents for item (c) above. Below, we describe these relations and the methods for their implementation. These are interactive proofs that can be transformed into non-interactive proofs using the Fiat-Shamir transformation [Fiat and Shamir(1987)].[14]–[15]

Proof of knowledge of discrete logarithm $(\mathcal{R}(x, w) \Leftrightarrow y = g^x, x = (y, g), w = x)$

1. Prv**:** select $\theta \xleftarrow{\$} \mathbb{Z}_q^*$, compute $a \leftarrow g^{\theta}$, and send a to Vrf.
2. Vrf**:** select $c \xleftarrow{\$} \mathbb{Z}_q$ and send c to Prv.
3. Prv**:** compute $z = \theta + cx \bmod q$ and send z to Vrf.
4. Vrf**:** check if $a \stackrel{?}{=} g^z y^{-c}$. Accept if true.

Proof of multiplicative relation for exponents

- $x = (g, h, A_1, A_2, A_3)$, and $w = (a_1, a_2, a_3, r_1, r_2, r_3)$.
- $\mathcal{R}(x, w)$: $A_1 = g^{a_1} h^{r_1} \wedge A_2 = g^{a_2} h^{r_2} \wedge A_3 = g^{a_3} h^{r_3} = g^{a_1 a_2} h^{r_3} \wedge A_3 = A_1^{a_2} h^{r}$ such that $r + r_1 a_2 = r_3 \bmod q$.

1. Prv: for $i = 1, 2, 3$, select $\theta_i, R_i \xleftarrow{\$} \mathbb{Z}_q$ and compute $v_i = g^{\theta_i} h^{R_i}$, $v = A_1^{\theta_2} h^{R}$ for $R \xleftarrow{\$} \mathbb{Z}_q$. Send $(\{v_i\}, v)$ for $i = 1, 2, 3$ to Vrf.
2. Vrf: choose $c \xleftarrow{\$} \mathbb{Z}_{2^k}$ (where $2^k < q$) and send c to Prv.
3. Prv: compute $s_i = \theta_i - c a_i$, $t_i = R_i - c r_i$, $t = R - cr$ for $i = 1, 2, 3$. Send the tuple (s_i, t_i) for $i = 1, 2, 3$ and t to Vrf.

[14] The context-message pair (ctx, m), which is part of the NIZK statement, is included in the Fiat-Shamir hash function computation by both the signer and the verifier. This has been abstracted away by $\mathcal{F}_{\mathsf{NIZK}}$.

[15] Note that to realize $\mathcal{F}_{\mathsf{NIZK}}$, Fischlin's transform [Kondi and abhi shelat(2022)] can be employed, as in [Lysyanskaya and Rosenbloom(2022)]. Alternatively, one can construct a simulation-extractable zero-knowledge proof from a simulation-sound zero-knowledge proof and a CPA-secure encryption scheme, as in [Damgård et al.(2021)].

4. Vrf: check if $v_i \stackrel{?}{=} A_i^c g^{s_i} h^{t_i}$ for $i = 1, 2, 3$ and $v \stackrel{?}{=} A_3^c A_1^{s_2} h^t$. Accept if all equations hold.

Implementing multiplication. There are well-known techniques for realizing multiplication via multiparty computation (MPC). In our implementation, we adopt the multiplication in the exponent protocol proposed by Doerner et al. [Doerner et al.(2023)] for Threshold BBS+ signatures. See Appendix B.5 for more details on [Doerner et al.(2023)]. Their multi-party protocol at its core utilizes a two-party multiplication method, which is a two-round protocol based on Oblivious Transfer (OT) [Even et al.(1985)]. Since the protocol is built upon OT, it can be instantiated from various cryptographic assumptions, such as the hardness of the computational Diffie-Hellman problem. Note that one could alternatively employ the OT-based multiplier of Haitner et al. [Haitner et al.(2022)], which performs numerous scalar operations over elliptic curves per invocation. However, this approach may introduce a computational bottleneck [Doerner et al.(2023)]. The two-party multiplication between Iss_1 and Iss_2 proceeds as follows. Let $\mathcal{F}$ denote a functionality. Then: (i) Iss_1 submits its input $a \in \mathbb{Z}_p$ to $\mathcal{F}$; (ii) $\mathcal{F}$ samples $c \xleftarrow{\$} \mathbb{Z}_p$ uniformly at random, stores (a, c), and sends (Iss_1, c) to Iss_2; (iii) Iss_2 submits its input $b \in \mathbb{Z}_p$ to $\mathcal{F}$; and (iv) $\mathcal{F}$ computes $d \leftarrow a \cdot b - c \mod q$ and returns d to Iss_1. This is commonly known as Oblivious Linear Evaluation (OLE).
Our multi-party multiplication protocol is executed among t servers, each holding shares of variable a_i, a vector of two variables $\vec{b}_i = (b_i^{(1)}, b_i^{(2)})$, and some auxiliary information aux_i required to check the well-formedness of b_i. The output to all servers is the inverse of the multiplication of first two values in the exponent: $g^{(a \cdot b)^{-1}}$, where $b = (b^{(1)} + b^{(2)})$. We define $\mathcal{F}_{\mathsf{mul}}$ (Appendix B.5) as an augmentation to the protocol of [Doerner et al.(2023)], introducing a well-formedness check for $\vec{b}_i = (\mathsf{isk}_i, s_i)$ using $\mathsf{aux}_i = (z_i, \{\hat{\mathsf{ivk}}_k\}_{k=1}^t, \mathsf{com}_{\mathbb{I}})$. The functionality checks if: (i) $\mathsf{isk}_i \in \vec{b}_i$ is the associated secret key of $\hat{\mathsf{ivk}}_i \in \{\hat{\mathsf{ivk}}_k\}_{k=1}^t$, (ii) $s_i \in \vec{b}_i$ is the committed value in $\mathsf{com}_i \in \mathsf{com}_{\mathbb{I}}$, and (iii) if all issuers are providing the same vectors of public keys and commitments. This enhancement to [Doerner et al.(2023)] is straightforward to realize, as the well-formedness proof is algebraic, and the witness resides in the exponent space, where the base elements are public and known to all .

Implementing distributed key generation (DKG). In our formal modeling, we employ the t-out-of-n distributed key generation (DKG) ideal functionality $\mathcal{F}_{\mathsf{DKG}}^{n,t}$ in [Doerner et al.(2023), Katz(2024)]. In [Katz(2024)], Katz presents a series of two-round DKGs that securely realize $\mathcal{F}_{\mathsf{DKG}}^{n,t}$ and result in a uniform, unbiasable joint public key and guaranteed output delivery. These constructions rely on a broadcast channel, a synchronous network, and an honest majority $t_c < n/2$, where t_c is the number of corrupt parties. A number of other widely deployed [Dfns(2024), Fedimint(2024), Cothority(2024), Labs(2024)], efficient, UC-secure DKGs [Gennaro et al.(2007), Canetti et al.(2020)] are suitable for integration with SyRA signatures to generate issuer keys, with the Gennaro et

al. DKG [Gennaro et al.(2007)] also achieving full simulatability and G.O.D. in the formalism of [Katz(2024)].

5.1 Performance evaluation

The performance evaluation of SyRA was conducted on a MacBook Pro M3 Max featuring a 16-core CPU and 48 GB of RAM, running macOS 15.1, and written in the Rust (v. 1.83). In our evaluations, we used the BLS12-381 elliptic curve [Company(2024)]. The system's performance metrics encompass phases of the threshold issuance protocol, signature generation, and verification.

Threshold issuance performance. The performance of the issuance protocol was evaluated under various configurations of threshold parameters, capturing issuer sets of varying sizes. The threshold for secret key reconstruction by the user was set to $n/2$. In this evaluation, issuers engaged in pairwise multiplicative operations. Figure 11 illustrates the performance of the threshold issuance protocol. The performance metrics presented in the diagram reflect the median times recorded after conducting 10 iterations for each configuration. The figure depicts the execution times for the threshold issuance protocol among issuers and share aggregation via the user across various configurations of issuer sets n and the required shares for aggregation $t = n/2$. These results are presented within an honest majority setting, where the adversary is assumed to corrupt at most $t_c = n/2 - 1$ participants. The x-axis represents issuer set configurations, while the y-axes denote time: milliseconds (blue, left) for the share aggregation, and seconds (red, right) for the threshold issuance.

Performance metrics for signature. The performance metrics for the signature scheme, including generation time, verification time, and signature size, are summarized in Table 1. Note that some of the target group elements in the signature generation phase can be precomputed once and reused, further improving efficiency.

Metric	Value
Signature generation time	10.42 ms
Signature verification time	11.24 ms
Signature size	5.703 KB

Table 1: Performance metrics for SASSI.

6 Deployment considerations and applications

In this section, we discuss deployment considerations and applications for SyRA signatures. In particular, we describe how recovery of keys can be handled, how

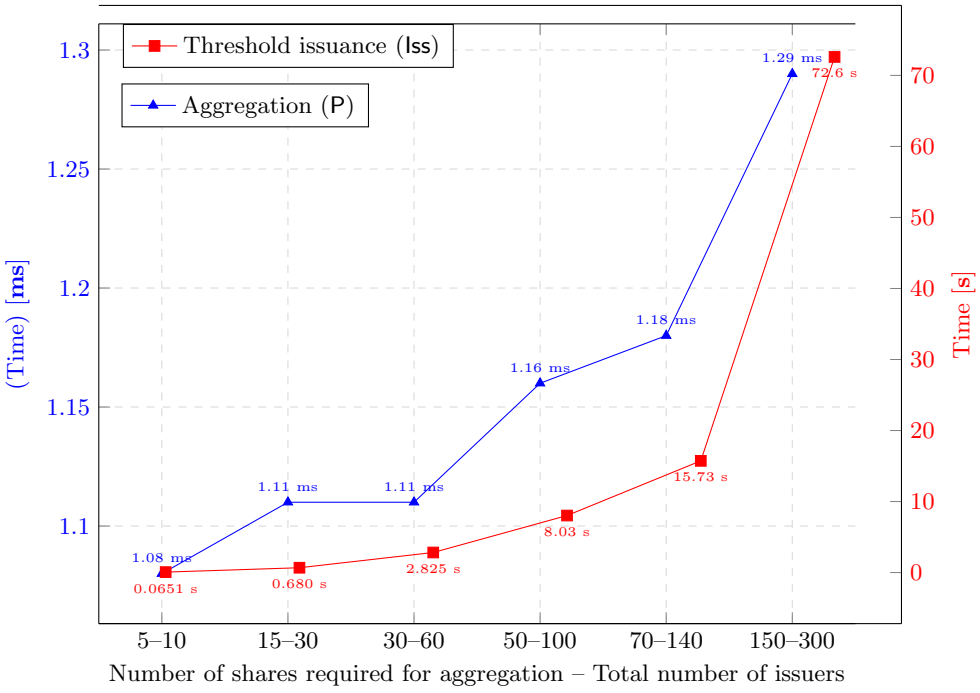

Fig. 11: Threshold issuance performance

the personhood relation may be realized in an actual deployment, how we can combine revocation and attribute-based credentials with SyRA, and how SyRA signatures can be used to solve various use cases, such as e-voting, privacy-preserving compliance, cryptocurrency airdrops, and peer DIDs.

Key recovery. Sybil resilience may complicate key recovery since a Sybil attacker can pretend that they have lost their key in order to be reissued a credential and break Sybil resilience. Various approaches have been suggested to mitigate this attack strategy in prior work, e.g., the key can be revoked using a credential revocation mechanism, e.g., [Baldimtsi et al.(2017)], and only then the user is reissued its new credential. Alternatively, the system can store the key information separately in a distributed fashion to facilitate key recovery directly, cf. [Maram et al.(2021)]. Our construction approach offers a strikingly simpler key recovery mechanism: A user simply reruns the issuing protocol with the issuers to recover their key, which is always the same for a given identity s. Hence, the system prevents a malicious user who attempts to reissue a DID as part of an attack that evades the system's abuse mitigation mechanisms.

Realizing personhood. One way to provide proof of personhood in our system is to use an oracle system like DECO [Zhang et al.(2020)] (or Town Crier [Zhang et al.(2016)]) that ensures data privacy and remains compatible with legacy systems without requiring alterations to data sources. DECO functions as a three-

party protocol involving a prover, a verifier, and a TLS server. Its purpose is to enable the prover to convince the verifier that a piece of data, which may potentially be private to the prover, retrieved from the server satisfies a specific predicate using zero-knowledge proofs without server-side modifications. In our specific context, the DECO verifier can either be the issuers themselves or parties designated as trustworthy by the issuers. Using this approach for personhood would require identifying a piece of information that is unique to individuals and possible to verify against a web service, for instance, a social security number (SSN) and a government web-site that includes the SSN as part of a TLS encrypted interaction. Another approach to establishing personhood is using biometric data — a recent embodiment of this idea that gained some traction is Worldcoin [Worldcoin(2023)]. It utilizes a biometric device that reads the iris of an individual and records information that can be used to subsequently prove the user's identity.
Another approach to realizing personhood, highlighted in [Baldimtsi et al.(2024)], is based on the OAuth 2.0 protocol through identity providers such as Google or Apple. Specifically, OpenID Connect [Sakimura et al.(2014)], an identity layer built on top of the OAuth 2.0 protocol, provides an ID Token in the form of a signed JSON Web Token (JWT), which contains key user information, such as a unique user identifier (`sub`) and the issuer (`iss`). `sub` provided by the OAuth provider could serve as the unique identifier s in our setting, where the user runs the protocol with the OAuth provider and the SyRA issuer. The JWT is used to authenticate the user personhood with each SyRA issuer participating in the issuance protocol. This can be achieved by verifying the JWT's signature using the OAuth provider's public key. Since JWTs may contain sensitive personal information, zero-knowledge proofs can be employed to conceal such details while still demonstrating the validity of the JWT.

Revocation. Incorporating revocation comprehensively with SyRA signatures is out of scope of this paper and an interesting direction for future work. Nevertheless, we outline here an initial set of ideas to illustrate how it is possible to add credential revocation in different scenarios. A simple method to incorporate revocation is by exploiting the secret key component $\hat{\mathsf{usk}}$ and utilize it in the context of a revocation list. Specifically, a revocation authority can maintain a list of $\hat{\mathsf{usk}}$ values associated with the identity strings and disclose publicly those that need to form the revocation list; alternatively, the issuers can reconstruct $\hat{\mathsf{usk}}$ on the spot given the identity string s of the user that should be revoked. Using this list, a verifier can locally compute $T_{\mathsf{rev}} = e(\mathsf{h}(\mathsf{ctx}), \hat{\mathsf{usk}})$ for each announced $\hat{\mathsf{usk}}$. Given this, the verifier can ignore all signatures with T' values such that $T' = T_{\mathsf{rev}}$. Note that despite disclosing $\hat{\mathsf{usk}}$, no one can sign on behalf of the associated user, as the signature $\sigma = (x, \pi)$ is a NIZK proof whose generation necessarily requires a witness for the relation $\mathcal{R}(x, w) = 1$ from Section 4. This means signature generation also requires usk, which cannot be derived from $\hat{\mathsf{usk}}$ because SyRA relies on Type-III (asymmetric) bilinear groups. In this way, by efficiently revealing only one group element, $\hat{\mathsf{usk}}$, it is possible for verifiers to identify all signatures of the revoked user upon receiving their signatures.

The above mechanism requires revealing $\widehat{\mathsf{usk}}$. In case the user's SyRA key is compromised, the user can provide s to the issuers along with the relevant proof of ownership of s, so that the issuers compute $\widehat{\mathsf{usk}}$. In this setting, the affected user would have to update their identity string s with a new one from the identity provider that is acknowledged by the personhood relation $\mathcal{R}_s$ (e.g., be issued a new social security number). Note that the identity provider needs to check that the old identity string has indeed been revoked. Furthermore, this will not provide forward security, at least in a non-interactive manner, since revealing the $\widehat{\mathsf{usk}}$ value of a user will link all its past pseudonyms. (Note that alternatively, verifiers could, in principle, run a multiparty protocol with issuers to check if an identity has been revoked instead of using $\widehat{\mathsf{usk}}$, but this is not a practical solution). Adding forward security to this revocation mechanism is an interesting direction for future work.

Identity attributes. SyRA signatures can be combined with anonymous attribute-based credentials to demonstrate user attributes in a privacy-preserving way while maintaining resilience against Sybil attacks. The approach is simple: users will obtain keys for both SyRA and anonymous attribute-based credentials. This allows subsequently to engage in attribute-based identification with each transcript signed using SyRA for the context requested by the verifier. While simple and effective, the above approach may be further improved in terms of efficiency by investigating a concrete design for attribute-based SyRA signatures; we leave this question for future work.

6.1 Applications

e-Voting and e-Petitions. There are many online services that require strict single access per ID, such as e-voting. SyRA enables verifiers to cluster all signatures with respect to their pseudonyms T for each context. Sybil resilience means that a user can create at most one pseudonym T per context. For instance, in e-voting or e-petitions [Diaz et al.(2008)], the election procedure identifier would be used to determine the context, and the choice of the voter would be the message. The user can sign multiple times for the same identifier (as is needed in many election schemes for coercion protection, for instance [Volkamer and Grimm(2006)]), and then the system can choose the latest "message." (Recall that all signatures of the user share the same T value.)

Privacy-preserving regulatory compliance. In the context of regulatory compliance, SyRA signatures provide a mechanism for verifying identities without exposing sensitive information. This capability is particularly relevant for financial institutions and other regulated entities that need to adhere to KYC (Know Your Customer), AML (Anti Money Laundering), and CFT (Combating Financing Terrorism) regulations. For example, a recipient may be asked to sign a transaction using SyRA to verify that they are not identified as a terrorist according to a list of terrorist identity strings. Such a list could be prepared by the issuers for a specific context of interest by calculating all the relevant T-values; note that this also maintains the privacy of the list itself. Adding this requirement

to the user's workflow would ensure compliance with CFT regulations while enabling the recipient to receive funds without disclosing their identity. Similarly, the sender of a transaction can be required to provide a SyRA signature on the transaction to prove that they are not associated with any revoked entities listed in AML regulations, enabling them to send funds. SyRA signatures can also be used to facilitate AML/CFT checks in anonymous payment systems like [Sasson et al.(2014), Kiayias et al.(2022), Fauzi et al.(2019), Bünz et al.(2020), Sarencheh et al.(2025b), Sarencheh et al.(2025a)] following the mechanism described above, offering a privacy-preserving, yet compliant, solution for these platforms.

Cryptocurrency airdrops. In a cryptocurrency airdrop, a user is set to receive a certain amount of cryptocurrency. The user generates an address A; utilizing SyRA signatures, they can use the name of the airdrop event as the context and the address A as the message to be signed. The primitive enables the user to sign multiple addresses if needed or perform various actions in the context of one specific airdrop. Note that if a user signs the same context twice, those two signatures are linkable (they have the same T), however still, the privacy of the user's real-world identifier s is preserved and the user cannot be linked if they participate in multiple airdrops. Our mechanism enables the airdrop provider to write policies such as "each user receives exactly X coins," with the Sybil resilient property of SyRA being the enforcer of this property. [16]

Peer DID Decentralized identifiers are often anchored to a public source of truth, such as a blockchain. While this ensures the security and immutability of transactions, it allows arbitrary parties to resolve users' DIDs (i.e., obtain their DID documents). Users in a DID system should control every aspect of the use of their digital identity, including the ability to interact with other people, institutions, and organizations in a private manner. One solution, put forth by the Decentralized Identity Foundation [Foundation(2023)], is Peer DID[17], which moves the resolution of DIDs to local, peer-to-peer interactions off chain. In this system, a user Alice can create a pairwise DID for each interaction with a different user or organization. For example, Alice may interact with her doctor, bank, etc. using these context-specific "credentials." Our SyRA signatures are directly applicable here, as Alice can create pairwise DIDs from her master credential, which is tied to her real-world identity, via context-specific pseudonyms. The security properties of SyRA signatures ensure that Alice's identity is protected, and her interactions are unlinkable across all contexts in which she engages, while Sybil resilience protects the verifiers from interacting with multiple users controlled by Alice.

[16] As an example of the relevance of our approach to practice, consider that in the Arbitrum airdrop, around 253 million Arbitrum tokens, 21.8% of all tokens, were acquired by Sybil accounts [X-explore(2023)].

[17] https://identity.foundation/peer-did-method-spec/index.html

7 Security proof

In this section, we state our main theorem regarding the security of our protocol Π_{SASSI} (Section 4). Our functionality $\mathcal{F}_{\mathsf{SyRA}}$, defined in Section 3, is general and allows for adaptive corruptions; however, we prove our scheme secure in the static corruption model. Concretely, this corresponds to $\cup_s \hat{\mathcal{P}}(s) = \emptyset$ in our functionality, and thus the Leak interface is always inactive.

Theorem 2. *Let q_t be an upper bound on the number of signatures of all honest parties and let q_h be an upper bound on the number of queries the adversary can make to the random oracle. Under the* IND-CPA *security of ElGamal encryption, the pseudorandomness of our asymmetric Dodis-Yampolsky VRF (*DY-VRF†*), binding property of Pedersen commitments, and the* DDH *assumption, in the $\{\mathcal{F}_{\mathsf{NIZK}}, \mathcal{F}_{\mathsf{RO}}, \mathcal{F}_{\mathsf{Ch}}^{\mathsf{sas}}, \mathcal{F}_{\mathsf{B}}, \mathcal{F}_{\mathsf{DKG}}^{n,t}, \mathcal{F}_{\mathsf{mul}}\}$-hybrid model, no PPT environment $\mathcal{Z}$ with static corruptions of up to $t-1$ issuers can distinguish the real-world execution $\mathsf{EXEC}_{\Pi_{\mathsf{SASSI}},\mathcal{A},\mathcal{Z}}$ from the ideal-world execution $\mathsf{EXEC}_{\mathcal{F}_{\mathsf{SyRA}},\mathcal{S},\mathcal{Z}}$ in the presence of an arbitrary number of corrupted signers and verifiers that are all colluding with advantage greater than $\mathsf{Adv}^{bind}_{\mathcal{A},\mathsf{COM}}(\lambda) + 2q_t \cdot \mathsf{Adv}^{\mathsf{IND\text{-}CPA}}_{\mathcal{A}}(\lambda) + (q_t+1) \cdot \mathsf{Adv}^{pseudo}_{\mathcal{A},\mathsf{DY\text{-}VRF}^\dagger}(\lambda) + q_t \cdot q_h \cdot \mathsf{Adv}^{\mathsf{DDH}}_{\mathcal{A}}(\lambda)$.*

We now prove Theorem 2.

7.1 Simulation

We refer to the real-world protocol as Π_{SASSI} and the adversary as $\mathcal{A}$. The simulator $\mathcal{S}$ is described in detail below. It ensures that the view of the real-world execution $\mathsf{EXEC}_{\Pi_{\mathsf{SASSI}},\mathcal{A},\mathcal{Z}}$ and the ideal-world execution $\mathsf{EXEC}_{\mathcal{F}_{\mathsf{SyRA}},\mathcal{S},\mathcal{Z}}$ are indistinguishable for any PPT environment $\mathcal{Z}$. The session identifier sid is selected by the environment $\mathcal{Z}$. Our general proof strategy is for the simulator $\mathcal{S}$ to internally run an instance of Π_{SASSI} and the real-world adversary $\mathcal{A}$, except that $\mathcal{S}$ simulates the behaviour of honest users. The purpose of the simulator $\mathcal{S}$ is to ensure that the view of the internally-run adversary $\mathcal{A}$ (in the ideal world) cannot be distinguished from the view of the real-world adversary $\mathcal{A}$. $\mathcal{S}$ instructs the functionality to provide indistinguishable outputs at the interfaces of honest users.

The adversary $\mathcal{A}$ has the authority to instruct corrupted parties in an arbitrary manner, and the simulator $\mathcal{S}$ engages in interactions with $\mathcal{F}_{\mathsf{SyRA}}$ on behalf of these corrupted parties. In the ideal-world execution $\mathsf{EXEC}_{\mathcal{F}_{\mathsf{SyRA}},\mathcal{S},\mathcal{Z}}$, the honest (dummy) parties transmit their inputs from the environment $\mathcal{Z}$ directly to $\mathcal{F}_{\mathsf{SyRA}}$. We define a simulator $\mathcal{S}$ that reproduces the real-world view of the adversary $\mathcal{A}$ and simulates the actions of honest parties during the execution. The simulator $\mathcal{S}$ engages in interactions with both the dummy adversary $\mathcal{A}$ and the functionality $\mathcal{F}_{\mathsf{SyRA}}$. Similar to the functionality $\mathcal{F}_{\mathsf{SyRA}}$ and our protocol Π_{SASSI} for realizing it, the simulator's details are described in four parts: *Key generation, Threshold issuance, Signature generation*, and *Verification*. As $\cup_s \hat{\mathcal{P}}(s) = \emptyset$, we do not consider the *Leak signatures* interface ($(\mathtt{Leaked.Sig}, \mathsf{sid}, \mathsf{ivk}, \perp)$ is

always sent to the simulator). The simulator $\mathcal{S}$ emulates the functionalities $\mathcal{F}_{\mathsf{NIZK}}, \mathcal{F}_{\mathsf{RO}}, \mathcal{F}_{\mathsf{Ch}}^{\mathsf{sas}}, \mathcal{F}_{\mathsf{B}}, \mathcal{F}_{\mathsf{DKG}}^{n,t}$ and $\mathcal{F}_{\mathsf{mul}}$. To achieve this, it must maintain specific lists associated with each functionality. However, for the sake of simplicity and without loss of generality, we assume that $\mathcal{S}$ keeps track of the states of these functionalities, but we do not explicitly discuss or address all these lists in detail. Without loss of generality, we assume in the following that the adversary corrupts $t-1$ issuers.
The simulator $\mathcal{S}$ maintains the following lists and sets, which are initialized to $\emptyset$:

- $\mathcal{L}_{\mathsf{CR}}$: list of corrupted-registered parties (who have been issued signing keys)
- $\mathbb{S}^{\mathsf{corrupted}}$: set of s values of corrupted parties
- $\mathcal{L}_{\mathsf{SS}}$: list of simulated signatures (by the simulator $\mathcal{S}$)
- $\mathbf{C}$: index set denoting the issuers corrupted by the adversary $\mathcal{A}$ where $|\mathbf{C}| = t-1$. We denote a malicious issuer by Iss^*.

Corruptions. At the beginning of the execution, the environment $\mathcal{Z}$ instructs the adversary $\mathcal{A}$ to corrupt specific parties by sending messages of the form $(\mathsf{Corrupt}, \mathsf{sid}, \mathsf{P})$, where P denotes a party identifier. All signers and verifiers, as well as up to $t-1$ issuers from the issuer set $\mathcal{I}$ can be corrupted. The simulator $\mathcal{S}$ observes these corruption messages and forwards them to $\mathcal{F}_{\mathsf{SyRA}}$ using the same format, $(\mathsf{Corrupt}, \mathsf{sid}, \mathsf{P})$, to indicate which parties have been corrupted.

Simulation of *key generation*
adversary corrupting $t-1$ issuers:

- For honest issuers (Iss_i): upon receiving $(\mathtt{Init}, \mathsf{sid}, \mathsf{Iss}_i)$ from $\mathcal{F}_{\mathsf{SyRA}}$, simulate the honest issuer by executing the algorithm provided in Figure 5. Emulating $\mathcal{F}_{\mathsf{DKG}}^{n,t}$ update the state of $\mathcal{F}_{\mathsf{DKG}}^{n,t}$, and leak $(\mathtt{DKeyGenReq}, \mathsf{sid}, \mathsf{Iss}_i)$ to $\mathcal{A}$.
- For malicious issuers (Iss_i^*): upon invoking $\mathcal{F}_{\mathsf{DKG}}^{n,t}$ with $(\mathtt{DKeyGen}, \mathsf{sid})$ via the adversary, emulate $\mathcal{F}_{\mathsf{DKG}}^{n,t}$ and update its state accordingly.
- Emulating $\mathcal{F}_{\mathsf{DKG}}^{n,t}$, upon receiving $(\mathtt{CPoints}, \mathsf{sid}, \{\mathsf{isk}_i^*\}_{i\in\mathbf{C}})$ from the adversary, proceed as follows if $(\mathtt{DKeyGen}, \mathsf{sid})$ has been recorded for all issuers $\mathsf{Iss}_i \in \mathcal{I}$ (including the honest emulated ones).
- Call $\mathcal{F}_{\mathsf{SyRA}}$ with $(\mathtt{Init}, \mathsf{sid})$ on behalf of all malicious issuers Iss_i^*, where $i \in \mathbf{C}$.
- Output $(\mathtt{PublicKeys}, \mathsf{sid}, \hat{\mathsf{ivk}}, \{\hat{\mathsf{ivk}}_i\}_{i=1}^n)$ to the adversary as the output of $\mathcal{F}_{\mathsf{DKG}}^{n,t}$.
- Output $(\mathtt{Point}, \mathsf{sid}, \hat{\mathsf{ivk}}, \mathsf{isk}_i^*, \{\hat{\mathsf{ivk}}_i\}_{i=1}^n)$ to the adversary for each malicious issuer Iss_i^*, where $i \in \mathbf{C}$.
- Output $(\mathtt{VerKey}, \mathsf{sid}, (\hat{\mathsf{ivk}}_i, \hat{\mathsf{ivk}}))$ to $\mathcal{F}_{\mathsf{SyRA}}$ for each issuer $\mathsf{Iss}_i \in \mathcal{I}$.

Simulation of *threshold issuance*

Recall, as discussed earlier, that without loss of generality, we assume the adversary corrupts $t-1$ issuers. We index the malicious issuers from 1 to $t-1$, and denote the associated set as $\mathbf{C}$. The honest issuer in the set of t issuers ($\mathbf{I}$) is denoted by Iss_t.
(i) honest party P and $t-1$ malicious issuers:

- Upon receiving $(\texttt{Issue}, \mathsf{sid}, \mathsf{id}, \mathcal{T})$ from $\mathcal{F}_{\mathsf{SyRA}}$, where $\mathcal{T} = (x_s, \mathbf{I})$, start emulating an honest party and the honest issuer $\mathsf{Iss}_t \in \mathbf{I}$.
- Choose a random s and compute $\{s_i\}_{i=1}^n \xleftarrow{\$} \mathsf{SSH.Share}^{n,t}(s)$.
- Choose a fresh identifier id.
- $\forall i \in \text{index}(\mathbf{I})$: set $\mathsf{com}_i \leftarrow g^{s_i} \cdot h^{z_i}$ for $z_i \xleftarrow{\$} \mathbb{Z}_p^*$.
- Set $\mathsf{com}_{\mathbb{I}} \leftarrow \{\mathsf{com}_i\}_{\forall i \in \text{index}(\mathbf{I})}$.
- Emulating $\mathcal{F}_{\mathsf{BC}}$, broadcast $(\texttt{Broadcast}, \mathsf{sid} \mid\mid \mathsf{id} \mid\mid \mathbf{I}, \mathsf{com}_{\mathbb{I}})$.
- Set $x \leftarrow (\mathsf{com}_{\mathbb{I}}, x_s)$ using $x_s \in \mathcal{T}$ received from $\mathcal{F}_{\mathsf{SyRA}}$.
- Emulating $\mathcal{F}_{\mathsf{NIZK}}$, compute $\pi \leftarrow \mathsf{SimProve}(x)$ and store (x, π).
- Emulating $\mathcal{F}_{\mathsf{Ch}}^{\mathsf{sas}}$, for each $\mathsf{Iss}_i \in \mathbf{I}$, leak $(\texttt{Send}, \mathsf{sid} \mid\mid \mathsf{id} \mid\mid \mathbf{I}, \mathsf{Iss}_i, \mathsf{mid}, |m|)$ to the dummy $\mathcal{A}$, where $m = (s_i, z_i, x, \pi)$.
- Emulating $\mathcal{F}_{\mathsf{Ch}}^{\mathsf{sas}}$, upon receiving $(\texttt{Ok}, \mathsf{sid} \mid\mid \mathsf{id} \mid\mid \mathbf{I}, \mathsf{mid})$ from $\mathcal{A}$, output $(\texttt{Received}, \mathsf{sid} \mid\mid \mathsf{id} \mid\mid \mathbf{I}, \mathsf{mid}, (s_i, z_i, x, \pi))$ to the adversary for each malicious issuer $\mathsf{Iss}_j \in \mathbf{I}, j \in \mathbf{C}$.
- Upon receiving a call from the adversary of the form $(\texttt{Verify}, \mathsf{sid}, x, \pi)$, emulate $\mathcal{F}_{\mathsf{NIZK}}$ and output $(\texttt{Verification}, \mathsf{sid}, 1)$.
- Emulating the honest issuer $\mathsf{Iss}_t \in \mathbf{I}$, sample $r_t \xleftarrow{\$} \mathbb{Z}_p$, and set $\mathsf{aux}_t \leftarrow (z_t, \{\hat{\mathsf{ivk}}_k\}_{k=1}^t, \mathsf{com}_{\mathbb{I}})$.
- Emulating $\mathcal{F}_{\mathsf{mul}}$ update its state.
- Emulating $\mathcal{F}_{\mathsf{mul}}$, after receiving messages from all malicious issuers $\mathsf{Iss}_j \in \mathbf{I}, j \in \mathbf{C}$ of the form $(\texttt{Compute}, \mathsf{sid} \mid\mid \mathsf{id} \mid\mid \mathbf{I}, r_j, (\mathsf{isk}_j, s_j), \mathsf{aux}_j)$, leak $(\texttt{Compute}, \mathsf{sid}, \alpha, \mathbf{I})$ to the adversary for a randomly chosen α.
- Emulating $\mathcal{F}_{\mathsf{mul}}$, upon receiving $(\texttt{Compute.Ok}, \mathsf{sid}, \alpha)$ from the adversary $\mathcal{A}$, output $(\texttt{Computed}, \mathsf{sid}, K, \hat{K})$ to each malicious issuer $\mathsf{Iss}_j \in \mathbf{I}, j \in \mathbf{C}$ (where K and $\hat{K}$ have been computed following the functionality of $\mathcal{F}_{\mathsf{mul}}$).
- Emulating the honest issuer $\mathsf{Iss}_t \in \mathbf{I}$, leak $(\texttt{SendBack}, \mathsf{sid} \mid\mid \mathsf{id} \mid\mid \mathbf{I}, \mathsf{Iss}_t, \mathsf{mid}, |m|)$ to the dummy $\mathcal{A}$, where $|m|$ is size of two group elements from two source groups.
- For each malicious issuer $\mathsf{Iss}_j \in \mathbf{I}, j \in \mathbf{C}$, leak $(\texttt{SendBack}, \mathsf{sid} \mid\mid \mathsf{id} \mid\mid \mathbf{I}, \mathsf{Iss}_j, \mathsf{mid}, |m|)$ to the dummy $\mathcal{A}$ whenever the adversary (on behalf of malicious issuer) calls $\mathcal{F}_{\mathsf{Ch}}^{\mathsf{sas}}$ with $(\texttt{SendBack}, \mathsf{sid}, \mathsf{mid}, (\omega_j, \hat{\omega}_j))$.
- Execute the honest party P algorithm provided in Figure 8. If either $e(\mathsf{usk}, \hat{g}) \neq e(g, \hat{\mathsf{usk}})$ or $e(\mathsf{usk}, \hat{\mathsf{ivk}} \cdot \hat{g}^s) \neq e(g, \hat{g})$, set $b \leftarrow 0$. Otherwise, set $b \leftarrow 1$.
- Submit $(\texttt{Issue}, b, \mathsf{sid}, \mathsf{id})$ to $\mathcal{F}_{\mathsf{SyRA}}$.

(ii) malicious party P and $t-1$ malicious issuers: We intentionally avoid describing the communication channel $\mathcal{F}_{\mathsf{Ch}}^{\mathsf{sas}}$ leakage regarding malicious issuers as it is similar to the case above.

- Emulating $\mathcal{F}_{\mathsf{Ch}}^{\mathsf{sas}}$, receive $(\texttt{Send}, \mathsf{sid} \mid\mid \mathsf{id} \mid\mid \mathbf{I}, \mathsf{Iss}_t, (s_i, z_i, x, \pi))$ whenever the adversary (on behalf of a malicious user P) calls $\mathcal{F}_{\mathsf{Ch}}^{\mathsf{sas}}$ and leak $(\texttt{Send}, \mathsf{sid} \mid\mid \mathsf{id} \mid\mid \mathbf{I}, \mathsf{Iss}_t, \mathsf{mid}, |m|)$ to the dummy $\mathcal{A}$ where $m = (s_i, z_i, x, \pi)$. Upon receiving $(\texttt{Ok}, \mathsf{sid} \mid\mid \mathsf{id} \mid\mid \mathbf{I}, \mathsf{mid})$ from $\mathcal{A}$, emulating the honest issuer Iss_t (who verifies the proof in the real-world) and emulating $\mathcal{F}_{\mathsf{NIZK}}$, extract the adversary's witness by running $w \leftarrow \mathsf{Extract}(x, \pi)$, where $w =$

$(s, w_s, \{s_j\}_{j=1}^{n}, \{z_i\}_{\forall i \in \text{index}(\mathbf{I})})$. Check if $(x, w) \in \mathcal{R}$ holds. If it does not hold, emulating honest issuer Iss_t abort.
Emulating $\mathcal{F}_{\mathsf{BC}}$, upon receiving $(\texttt{Broadcasted}, \mathsf{sid} \mid\mid \mathsf{id} \mid\mid \mathbf{I}, \mathsf{com}_{\mathbb{I}})$, set $c \leftarrow 1$ that is initially $c = 0$ if condition (2) in Figure 7 are satisfied.

- Emulating the honest issuer $\mathsf{Iss}_t \in \mathbf{I}$, sample $r_t \xleftarrow{\$} \mathbb{Z}_p$ and set $\mathsf{aux}_t \leftarrow (z_t, \{\hat{\mathsf{ivk}}_k\}_{k=1}^{t}, \mathsf{com}_{\mathbb{I}})$.
- Following the real-world algorithm for honest issuer update the state of $\mathcal{F}_{\mathsf{mul}}$.
- Emulating $\mathcal{F}_{\mathsf{mul}}$, after receiving messages from all malicious issuers $\mathsf{Iss}_j \in \mathbf{I}, j \in \mathbf{C}$ of the form $(\texttt{Compute}, \mathsf{sid} \mid\mid \mathsf{id} \mid\mid \mathbf{I}, r_j, (\mathsf{isk}_j, s_j), \mathsf{aux}_j)$, and if $c = 1$, leak $(\texttt{Compute}, \mathsf{sid}, \alpha, \mathbf{I})$ to the adversary for a randomly chosen α.
- Emulating $\mathcal{F}_{\mathsf{mul}}$, upon receiving $(\texttt{Compute.Ok}, \mathsf{sid}, \alpha)$ from the adversary $\mathcal{A}$, output $(\texttt{Computed}, \mathsf{sid}, K, \hat{K})$ to each malicious issuer $\mathsf{Iss}_j \in \mathbf{I}, j \in \mathbf{C}$ (where K and $\hat{K}$ have been computed following the functionality of $\mathcal{F}_{\mathsf{mul}}$).
- Submit $(\texttt{Issue}, \mathsf{sid}, s, x_s, w_s, \mathbf{I}^*)$ on behalf of the malicious party P to $\mathcal{F}_{\mathsf{SyRA}}$, where x_s is retrieved from $x_i = (s_i, x_s)$ and $\mathbf{I}^* = \{\mathsf{Iss}_j\}_{j \in \mathbf{C}} \cup \{\mathsf{Iss}_t\}$.
- Compute $\{\omega_j\}_{j \in \mathbf{C}} \leftarrow K^{\{r_j\}_{j \in \mathbf{C}}}$ and $\{\hat{\omega}_j\}_{j \in \mathbf{C}} \leftarrow \hat{K}^{\{r_j\}_{j \in \mathbf{C}}}$.
- Execute the algorithm provided in Figure 8. If either $e(\mathsf{usk}, \hat{g}) \neq e(g, \hat{\mathsf{usk}})$ or $e(\mathsf{usk}, \hat{\mathsf{ivk}} \cdot \hat{g}^s) \neq e(g, \hat{g})$, set $b \leftarrow 0$. Otherwise, set $b \leftarrow 1$.
- Submit $(\texttt{Issue}, b, \mathsf{sid}, \mathsf{id})$ to $\mathcal{F}_{\mathsf{SyRA}}$. upon receiving $(\texttt{Issue}, \mathsf{sid}, \mathsf{id}, \mathcal{T})$ from $\mathcal{F}_{\mathsf{SyRA}}$.

Simulation of *signature generation*
(i) honest Party P:

- Receive $(\texttt{Sign}, \mathsf{sid}, \mathsf{ucid}, \mathsf{ctx}, m, \mathsf{tid})$ from $\mathcal{F}_{\mathsf{SyRA}}$.
- Emulate $\mathcal{F}_{\mathsf{RO}}$ and retrieve Z.
- If an entry (ucid', T') exists where $\mathsf{ucid}' = \mathsf{ucid}$, set $T \leftarrow T'$. Otherwise, select $T \xleftarrow{\$} \mathbb{G}_T$, and record (ucid, T).
- Select $(\beta, \alpha) \xleftarrow{\$} \mathbb{Z}_p^*$, choose $A \xleftarrow{\$} \mathbb{G}$, and $\hat{B} \xleftarrow{\$} \hat{G}$.
- Compute $C \leftarrow (g^{\beta}, W^{\beta} \cdot A)$, and $\hat{C} \leftarrow (\hat{g}^{\alpha}, \hat{W}^{\alpha} \cdot \hat{B})$.
- Emulate $\mathcal{F}_{\mathsf{NIZK}}$ and execute $\pi \leftarrow \mathsf{SimProve}(x)$ where $x = (Z, C, \hat{C}, T, \mathsf{ivk}, \mathsf{ctx}, m)$, and record the pair (π, x).
- Set $\sigma \leftarrow (\pi, x)$ and update $\mathcal{L}_{\mathsf{SS}} \leftarrow \mathcal{L}_{\mathsf{SS}} \cup \{\sigma\}$.
- Submit $(\texttt{Signature}, \mathsf{sid}, \sigma, T, s^*, \mathsf{tid})$ to $\mathcal{F}_{\mathsf{SyRA}}$, where $s^* = 0$.

(ii) corrupted party P: Output to the environment $\mathcal{Z}$ whatever is produced internally by $\mathcal{A}$. Note that submitting a signature generation instruction to $\mathcal{F}_{\mathsf{SyRA}}$, on behalf of a corrupted P via $\mathcal{S}$, occurs later, when $\mathcal{S}$, who emulates an honest verifier V, receives a (valid) signature generated by a corrupted party.
It is important to note that a corrupted party may locally compute signatures. However, as long as no honest verifier has encountered any of these signatures, there is no need for any security guarantees to be invoked. In other words, as will be demonstrated, once $\mathcal{Z}$ submits a valid signature for verification, the simulator first checks whether the signature has already been simulated. If the signature

has not been simulated and is valid, the simulator should register it with the ideal functionality $\mathcal{F}_{\mathsf{SyRA}}$ (thus recording it in the set *Sig* by $\mathcal{F}_{\mathsf{SyRA}}$). However, if $\mathcal{Z}$ has not submitted the signature, no further action is necessary.

Simulation of *verification*
(i) honest verifier V:

- Receive $(\mathtt{Ver}, \mathsf{sid}, \mathsf{ivk}', \mathsf{ctx}, m, \sigma)$ from $\mathcal{F}_{\mathsf{SyRA}}$. If there exists an entry $(\mathtt{Ver.Ok}, \mathsf{sid}, \mathsf{ivk}', \mathsf{ctx}, m, \sigma, T, \theta)$, submit it to $\mathcal{F}_{\mathsf{SyRA}}$.
- Else, submit $(\mathtt{Ver.Ok}, \mathsf{sid}, \mathsf{ivk}', \mathsf{ctx}, m, \sigma, \bot, 0)$ to $\mathcal{F}_{\mathsf{SyRA}}$ and record it if $\mathsf{ivk}' \neq \mathsf{ivk}$. Else, act as follows:
- If there exists $\sigma' \in \mathcal{L}_{\mathsf{SS}}$ where $\sigma' = \sigma$, parse $\sigma = (\pi, x)$ and $x = (Z, C, \hat{C}, T, \mathsf{ivk}, \mathsf{ctx}, m)$. Submit $(\mathtt{Ver.Ok}, \mathsf{sid}, \mathsf{ivk}', \mathsf{ctx}, m, \sigma, T, 1)$ to $\mathcal{F}_{\mathsf{SyRA}}$ and record it.
- Else ($\sigma \notin \mathcal{L}_{\mathsf{SS}}$), act as follows:
  - Parse $\sigma = (\pi, x)$ and $x = (Z, C, \hat{C}, T, \mathsf{ivk}, \mathsf{ctx}, m)$. If in emulating $\mathcal{F}_{\mathsf{RO}}$ the recorded entry for context ctx is $(\mathsf{sid}, \mathsf{ctx}, h)$ where $h \neq Z$, submit $(\mathtt{Ver.Ok}, \mathsf{sid}, \mathsf{ivk}', \mathsf{ctx}, m, \sigma, \bot, 0)$ to $\mathcal{F}_{\mathsf{SyRA}}$ and record it.
  - Else, let $w \leftarrow \mathsf{Extract}(x, \pi)$ and check $(x, w) \in \mathcal{R}$ and if the relation does not hold, submit $(\mathtt{Ver.Ok}, \mathsf{sid}, \mathsf{ivk}', \mathsf{ctx}, m, \sigma, \bot, 0)$ to $\mathcal{F}_{\mathsf{SyRA}}$ and record it.
  - Else, parse $w = (s, \alpha, \beta)$.
    - If $s \in \mathbb{S}^{\mathsf{corrupted}}$, submit $(\mathtt{Sign}, \mathsf{sid}, \mathsf{ctx}, m)$ to $\mathcal{F}_{\mathsf{SyRA}}$ on behalf of P where $\mathsf{P} \in \mathcal{L}_{\mathsf{CR}}$ holds. (If there exist multiple such P values, pick one at random.)
      Upon receiving $(\mathtt{Sign}, \mathsf{sid}, \mathsf{ucid}, \mathsf{ctx}, m, \mathsf{tid})$ from $\mathcal{F}_{\mathsf{SyRA}}$, parse $\sigma = (\pi, x)$ and $x = (Z, C, \hat{C}, T, \mathsf{ivk}, \mathsf{ctx}, m)$, and submit $(\mathtt{Signature}, \mathsf{sid}, \sigma, T, s, \mathsf{tid})$.
      Finally, output $(\mathtt{Ver.Ok}, \mathsf{sid}, \mathsf{ivk}', \mathsf{ctx}, m, \sigma, T, 1)$ to $\mathcal{F}_{\mathsf{SyRA}}$ and record it.
    - Else, the simulation *fails*. We later show that the probability of $\mathcal{A}$ forging a valid signature on behalf of an honest party, or generating a valid signature for a new party that never existed before, is negligible. Also, consider the following two bad events: for every two signatures σ_1 and σ_2 with associated T_1 and T_2, (1) If $T_1 = T_2$ and $s_1 \neq s_2$, or (2) If $T_1 \neq T_2$, and $s_1 = s_2$. We will show that the probability of the these bad events is zero.

(ii) corrupted verifier V: Output to $\mathcal{Z}$ whatever $\mathcal{A}$ outputs.

7.2 Sequence of games and reductions

In a series of games, we demonstrate that the two random variables $\mathsf{EXEC}_{\Pi_{\mathsf{SASSI}}, \mathcal{A}, \mathcal{Z}}$ and $\mathsf{EXEC}_{\mathcal{F}_{\mathsf{SyRA}}, \mathcal{S}, \mathcal{Z}}$ are statistically close. We use the notation $\Pr[\mathsf{Game}^{(i)}(\lambda) = 1]$ to represent the likelihood that the environment $\mathcal{Z}$ produces an outcome of 1 in $\mathsf{Game}^{(i)}$. Each specific game $\mathsf{Game}^{(i)}$ is associated with its own ideal functionality $\mathcal{F}^{(i)}_{\mathsf{SyRA}}$ and simulator $\mathcal{S}^{(i)}$. We begin with the least secure

(most leaky) functionality $\mathcal{F}_{\mathsf{SyRA}}^{(0)}$ and corresponding simulator $\mathcal{S}^{(0)}$, and progressively move towards our primary functionality $\mathcal{F}_{\mathsf{SyRA}}$ and primary simulator $\mathcal{S}$. Let q_t denote the upper bound on the number of signatures of all honest parties and let q_h denote the upper bound on the number of queries the adversary can make to the random oracle, which are both polynomial in the security parameter λ.

Summary of games. First, we provide an overview of six distinct games $\mathsf{Game}^{(0)}, \ldots, \mathsf{Game}^{(6)}$, where $\mathsf{Game}^{(0)}$ corresponds to the real-world execution $\mathsf{EXEC}_{\Pi_{\mathsf{SASSI}},\mathcal{A},\mathcal{Z}}$ and $\mathsf{Game}^{(6)}$ corresponds to the ideal-world execution $\mathsf{EXEC}_{\mathcal{F}_{\mathsf{SyRA}},\mathcal{S},\mathcal{Z}}$.

$\underline{\mathsf{Game}^{(1)}}$: Same as $\mathsf{Game}^{(0)}$, except that $\mathsf{Game}^{(1)}$ additionally checks whether a flag is raised. The flag is raised when the adversary $\mathcal{A}$ outputs two commitments $\mathsf{com} \in x$ and $\mathsf{com}' \in x'$ where $\mathsf{com} = \mathsf{com}'$ but for different committed values $s \neq s'$. Any difference between $\mathsf{Game}^{(0)}$ and $\mathsf{Game}^{(1)}$ arises from a violation of the binding property of the underlying Pedersen commitment scheme. Accordingly, the distinguishing advantage of $\mathcal{Z}$ between $\mathsf{Game}^{(0)}$ and $\mathsf{Game}^{(1)}$ can be bounded by: $|\Pr\left[\mathsf{Game}^{(1)}(\lambda) = 1\right] - \Pr\left[\mathsf{Game}^{(0)}(\lambda) = 1\right]| \leq \mathsf{Adv}^{\mathrm{bind}}_{\mathcal{A},\mathsf{COM}}(\lambda)$.

$\underline{\mathsf{Game}^{(2)}}$: All secret key pairs $(\mathsf{usk}, \hat{\mathsf{usk}})$ of honest parties with corresponding ElGamal ciphertexts $(C, \hat{C})$ are changed to $(A, \hat{B})$, where $A \xleftarrow{\$} \mathbb{G}, \hat{B} \xleftarrow{\$} \hat{\mathbb{G}}$. All values $T = e(Z, \hat{\mathsf{usk}})$ remain the same, and all q_t signatures are with respect to $(A, \hat{B})$. Thus, under the $\mathsf{IND\text{-}CPA}$ security of ElGamal encryption, we show: $|\Pr[\mathsf{Game}^{(2)}(\lambda) = 1] - \Pr[\mathsf{Game}^{(0)}(\lambda) = 1]| \leq 2q_t \cdot \mathsf{Adv}^{\mathsf{IND\text{-}CPA}}_{\mathcal{A}}(\lambda)$.

$\underline{\mathsf{Game}^{(3)}}$: All pseudonyms $T = e(Z, \hat{\mathsf{usk}})$ of honest parties are changed $T = e(Z, \hat{D})$, where $\hat{D} \xleftarrow{\$} \hat{\mathbb{G}}$, and all q_t signatures are computed with these values. Note that in this game, the simulator consistently employs an identical random group element $\hat{D}$ for a given identity string s holder. In the reduction to the pseudorandomness of $\mathsf{DY\text{-}VRF}^{\dagger}$, the reduction uses the pseudo-random values received from the $\mathcal{O}^{\mathsf{Sign}}$ oracle of the challenger as the secret keys for malicious users, and uses the real or random output of $\mathcal{O}^{\mathsf{Chal}}$ to generate the pseudonym of an honest user, while faking the NIZK proof by emulating $\mathcal{F}_{\mathsf{NIZK}}$. The reduction does not need to know the honest user's secret key as it fakes the NIZK proof. Under the pseudorandomness of $\mathsf{DY\text{-}VRF}^{\dagger}$, we show: $|\Pr[\mathsf{Game}^{(3)}(\lambda) = 1] - \Pr[\mathsf{Game}^{(2)}(\lambda) = 1]| \leq q_t \cdot \mathsf{Adv}^{\mathrm{pseudo}}_{\mathcal{A},\mathsf{DY\text{-}VRF}^{\dagger}}(\lambda)$.

$\underline{\mathsf{Game}^{(4)}}$: If a particular identity string s holder is directed to sign distinct contexts, e.g. $T = (\mathsf{h}(\mathsf{ctx}), \hat{D})$ and $T' = (\mathsf{h}(\mathsf{ctx}'), \hat{D})$, the simulator substitutes distinct random group elements for these values, e.g., $T, T' \xleftarrow{\$} \mathbb{G}_T$. Under the DDH assumption, we show: $|\Pr[\mathsf{Game}^{(4)}(\lambda) = 1] - \Pr[\mathsf{Game}^{(3)}(\lambda) = 1]| \leq q_t \cdot q_h \cdot \mathsf{Adv}^{\mathsf{DDH}}_{\mathcal{A}}(\lambda)$.

$\underline{\mathsf{Game}^{(5)}}$: $\mathcal{F}_{\mathsf{SyRA}}^{(5)}$ prohibits $\mathcal{S}^{(5)}$ from submitting any message to $\mathcal{F}_{\mathsf{SyRA}}^{(5)}$ on behalf of the adversary $\mathcal{A}$ (corrupted party) who generates a valid signature for a new party (not belonging to the group of honest and corrupted parties) or generates a valid signature on behalf of an honest party. Under the pseudoran-

domness of DY-VRF†, we show: $|\Pr[\mathsf{Game}^{(5)}(\lambda)=1]-\Pr[\mathsf{Game}^{(4)}(\lambda)=1]| \leq \mathsf{Adv}^{\text{pseudo}}_{\mathcal{A},\mathsf{DY\text{-}VRF}^\dagger}(\lambda)$.

$\underline{\mathsf{Game}^{(6)}}$: $\mathcal{F}^{(6)}_{\mathsf{SyRA}}$ does not allow $\mathcal{S}^{(6)}$ to submit any message to $\mathcal{F}^{(6)}_{\mathsf{SyRA}}$ on behalf of the adversary $\mathcal{A}$ (corrupted party) who generates two valid signatures with one unique identifier s on a given context where $T_1 \neq T_2$, or generates two valid signatures using two identifiers $s_1 \neq s_2$ where $T_1 = T_2$. It is argued that the probability of such events is zero. Hence: $\Pr[\mathsf{Game}^{(6)}(\lambda)=1] = \Pr[\mathsf{Game}^{(5)}(\lambda)=1]$.

Details of games and associated reductions In this section, we provide a detailed description of each game along with the corresponding reductions employed in our security proof.

To enhance simplicity and avoid redundancy, we omit details such as channel leakages in the following games. These aspects have already been discussed in the simulator description in Section 7.1.

$\underline{\mathsf{Game}^{(0)}}$: In the initial game, $\mathcal{F}^{(0)}_{\mathsf{SyRA}}$ simply forwards all communications with $\mathcal{Z}$. The simulator $\mathcal{S}^{(0)}$ corresponds to the execution of the real-world protocol $\mathsf{EXEC}_{\Pi_{\mathsf{SASSI}},\mathcal{A},\mathcal{Z}}$.

$\underline{\mathsf{Game}^{(1)}}$: Same as $\mathsf{Game}^{(0)}$, except that $\mathsf{Game}^{(1)}$ additionally checks whether a flag is raised. The flag is raised when the adversary $\mathcal{A}$ outputs two commitments $\mathsf{com} \in x$ and $\mathsf{com}' \in x'$ where $\mathsf{com} = \mathsf{com}'$ but for different committed values $s \neq s'$. The associated statements and proofs are (x, π) and (x', π'), respectively. The simulator emulating $\mathcal{F}_{\mathsf{NIZK}}$ extracts the witnesses via $w \leftarrow \mathsf{Extract}(x, \pi)$ and $w' \leftarrow \mathsf{Extract}(x', \pi')$, and checks whether $s \in w \neq s' \in w'$. The flag is raised if $s \neq s'$. Any difference between $\mathsf{Game}^{(0)}$ and $\mathsf{Game}^{(1)}$ arises from a violation of the binding property of the underlying Pedersen commitment scheme. Accordingly, the distinguishing advantage of $\mathcal{Z}$ between $\mathsf{Game}^{(0)}$ and $\mathsf{Game}^{(1)}$ can be bounded by: $|\Pr\left[\mathsf{Game}^{(1)}(\lambda)=1\right] - \Pr\left[\mathsf{Game}^{(0)}(\lambda)=1\right]| \leq \mathsf{Adv}^{\text{bind}}_{\mathcal{A},\mathsf{COM}}(\lambda)$. (we omit a formal proof of this fact)

$\underline{\mathsf{Game}^{(2)}}$: This game is identical to $\mathsf{Game}^{(1)}$, except for the following modification: All honest parties' plaintexts $(\mathsf{usk}, \hat{\mathsf{usk}})$ within the encryptions $(C, \hat{C})$ are replaced with random group elements chosen uniformly from $\mathbb{G}$ and $\hat{\mathbb{G}}$, respectively. However, note that T values are still computed using real-world values.

We show that $\mathsf{Game}^{(2)}$ and $\mathsf{Game}^{(1)}$ are indistinguishable under the IND-CPA security of ElGamal encryption, which allows us to bound the probability that $\mathcal{Z}$ distinguishes $\mathsf{Game}^{(2)}$ from $\mathsf{Game}^{(1)}$.

We introduce a sequences of sub-games:

$$(\mathsf{Game}^{(1)}_0 := \mathsf{Game}^{(1)}, \ldots, \mathsf{Game}^{(1)}_{i-1}, \mathsf{Game}^{(1)}_i, \ldots, \mathsf{Game}^{(1)}_{2q_t} := \mathsf{Game}^{(2)})$$

where q_t denotes the upper bound on the number of all signatures of all honest parties. Defining $\mathsf{Game}^{(1)}_0 := \mathsf{Game}^{(1)}$, $\mathsf{Game}^{(1)}_1$ is the same as $\mathsf{Game}^{(1)}_0$, except

we change the plaintext of the first ciphertext of the first honest party from the real-world value to the ideal-world random group element. Finally, we do the same for the last (second) ciphertext of the last honest party such that in $\mathsf{Game}^{(1)}_{2q_t} := \mathsf{Game}^{(2)}$, all ciphertexts are generated from random group elements by $\mathcal{S}^{(1)}_{2q_t} = \mathcal{S}^{(2)}$. The reduction between $\mathsf{Game}^{(1)}_{i-1}$ and $\mathsf{Game}^{(1)}_{i}$ is described below, where any difference between them is upper bounded by $\mathsf{Adv}^{\mathsf{IND\text{-}CPA}}_{\mathcal{A}}(\lambda)$.

In leaky functionalities $\mathcal{F}^{(1)}_{\mathsf{SyRA},i}, 1 \leq i \leq 2q_t$, where $\mathcal{F}^{(1)}_{\mathsf{SyRA},1} := \mathcal{F}^{(1)}_{\mathsf{SyRA}}$ and $\mathcal{F}^{(1)}_{\mathsf{SyRA},2q_t} := \mathcal{F}^{(2)}_{\mathsf{SyRA}}$, the leaked message to the simulator in the `Issue` and `Sign` commands are $(\mathtt{Issue}, \mathsf{sid}, \mathsf{id}, \mathsf{P}, s, x_s, \mathbf{I})$ and $(\mathtt{Sign}, \mathsf{sid}, \mathsf{P}, \mathsf{ctx}, m, \mathsf{tid})$, respectively, for honest parties. Hence, the simulator knows (P, s) values for all honest parties. Later, in subsequent games, the associated functionalities become less leaky and closer to our main functionality $\mathcal{F}_{\mathsf{SyRA}}$.

Associated reduction between $\mathsf{Game}^{(1)}_{i-1}$ and $\mathsf{Game}^{(1)}_{i}$ ($\mathsf{IND\text{-}CPA}$ security of ElGamal encryption; for $1 \leq i \leq 2q_t$). If $\mathcal{Z}$ distinguishes $\mathsf{Game}^{(1)}_{i-1}$ and $\mathsf{Game}^{(1)}_{i}$, we can construct $\mathcal{A}'$ that breaks the $\mathsf{IND\text{-}CPA}$ security of ElGamal encryption used in our construction.

- For $1 \leq j \leq i-1$: all (real-world) plaintexts have already been substituted with (ideal-world) random values.
- For $i+1 \leq j \leq 2q_t$: all ciphertexts are created using real-world plaintexts.
- The challenger $\mathcal{C}$ of the $\mathsf{IND\text{-}CPA}$ game outputs pk to $\mathcal{A}'$.
- $\mathcal{A}'$ sets $W \leftarrow \mathsf{pk}$. For simplicity, we omit writing the details for $\hat{C}$, as it is similar to C. (In our construction, pk for ElGamal ciphertext C is $W \in \mathbb{G}$ and for $\hat{C}$ it is $\hat{W} \in \hat{\mathbb{G}}$.)
- The real-world plaintext value $g^{1/(s+\mathsf{isk})}$ (associated to $b = 0$ in the $\mathsf{IND\text{-}CPA}$ game) is changed to A in $\mathsf{Game}^{(1)}_{i}$, where $A \xleftarrow{\$} \mathbb{G}$ (which is the ideal-world value associated to $b = 1$ in the $\mathsf{IND\text{-}CPA}$ game).
- The challenger provides C_b to distinguisher $\mathcal{A}'$: for $b = 0$, C_0 encrypts $g^{1/(s+\mathsf{isk})}$, and for $b = 1$, C_1 encrypts A. Therefore, for $\mathcal{Z}$, $\mathsf{Game}^{(1)}_{i}$ is the same as running the real-world protocol with real-world value usk for an honest party, rather than a random group element. Hence, if $\mathcal{Z}$ distinguishes $\mathsf{Game}^{(1)}_{i-1}$ from $\mathsf{Game}^{(1)}_{i}$, $\mathcal{A}'$ uses this to win the $\mathsf{IND\text{-}CPA}$ game.

The next game hop, namely between $\mathsf{Game}^{(1)}_{i}$ and $\mathsf{Game}^{(1)}_{i+1}$, for the value $\hat{\mathsf{usk}}$ of honest parties encrypted to $\hat{C}$ is similar, so we omit the details.

Simulating honest party signature. $\mathcal{A}'$ simulates signatures σ of honest parties as follows:

- Emulating $\mathcal{F}_{\mathsf{NIZK}}$, call $\pi \leftarrow \mathsf{SimProve}(x)$ where $x = (Z, C, \hat{C}, T, \mathsf{ivk}, \mathsf{ctx}, m)$ and record (π, x).
- Set $\sigma \leftarrow (\pi, x)$ and $\mathcal{L}_{\mathsf{SS}} \leftarrow \mathcal{L}_{\mathsf{SS}} \cup \{\sigma\}$. Recall that $\mathcal{L}_{\mathsf{SS}}$ is the list of simulated signatures.
- Submit $(\mathtt{Signature}, \mathsf{sid}, \sigma, s^*, \mathsf{tid})$ to $\mathcal{F}^{(1)}_{\mathsf{SyRA},i}$ where $s^* = 0$.

Therefore, under the IND-CPA security of ElGamal encryption, the following inequality holds:

$$|\Pr[\mathsf{Game}^{(2)}(\lambda)=1]-\Pr[\mathsf{Game}^{(1)}(\lambda)=1]| \leq 2q_t \cdot \mathsf{Adv}^{\mathsf{IND\text{-}CPA}}_{\mathcal{A}}(\lambda)$$

$\underline{\mathsf{Game}^{(3)}}$: Same as $\mathsf{Game}^{(2)}$, except that in $\mathsf{Game}^{(3)}$, we change all tags $T = e(Z, \hat{\mathsf{usk}})$ of honest parties to tags of the form $T = e(Z, \hat{D})$ where $\hat{D} \xleftarrow{\$} \hat{\mathbb{G}}$. We highlight that in this game, for a fixed identifier s, the simulator always uses the *same* random group element $\hat{D}$. Hence, $\mathsf{Game}^{(3)}$ is the same as $\mathsf{Game}^{(2)}$, except $\mathcal{S}^{(3)}$ computes the T values using random group elements, rather than real-world $\hat{\mathsf{usk}}$ values, for all honest parties.

We show that $\mathsf{Game}^{(3)}$ and $\mathsf{Game}^{(2)}$ are indistinguishable under the pseudorandomness of DY-VRF†, which allows us to bound the probability that $\mathcal{Z}$ distinguishes $\mathsf{Game}^{(3)}$ from $\mathsf{Game}^{(2)}$.

We introduce a sequence of sub-games:

$$(\mathsf{Game}^{(2)}_0 := \mathsf{Game}^{(2)}, \ldots, \mathsf{Game}^{(2)}_{j-1}, \\ \mathsf{Game}^{(2)}_j, \ldots, \mathsf{Game}^{(2)}_{q_t} := \mathsf{Game}^{(3)})$$

(where q_t denotes the upper bound on the number of all signatures of all honest parties.) Define $\mathsf{Game}^{(2)}_0 := \mathsf{Game}^{(2)}$. $\mathsf{Game}^{(2)}_1$ is the same as $\mathsf{Game}^{(2)}_0$, except in $\mathsf{Game}^{(2)}_1$, we change the tag T from the real-world value $T = e(Z, \hat{\mathsf{usk}})$ to the ideal-world value $T = e(Z, \hat{D})$, where $\hat{D} \xleftarrow{\$} \hat{\mathbb{G}}$. Finally, we do the same for the last tag such that in $\mathsf{Game}^{(2)}_{q_t} := \mathsf{Game}^{(3)}$, all tags are generated using random group elements (instead of real-world values $\hat{\mathsf{usk}}$). The reduction between $\mathsf{Game}^{(2)}_{j-1}$ and $\mathsf{Game}^{(2)}_j$ is described below, where any difference between them is upper bounded by $\mathsf{Adv}^{\mathrm{pseudo}}_{\mathcal{A},\mathsf{DY\text{-}VRF}^{\dagger}}(\lambda)$.

In leaky functionalities $\mathcal{F}^{(2)}_{\mathsf{SyRA},j}, 1 \leq j \leq q_t$, where $\mathcal{F}^{(2)}_{\mathsf{SyRA},1} := \mathcal{F}^{(2)}_{\mathsf{SyRA}}$ and $\mathcal{F}^{(2)}_{\mathsf{SyRA},q_t} := \mathcal{F}^{(3)}_{\mathsf{SyRA}}$), the leaked message to the simulator in the `Issue` and `Sign` commands are $(\mathtt{Issue}, \mathsf{sid}, \mathsf{id}, \mathsf{P}, s, x_s, \mathbf{I})$ and $(\mathtt{Sign}, \mathsf{sid}, \mathsf{P}, \mathsf{ctx}, m, \mathsf{tid})$, respectively, for an honest party P.

Associated reduction between $\mathsf{Game}^{(2)}_{j-1}$ and $\mathsf{Game}^{(2)}_j$ (pseudorandomness of DY-VRF†; for $1 \leq j \leq q_t$). If $\mathcal{Z}$ distinguishes $\mathsf{Game}^{(2)}_{j-1}$ and $\mathsf{Game}^{(2)}_j$, we can construct $\mathcal{A}'$ that breaks the pseudorandomness of DY-VRF†.

High-level overview of the reduction. The reduction proceeds by constructing an adversary $\mathcal{A}'$ that leverages $\mathcal{Z}$ who is capable of distinguishing between $\mathsf{Game}^{(2)}_{j-1}$ and $\mathsf{Game}^{(2)}_j$, to break the pseudorandomness of DY-VRF† (see Definition 5). First, $\mathcal{A}'$ interacts with the challenger and uses its oracle access to simulate responses for the adversary $\mathcal{A}$. As described in Definition 5, the pseudorandomness game grants the adversary oracle access ($\mathcal{O}^{\mathsf{Sign}}$) to a signing function that outputs proofs. Specifically, $\mathcal{A}'$ uses this access to emulate the issuance of malicious user secret keys: it submits a malicious user identity s to the challenger and receives a

corresponding response, which it sets as the malicious user's secret key. Later, in the challenge phase, $\mathcal{A}'$ submits an identity s corresponding to an honest user. The challenger then returns a group element, which is either (a) a uniformly random element or (b) a pseudorandom element derived in a specific way (see Definition 5). $\mathcal{A}'$ uses this challenge to generate the honest user's pseudonym and sets random group elements as the plaintexts of ciphertexts $(C, \hat{C})$ while faking the NIZK proof via emulating $\mathcal{F}_{\mathsf{NIZK}}$.

- For $1 \leq k \leq j-1$: all (real-world) $\hat{\mathsf{usk}}$ values in tags $T = e(Z, \hat{\mathsf{usk}})$ have already been substituted with (ideal-world) random values $\hat{D} \xleftarrow{\$} \hat{\mathbb{G}}$, $T = e(Z, \hat{D})$.
- For $j+1 \leq k \leq q_t$: all tags are created using real-world $\hat{\mathsf{usk}}$ values.
- The challenger $\mathcal{C}$ of the pseudorandomness game outputs $\hat{\mathsf{IVK}}$ to $\mathcal{A}'$.
- $\mathcal{A}'$ sets $\hat{\mathsf{ivk}} \leftarrow \hat{\mathsf{IVK}}$.
- Program random oracle as: $Z_l = \mathsf{h}(m_{1l}) \leftarrow g^{r_l}$ for $r_l \xleftarrow{\$} \mathbb{Z}_p^*$.
- Upon receiving $(\mathtt{Sign}, \mathsf{sid}, \mathsf{P}, \mathsf{ctx}, m, \mathsf{tid})$ from $\mathcal{F}^{(2)}_{\mathsf{SyRA},j}$, $\mathcal{A}'$ retrieves the associated s value of P using the leaked message $(\mathtt{Issue}, \mathsf{sid}, \mathsf{id}, \mathsf{P}, s, x_s, \mathbf{I})$.
- If there exists an entry (s', r'_s) recorded where $s' = s$, set $\hat{D} \leftarrow \hat{g}^{r'_s}$. Else, pick $r_s \xleftarrow{\$} \mathbb{Z}_p^*$. Record (s, r_s), and set $\hat{D} \leftarrow \hat{g}^{r_s}$.
- The real-world tag value $T_{\mathsf{real}} = e(g, \hat{g}^{1/(s+\mathsf{isk})})^{r_l}$ (associated to $b = 0$ in the pseudorandomness game) is changed to the ideal-world value $T_{\mathsf{ideal}} = e(g, \hat{D})^{r_l}$ in $\mathsf{Game}^{(2)}_j$ (associated to $b = 1$ in the pseudorandomness game).
- $\mathcal{A}'$ provides s (where $s \notin \mathbb{S}^{\mathsf{corrupted}}$) to the challenger $\mathcal{C}$ (note that $s \notin \mathcal{Q}$; $\mathcal{Q}$ is a query list maintained by $\mathcal{C}$ which contains only s values of corrupted parties).
- Upon receiving the challenge target group element T_b from the challenger $\mathcal{C}$, the adversary $\mathcal{A}'$ proceeds as follows: for the case $b = 0$, $T_{\mathsf{real}} = T_0^{r_l}$; and for $b = 1$, $T_{\mathsf{ideal}} = T_1^{r_l}$. This is because as defined in Definition 5, $T_0 = e(g, \hat{g})^{1/(s+\mathsf{isk})}$ and T_1 is sampled randomly. $\mathcal{A}'$ sets $T_b^{r_l}$ as pseudonym T of honest user with identity string s (provided to the challenger as the challenge input) and emulating $\mathcal{F}_{\mathsf{NIZK}}$ computes $\pi \leftarrow \mathsf{SimProve}(x)$ for $T_b^{r_l} \in x$ and check that $\mathsf{Verify}(x, \pi) = 1$. If so, records the entry (x, π).
- If $\mathcal{Z}$ distinguishes $\mathsf{Game}^{(2)}_{j-1}$ (with $T_{\mathsf{real}} = T_b^{r_l}$ for $b = 0$) from $\mathsf{Game}^{(2)}_j$ (with $T_{\mathsf{ideal}} = T_b^{r_l}$ for $b = 1$), then $\mathcal{A}'$ uses this advantage to win the pseudorandomness game as the pseudonym of the honest user is derived using the challenge. Therefore, for $\mathcal{Z}$, $\mathsf{Game}^{(2)}_j$ is indistinguishable from the real-world protocol where the real-world $\hat{\mathsf{usk}}$ value is used in tag computation for an honest party, rather than a random group element from $\hat{\mathbb{G}}$.

Simulating honest issuer public key. $\mathcal{A}'$ simulates the public keys of honest issuers as follows without knowing their secret keys. Without loss of generality, we assume the index set $\mathbf{C} = \{1, \ldots, t-1\}$ corresponds to the malicious issuers, while the indices $\mathbf{H} = \{t, \ldots, n\}$ are assigned to the honest ones.

- Set $\hat{\mathsf{ivk}} \leftarrow \hat{\mathsf{IVK}}$, where $\hat{\mathsf{IVK}}$ is provided by the challenger of the DY-VRF† pseudorandomness game.

– Emulating $\mathcal{F}_{\mathsf{DKG}}^{n,t}$, upon receiving $(\texttt{CPoints}, \mathsf{sid}, \{\alpha_j\}_{j\in\mathbf{C}})$ from $\mathcal{A}$, compute the public keys of the malicious issuers as: $\hat{\mathsf{ivk}}_j \leftarrow \hat{g}^{\alpha_j}$ for all $j \in \mathbf{C}$.
– Compute the public keys of the honest issuers using Lagrange interpolation:

$$\hat{\mathsf{ivk}}_k = (\hat{\mathsf{ivk}})^{\prod_{m\in\mathbf{C}} \frac{m-k}{m}} \cdot \prod_{j\in\mathbf{C}} (\hat{\mathsf{ivk}}_j)^{\prod_{m\in\mathbf{C}\cup\{0\},m\neq j} \frac{k-m}{j-m}}$$

for all $k \in \mathbf{H}$.

– Output $(\texttt{VerKey}, \mathsf{sid}, (\hat{\mathsf{ivk}}_i, \hat{\mathsf{ivk}}))$ for all issuers to $\mathcal{F}_{\mathsf{SyRA},j}^{(2)}$.

Simulating honest issuer share of malicious user secret key.[18] $\mathcal{A}'$ simulates the shares of honest issuers during the threshold issuance protocol (for the malicious user's secret key generation). Similar to the previous simulation we assume the index set $\mathbf{C} = \{1, \ldots, t-1\}$ corresponds to the malicious issuers, while the indices $\mathbf{H} = \{t, \ldots, n\}$ are assigned to the honest ones. We intentionally avoid addressing leakages and calls that have been already addressed in Section 7.1 for malicious user case.

– Emulating $\mathcal{F}_{\mathsf{NIZK}}$ and an honest issuer Iss_k in the set of t issuers $\mathbf{I}$, extract the adversary's witness by executing: $w \leftarrow \mathsf{Extract}(x, \pi)$, where $w = (s, w_s, \{s_j\}_{j=1}^n, \{z_i\}_{\forall i\in\mathrm{index}(\mathbf{I})})$ for $k \in \mathbf{H}$. Check if $(x, w) \in \mathcal{R}$ holds. If it does not hold, abort.
– Query the challenger $\mathcal{C}$ using the extracted s.
[Note that: $\mathcal{C}$ sets $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{s\}$ and sends $(\chi, \hat{\chi})$ to $\mathcal{A}'$, where $\chi = g^{1/(s+\mathsf{ISK})}$ and $\hat{\chi} = \hat{g}^{1/(s+\mathsf{ISK})}$. Here, ISK is the secret key associated with $\hat{\mathsf{IVK}}$. Upon receiving $(\chi, \hat{\chi})$ from $\mathcal{C}$, $\mathcal{A}'$ must simulate the honest issuer's share such that the secret key generated by the malicious user becomes $(\chi, \hat{\chi})$. The adversary (malicious user P) receives $(\texttt{ReceivedBack}, \mathsf{sid}, \mathsf{Iss}_t, (\omega_t, \hat{\omega}_t))$ from $\mathcal{F}_{\mathsf{Ch}}^{\mathsf{sas}}$, sent by the honest issuer Iss_t. Thus, $\mathcal{A}'$ must simulate two group elements ensuring the following equalities hold upon aggregation: $\omega \leftarrow \mathsf{SSH.Agg}^{n,t}\{\omega_i\}_{\forall i\in\mathrm{index}(\mathbf{I})}$ and $\hat{\omega} \leftarrow \mathsf{SSH.Agg}^{n,t}\{\hat{\omega}_i\}_{\forall i\in\mathrm{index}(\mathbf{I})}$, such that $\omega = \chi$ and $\hat{\omega} = \hat{\chi}$ where $\mathbf{I}$ contains $t-1$ malicious issuers and an honest issuer simulated by $\mathcal{A}'$.]
– Emulating $\mathcal{F}_{\mathsf{DKG}}^{n,t}$, $\mathcal{A}'$ knows secret keys of malicious issuers: isk_j for $j \in \mathbf{C}$. And given the witness extracted by $\mathcal{A}'$: $w = (s, w_s, \{s_j\}_{j=1}^n, \{z_i\}_{\forall i\in\mathrm{index}(\mathbf{I})})$, $\mathcal{A}'$ checks if calls to $\mathcal{F}_{\mathsf{mul}}$ via the adversary (on behalf of malicious issuers) of the form: $(\texttt{Compute}, \mathsf{sid} \,||\, \mathsf{id} \,||\, \mathbf{I}, r_j, (\mathsf{isk}_j, s_j), \mathsf{aux}_j)$ for some r_j are consistent with isk_j and s_j.
  - If there exits at least one inconsistency either for isk_j or s_j for $j \in \mathbf{C}$: sample $\gamma \xleftarrow{\$} \mathbb{Z}_p$ and set $K \leftarrow g^\gamma$ and $\hat{K} \leftarrow \hat{g}^\gamma$. Output $(\texttt{Computed}, \mathsf{sid} \,||\, \mathsf{id} \,||\, \mathbf{I}, K, \hat{K})$ to all $t-1$ malicious issuers who have called $\mathcal{F}_{\mathsf{mul}}$ as output of

[18] All honest users secret keys that are DY-VRF† proofs in both source groups are hidden under ciphertexts from the adversary's view. In the following, we elaborate on the simulation of DY-VRF† proof issuance for malicious users.

$\mathcal{F}_{\mathsf{mul}}$. Emulating honest issuer Iss_t and $\mathcal{F}_{\mathsf{Ch}}^{\mathsf{sas}}$, sample $r_t \xleftarrow{\$} \mathbb{Z}_p$ and send $(\texttt{ReceivedBack}, \mathsf{sid}, \mathsf{Iss}_t, (\omega_t, \hat{\omega}_t))$ to the malicious user where $\omega_t \leftarrow K^{r_t}$, and $\hat{\omega}_t \leftarrow \hat{K}^{r_t}$.

- Else, sample $r_t \xleftarrow{\$} \mathbb{Z}_p$ and compute $r \leftarrow \mathsf{SSH.Agg}^{n,t}\left(\{r_j\}_{j\in\mathbf{C}} \cup \{r_t\}\right)$. Set $K \leftarrow \chi^{r^{-1}}$ and $\hat{K} \leftarrow \hat{\chi}^{r^{-1}}$. Output $(\texttt{Computed}, \mathsf{sid} \mid\mid \mathsf{id} \mid\mid \mathbf{I}, K, \hat{K})$ to all $t-1$ malicious issuers who have called $\mathcal{F}_{\mathsf{mul}}$ as output of $\mathcal{F}_{\mathsf{mul}}$. Emulating honest issuer Iss_t and $\mathcal{F}_{\mathsf{Ch}}^{\mathsf{sas}}$, send $(\texttt{ReceivedBack}, \mathsf{sid}, \mathsf{Iss}_t, (\omega_t, \hat{\omega}_t))$ to the malicious user where $\omega_t \leftarrow K^{r_t}$, and $\hat{\omega}_t \leftarrow \hat{K}^{r_t}$. Update: $\mathcal{L}_{\mathsf{CR}} \leftarrow \mathcal{L}_{\mathsf{CR}} \cup \{\mathsf{P}\}$, $\mathbb{S}^{\mathsf{corrupted}} \leftarrow \mathbb{S}^{\mathsf{corrupted}} \cup \{s\}$, where $\mathcal{L}_{\mathsf{CR}}$ denotes the list of corrupted-registered parties (who have been issued signing keys).

$\mathcal{A}'$ simulates the signatures σ of honest parties similar to how this simulation was done in the reduction between $\mathsf{Game}_{i-1}^{(1)}$ and $\mathsf{Game}_{i}^{(1)}$. $\mathcal{A}'$ submits $(\texttt{Signature}, \mathsf{sid}, \sigma, s^*, \mathsf{tid})$ to $\mathcal{F}_{\mathsf{SyRA},j}^{(2)}$, where $s^* = 0$.

Therefore, under the pseudorandomness of DY-VRF†, the following inequality holds:

$$|\Pr[\mathsf{Game}^{(3)}(\lambda) = 1] - \Pr[\mathsf{Game}^{(2)}(\lambda) = 1]| \leq q_t \cdot \mathsf{Adv}^{\text{pseudo}}_{\mathcal{A},\mathsf{DY\text{-}VRF}^\dagger}(\lambda)$$

$\underline{\mathsf{Game}^{(4)}}$: Same as $\mathsf{Game}^{(3)}$, except that in $\mathsf{Game}^{(4)}$, we change all tags $T = e(Z, \hat{D})$ of honest parties to random values $T \xleftarrow{\$} \mathbb{G}_T$. We highlight that in this game, for a fixed identifier s, the simulator samples fresh random group elements for different contexts unless the same identity string s holder is instructed to sign the same context. (In this case, the simulator receives the same ucid from the ideal functionality and uses the same tag value T, as we will see.)
Specifically, in $\mathsf{Game}^{(3)}$, if the same s holder was instructed to sign different contexts, the simulator used the same $\hat{D}$ value for tags (e.g., $T_1 = e(\mathsf{h}(\mathsf{ctx}_1), \hat{D})$ and $T_2 = e(\mathsf{h}(\mathsf{ctx}_2), \hat{D})$, where $\hat{D} \xleftarrow{\$} \hat{\mathbb{G}}$, and $\mathsf{h}(\cdot)$ is the output of $\mathcal{F}_{\mathsf{RO}}$). In $\mathsf{Game}^{(4)}$, if the same s holder is instructed to sign different contexts, the simulator uses different random group elements for tag computation: $(T_1, T_2) \xleftarrow{\$} \mathbb{G}_T^2$.
In leaky functionalities $\mathcal{F}_{\mathsf{SyRA},j}^{(3)}, 1 \leq j \leq q_t$, where $\mathcal{F}_{\mathsf{SyRA},1}^{(3)} := \mathcal{F}_{\mathsf{SyRA}}^{(3)}$ and $\mathcal{F}_{\mathsf{SyRA},q_t}^{(3)} := \mathcal{F}_{\mathsf{SyRA}}^{(4)}$), the leaked message to the simulator in the $\texttt{Issue}$ and $\texttt{Sign}$ commands $(\texttt{Issue}, \mathsf{sid}, \mathsf{id}, \mathsf{P}, x_s, \mathbf{I})$ and $(\texttt{Sign}, \mathsf{sid}, \mathsf{P}, \mathsf{ctx}, m, \mathsf{tid})$, respectively, for an honest party P.
We show that $\mathsf{Game}^{(4)}$ and $\mathsf{Game}^{(3)}$are indistinguishable under the DDH assumption, which allows us to bound the probability that $\mathcal{Z}$ distinguishes $\mathsf{Game}^{(4)}$ from $\mathsf{Game}^{(3)}$.
We introduce a sequences of sub-games:

$$(\mathsf{Game}_0^{(3)} := \mathsf{Game}^{(3)}, \ldots, \mathsf{Game}_{j-1}^{(3)}, \mathsf{Game}_j^{(3)}, \ldots, \mathsf{Game}_{q_t}^{(3)} := \mathsf{Game}^{(4)})$$

(where q_t denotes the upper bound on the number of all signatures of all honest parties.) Define $\mathsf{Game}_0^{(3)} := \mathsf{Game}^{(3)}$. $\mathsf{Game}_1^{(3)}$ is the same as $\mathsf{Game}_0^{(3)}$, except in $\mathsf{Game}_1^{(3)}$, we change the *first* tag value of the *first* honest party (e.g., who holds s_1) from $T_{1,1} = e(Z, \hat{D})$ to $T_{1,1} \xleftarrow{\$} \mathbb{G}_T$. in $\mathsf{Game}_2^{(3)}$, we change the *second* tag value of the *first* honest party (holding s_1) from $T_{2,1} = e(Z', \hat{D})$ to $T_{2,1} \xleftarrow{\$} \mathbb{G}_T$, and so on. The reduction between $\mathsf{Game}_{j-1}^{(3)}$ and $\mathsf{Game}_j^{(3)}$ is described below, where any difference between them is upper bounded by $\mathsf{Adv}_{\mathcal{A}}^{\mathsf{DDH}}(\lambda)$.

Associated reduction between $\mathsf{Game}_{j-1}^{(3)}$ and $\mathsf{Game}_j^{(3)}$ (DDH; for $1 \leq j \leq q_t$). If $\mathcal{Z}$ distinguishes $\mathsf{Game}_{j-1}^{(3)}$ and $\mathsf{Game}_j^{(3)}$, we can construct $\mathcal{A}'$ that breaks the DDH assumption.

- For $1 \leq k \leq j-1$: all $\hat{D}$ values (associated to each honest party P) in tags $T = e(Z, \hat{D})$ have already been substituted. For instance, $T_1 = e(Z, \hat{D})$ and $T_2 = e(Z', \hat{D})$ associated with an honest party have changed to $(T_1, T_2) \xleftarrow{\$} \mathbb{G}_T^2$.
- For $j+1 \leq k \leq q_t$: all tags are created such that for a fixed s holder the simulator always uses the same group element $\hat{D}$ in tag computation.
- The challenger $\mathcal{C}$ of DDH for $g \in \mathbb{G}$ computes: $X = g^x, Y = g^y, C_0 = g^{xy}$, and $C_1 = g^r$ where $\mathcal{C}$ samples (x, y, r) randomly: $(x, y, r) \xleftarrow{\$} \mathbb{Z}_p^*$.
- The challenger $\mathcal{C}$ provides (g, X, Y, C_b) to $\mathcal{A}'$ where $b = 0$ or $b = 1$.
  Let q_h be the total number of queries the adversary can make to the random oracle. Then:
- $i \xleftarrow{\$} [1, q_h]$, $\mathcal{A}'$ sets $Z = \mathsf{h}(\mathsf{ctx}) \leftarrow X$ if $\mathsf{ctr} = i$; (ctr is initially set to zero). Else, program the random oracle as: $Z_k = \mathsf{h}(\mathsf{ctx}_k) \leftarrow g^{x_k}$ where $x_k \xleftarrow{\$} \mathbb{Z}_p^*$ and set $\mathsf{ctr} \leftarrow \mathsf{ctr} + 1$.
  The tag value $T = e(\mathsf{h}(\mathsf{ctx}), \hat{D})$, which is computed as $e(C_0, \hat{g})$ (associated to $b = 0$ in the DDH game), is changed to $T = e(C_1, \hat{g})$ (associated to $b = 1$ in the DDH game) in $\mathsf{Game}_j^{(3)}$. Therefore, if $\mathcal{Z}$ distinguishes $\mathsf{Game}_{j-1}^{(3)}$ from $\mathsf{Game}_j^{(3)}$, $\mathcal{A}'$ uses this to win the DDH game.

$\mathcal{A}'$ simulates signatures σ of honest parties similar to how this simulation was done in the reduction between $\mathsf{Game}_{i-1}^{(1)}$ and $\mathsf{Game}_i^{(1)}$. $\mathcal{A}'$ submits $(\mathtt{Signature}, \mathsf{sid}, \sigma, s^*, \mathsf{tid})$ to $\mathcal{F}_{\mathsf{SyRA},j}^{(3)}$ where $s^* = 0$.
Therefore, under the DDH assumption, the following inequality holds:

$$|\Pr[\mathsf{Game}^{(4)}(\lambda) = 1] - \Pr[\mathsf{Game}^{(3)}(\lambda) = 1]| \leq q_t \cdot q_h \cdot \mathsf{Adv}_{\mathcal{A}}^{\mathsf{DDH}}(\lambda)$$

$\underline{\mathsf{Game}^{(5)}}$: Same as $\mathsf{Game}^{(4)}$, except that in $\mathsf{Game}^{(5)}$, $\mathcal{F}_{\mathsf{SyRA}}^{(5)}$ does not allow $\mathcal{S}^{(5)}$ to submit any message to $\mathcal{F}_{\mathsf{SyRA}}^{(5)}$ on behalf of adversary $\mathcal{A}$ (corrupted party) who generates a valid signature for a new party (who is not among honest and corrupted parties) or who generates a valid signature on behalf of an honest party. Hence, $\mathsf{Game}^{(5)}$ is the same as $\mathsf{Game}^{(4)}$, except that it checks whether a flag is raised or not. The flag is raised if $\mathcal{A}$ can generate a valid signature for a completely new party, who is neither honest nor corrupted, or forge a valid signature for an honest party.

We show that $\mathsf{Game}^{(5)}$ and $\mathsf{Game}^{(4)}$ are indistinguishable under the pseudorandomness of $\mathsf{DY\text{-}VRF}^{\dagger}$, which allows us to bound the probability that $\mathcal{Z}$ distinguishes $\mathsf{Game}^{(5)}$ from $\mathsf{Game}^{(4)}$.
Note that $\mathcal{F}_{\mathsf{SyRA}}^{(5)}$, in the `Issue` and `Sign` commands, does not leak the identity of the (honest) party P to $\mathcal{S}^{(5)}$. $\mathcal{F}_{\mathsf{SyRA}}^{(5)}$ leaks to ideal-world adversary $\mathcal{S}$ whatever our final $\mathcal{F}_{\mathsf{SyRA}}$ leaks to $\mathcal{S}$, which is $(\mathtt{Sign}, \mathsf{sid}, \mathsf{ucid}, \mathsf{ctx}, m, \mathsf{tid})$. As a result, $\mathcal{S}^{(5)}$ picks a random group element as the tag value, except in the instance that the same ucid is received from the functionality.
Associated reduction between $\mathsf{Game}^{(5)}$ and $\mathsf{Game}^{(4)}$ (pseudorandomness of $\mathsf{DY\text{-}VRF}^{\dagger}$). If $\mathcal{Z}$ distinguishes $\mathsf{Game}^{(5)}$ and $\mathsf{Game}^{(4)}$, we can construct $\mathcal{A}'$ that breaks the pseudorandomness of $\mathsf{DY\text{-}VRF}^{\dagger}$.
The final simulator $\mathcal{S}$ is described in Section 7.1. To prevent redundancy, we will highlight distinctions from this simulator. $\mathcal{S}^{(5)}$ fails with probability at most $\mathsf{Adv}^{\mathrm{pseudo}}_{\mathcal{A},\mathsf{DY\text{-}VRF}^{\dagger}}(\lambda)$ as it is shown below in cases where emulating honest verifier $\mathcal{S}^{(5)}$ receives a valid signature (from $\mathcal{A}$) that is never simulated by $\mathcal{S}^{(5)}$.

- Given the extracted witness $w = (s, \alpha, \beta)$ from $\mathcal{A}$'s statement and proof (x, π) (see Section 7.1 for more details), $\mathcal{A}'$ provides $s \notin \mathbb{S}^{\mathsf{corrupted}}$ to the challenger $\mathcal{C}$ of the pseudorandomness game (where $s \notin \mathcal{Q}$; recall that $\mathcal{Q}$ is a query list maintained by $\mathcal{C}$, which only contains s values of corrupted parties).
- The challenger $\mathcal{C}$ provides T_b to $\mathcal{A}'$.
- Parse $x = (Z, C, \hat{C}, T, \mathsf{ivk}, \mathsf{ctx}, m)$. For case $b = 0$: $T_0 = T$; and for $b = 1$: $T_1 \neq T$. Hence, if $\mathcal{A}$ provides a valid signature (see Section 7.1 for a comprehensive description of the steps involved in verifying the signature) for $s \notin \mathbb{S}^{\mathsf{corrupted}}$, $\mathcal{A}'$ uses that to win the pseudorandomness game. Therefore, the probability of failure for $\mathcal{S}^{(5)}$ is upper bounded by $\mathsf{Adv}^{\mathrm{pseudo}}_{\mathcal{A},\mathsf{DY\text{-}VRF}^{\dagger}}(\lambda)$.

To avoid repetition, we omit the description of other simulations, as they are similar to those provided in $\mathsf{Game}^{(3)}$.
Therefore, under the pseudorandomness of $\mathsf{DY\text{-}VRF}^{\dagger}$, the following inequality holds:

$$|\Pr[\mathsf{Game}^{(5)}(\lambda) = 1] - \Pr[\mathsf{Game}^{(4)}(\lambda) = 1]| \leq \mathsf{Adv}^{\mathrm{pseudo}}_{\mathcal{A},\mathsf{DY\text{-}VRF}^{\dagger}}(\lambda)$$

$\underline{\mathsf{Game}^{(6)}}$: Same as $\mathsf{Game}^{(5)}$, except that in $\mathsf{Game}^{(6)}$, $\mathcal{F}_{\mathsf{SyRA}}^{(6)}$ does not allow $\mathcal{S}^{(6)}$ to submit any message to $\mathcal{F}_{\mathsf{SyRA}}^{(6)}$ on behalf of adversary $\mathcal{A}$ (corrupted party) who:

- generates two valid signatures on behalf of a corrupted party on a context where $T_1 \neq T_2$; or
- generates two valid signatures on a context using two real-world identifiers $s_1 \neq s_2$ where $T_1 = T_2$.

Hence, $\mathsf{Game}^{(6)}$ is the same as $\mathsf{Game}^{(5)}$, except that it checks whether a flag is raised or not. The flag is raised if one of the two events above occurs. Hence, the probability that $\mathcal{Z}$ distinguishes $\mathsf{Game}^{(6)}$ from $\mathsf{Game}^{(5)}$ is zero, as tag values have uniqueness with respect to the context and s value of the party. More precisely:

- For a given s and context ctx, all generated signatures have the same tag $T = e(Z, \hat{g})^{1/(\mathsf{isk}+s)}$.
- For a given context ctx, s_1, and s_2 where $s_1 \neq s_2$, we always have $T_1 = e(Z, \hat{g})^{1/(\mathsf{isk}+s_1)} \neq T_2 = e(Z, \hat{g})^{1/(\mathsf{isk}+s_2)}$, as $s \ll |\mathbb{Z}_p^*|$.

Hence, the flag is never raised, and the following equality holds:

$$|\Pr[\mathsf{Game}^{(6)}(\lambda) = 1] = \Pr[\mathsf{Game}^{(5)}(\lambda) = 1]$$

As $\mathcal{F}_{\mathsf{SyRA}}^{(6)} = \mathcal{F}_{\mathsf{SyRA}}$ and $\mathcal{S}^{(6)} = \mathcal{S}$, so that $\mathsf{Game}^{(6)}$ corresponds to the ideal-world execution $\mathsf{EXEC}_{\mathcal{F}_{\mathsf{SyRA}},\mathcal{S},\mathcal{Z}}$, we argue that random variables $\mathsf{EXEC}_{\Pi_{\mathsf{SASSI}},\mathcal{A},\mathcal{Z}}$ and $\mathsf{EXEC}_{\mathcal{F}_{\mathsf{SyRA}},\mathcal{S},\mathcal{Z}}$ are statistically close. Indeed, the probability for any PPT environment $\mathcal{Z}$ to distinguish $\mathsf{EXEC}_{\Pi_{\mathsf{SASSI}},\mathcal{A},\mathcal{Z}}$ from $\mathsf{EXEC}_{\mathcal{F}_{\mathsf{SyRA}},\mathcal{S},\mathcal{Z}}$ is upper bounded by

$$\begin{aligned}&\mathsf{Adv}^{\mathrm{bind}}_{\mathcal{A},\mathsf{COM}}(\lambda) + 2q_t \cdot \mathsf{Adv}^{\mathsf{IND\text{-}CPA}}_{\mathcal{A}}(\lambda)\\ &+ (q_t + 1) \cdot \mathsf{Adv}^{\mathrm{pseudo}}_{\mathcal{A},\mathsf{DY\text{-}VRF}^\dagger}(\lambda) + q_t \cdot q_h \cdot \mathsf{Adv}^{\mathsf{DDH}}_{\mathcal{A}}(\lambda)\end{aligned}$$

which concludes the security proof.

Acknowledgements

This work has been supported by Input Output (iohk.io) through their funding of the University of Edinburgh Blockchain Technology Lab. We thank Lovesh Harchandani for implementing the system and contributing to the performance evaluation.

References

Au et al.(2013). Man Ho Au, Joseph K. Liu, Willy Susilo, and Tsz Hon Yuen. 2013. Secure ID-based linkable and revocable-iff-linked ring signature with constant-size construction. *Theor. Comput. Sci.* 469 (2013), 1–14. https://doi.org/10.1016/j.tcs.2012.10.031

Baldimtsi et al.(2017). Foteini Baldimtsi, Jan Camenisch, Maria Dubovitskaya, Anna Lysyanskaya, Leonid Reyzin, Kai Samelin, and Sophia Yakoubov. 2017. Accumulators with Applications to Anonymity-Preserving Revocation. In *2017 IEEE European Symposium on Security and Privacy, EuroS&P 2017, Paris, France, April 26-28, 2017*. IEEE, 301–315. https://doi.org/10.1109/EuroSP.2017.13

Baldimtsi et al.(2024). Foteini Baldimtsi, Konstantinos Kryptos Chalkias, Yan Ji, Jonas Lindstrøm, Deepak Maram, Ben Riva, Arnab Roy, Mahdi Sedaghat, and Joy Wang. 2024. zklogin: Privacy-preserving blockchain authentication with existing credentials. *arXiv preprint arXiv:2401.11735* (2024).

Ben-Sasson et al.(2014). Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. 2014. Zerocash: Decentralized Anonymous Payments from Bitcoin. In *2014 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, 459–474. https://doi.org/10.1109/SP.2014.36

Bender et al.(2012). Jens Bender, Özgür Dagdelen, Marc Fischlin, and Dennis Kügler. 2012. Domain-specific pseudonymous signatures for the german identity card. In *Information Security: 15th International Conference, ISC 2012, Passau, Germany, September 19-21, 2012. Proceedings 15*. Springer, 104–119.

Błaśkiewicz et al.(2019). Przemysław Błaśkiewicz, Lucjan Hanzlik, Kamil Kluczniak, Łukasz Krzywiecki, Mirosław Kutyłowski, Marcin Słowik, and Marta Wszoła. 2019. Pseudonymous signature schemes. *Advances in Cyber Security: Principles, Techniques, and Applications* (2019), 185–255.

Boneh et al.(2004a). Dan Boneh, Xavier Boyen, and Hovav Shacham. 2004a. Short Group Signatures. In *CRYPTO 2004 (LNCS, Vol. 3152)*, Matthew Franklin (Ed.). Springer, Heidelberg, 41–55. https://doi.org/10.1007/978-3-540-28628-8_3

Boneh et al.(2004b). Dan Boneh, Ben Lynn, and Hovav Shacham. 2004b. Short Signatures from the Weil Pairing. *Journal of Cryptology* 17, 4 (Sept. 2004), 297–319. https://doi.org/10.1007/s00145-004-0314-9

Boyle et al.(2021). Elette Boyle, Ran Cohen, and Aarushi Goel. 2021. Breaking the $O(\sqrt{n})$-Bit Barrier: Byzantine Agreement with polylog bits per party. In *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing*. 319–330.

Brickell and Li(2010). Ernie Brickell and Jiangtao Li. 2010. A pairing-based DAA scheme further reducing TPM resources. In *International Conference on Trust and Trustworthy Computing*. Springer, 181–195.

Bringer et al.(2014). Julien Bringer, Hervé Chabanne, Roch Lescuyer, and Alain Patey. 2014. Efficient and strongly secure dynamic domain-specific pseudonymous signatures for ID documents. In *Financial Cryptography and Data Security: 18th International Conference, FC 2014, Christ Church, Barbados, March 3-7, 2014, Revised Selected Papers 18*. Springer, 255–272.

Bünz et al.(2020). Benedikt Bünz, Shashank Agrawal, Mahdi Zamani, and Dan Boneh. 2020. Zether: Towards privacy in a smart contract world. In *International Conference on Financial Cryptography and Data Security*. Springer, 423–443.

Burdges et al.(2023). Jeffrey Burdges, Oana Ciobotaru, Elizabeth Crites, Handan Kılınç Alper, Alistair Stewart, and Sergey Vasilyev. 2023. Ring Verifiable Random Functions. Cryptology ePrint Archive, Paper 2023/002. https://eprint.iacr.org/2023/002

Camenisch et al.(2018). Jan Camenisch, Manu Drijvers, Tommaso Gagliardoni, Anja Lehmann, and Gregory Neven. 2018. The wonderful world of global random oracles. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 280–312.

Camenisch et al.(2006b). Jan Camenisch, Susan Hohenberger, Markulf Kohlweiss, Anna Lysyanskaya, and Mira Meyerovich. 2006b. How to win the clonewars: efficient periodic n-times anonymous authentication. In *Proceedings of the 13th ACM Conference on Computer and Communications Security, CCS 2006, Alexandria, VA, USA, October 30 - November 3, 2006*. ACM, 201–210. https://doi.org/10.1145/1180405.1180431

Camenisch et al.(2005). Jan Camenisch, Susan Hohenberger, and Anna Lysyanskaya. 2005. Compact E-Cash. In *EUROCRYPT 2005 (LNCS, Vol. 3494)*, Ronald Cramer (Ed.). Springer, Heidelberg, 302–321. https://doi.org/10.1007/11426639_18

Camenisch et al.(2006a). Jan Camenisch, Susan Hohenberger, and Anna Lysyanskaya. 2006a. Balancing Accountability and Privacy Using E-Cash (Extended Abstract). In *SCN 06 (LNCS, Vol. 4116)*, Roberto De Prisco and Moti Yung (Eds.). Springer, Heidelberg, 141–155. https://doi.org/10.1007/11832072_10

Camenisch and Lysyanskaya(2001). Jan Camenisch and Anna Lysyanskaya. 2001. An Efficient System for Non-transferable Anonymous Credentials with Optional Anonymity Revocation. In *EUROCRYPT 2001 (LNCS, Vol. 2045)*, Birgit Pfitzmann (Ed.). Springer, Heidelberg, 93–118. https://doi.org/10.1007/3-540-44987-6_7

Canetti et al.(2020). Ran Canetti, Rosario Gennaro, Steven Goldfeder, Nikolaos Makriyannis, and Udi Peled. 2020. UC Non-Interactive, Proactive, Threshold ECDSA with Identifiable Aborts. In *ACM CCS 2020*, Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna (Eds.). ACM Press, 1769–1787. https://doi.org/10.1145/3372297.3423367

Chase and Lysyanskaya(2006). Melissa Chase and Anna Lysyanskaya. 2006. On Signatures of Knowledge. Cryptology ePrint Archive, Report 2006/184. https://eprint.iacr.org/2006/184.

Chaum(1985). David Chaum. 1985. Security Without Identification: Transaction Systems to Make Big Brother Obsolete. *Commun. ACM* 28, 10 (1985), 1030–1044. https://doi.org/10.1145/4372.4373

Chaum and Evertse(1986). David Chaum and Jan-Hendrik Evertse. 1986. A Secure and Privacy-protecting Protocol for Transmitting Personal Information Between Organizations. In *Advances in Cryptology - CRYPTO '86, Santa Barbara, California, USA, 1986, Proceedings (Lecture Notes in Computer Science, Vol. 263)*. Springer, 118–167. https://doi.org/10.1007/3-540-47721-7_10

Chaum and van Heyst(1991). David Chaum and Eugène van Heyst. 1991. Group Signatures. In *EUROCRYPT'91 (LNCS, Vol. 547)*, Donald W. Davies (Ed.). Springer, Heidelberg, 257–265. https://doi.org/10.1007/3-540-46416-6_22

Company(2024). Electric Coin Company. 2024. New SNARK Curve. https://electriccoin.co/blog/new-snark-curve/.

Consortium(2022). The World Wide Web Consortium. 2022. Decentralized Identifiers (DIDs). https://w3c.github.io/did-core/.

Cothority(2024). Cothority. 2024. DKG Implementation. Available at: Link..

Damgård et al.(2006). Ivan Damgård, Kasper Dupont, and Michael Østergaard Pedersen. 2006. Unclonable Group Identification. In *Advances in Cryptology - EUROCRYPT 2006, 25th Annual International Conference on the Theory and Applications of Cryptographic Techniques, St. Petersburg, Russia, May 28 - June 1, 2006, Proceedings (Lecture Notes in Computer Science, Vol. 4004)*. Springer, 555–572. https://doi.org/10.1007/11761679_33

Damgård et al.(2021). Ivan Damgård, Chaya Ganesh, Hamidreza Khoshakhlagh, Claudio Orlandi, and Luisa Siniscalchi. 2021. Balancing Privacy and Accountability in Blockchain Identity Management. In *CT-RSA 2021 (LNCS, Vol. 12704)*, Kenneth G. Paterson (Ed.). Springer, Heidelberg, 552–576. https://doi.org/10.1007/978-3-030-75539-3_23

David et al.(2024). Bernardo David, Rafael Dowsley, Anders Konring, and Mario Larangeira. 2024. MUSEN: Aggregatable Key-Evolving Verifiable Random Functions and Applications. Cryptology ePrint Archive, Paper 2024/628. https://eprint.iacr.org/2024/628

Dfns(2024). Dfns. 2024. CGGMP21 In Rust, At Last. Available at: Link..

Diaz et al.(2008). Claudia Diaz, Eleni Kosta, Hannelore Dekeyser, Markulf Kohlweiss, and Girma Nigusse. 2008. Privacy preserving electronic petitions. *Identity in the Information Society* 1, 1 (2008), 203–219.

Dingledine et al.(2004). Roger Dingledine, Nick Mathewson, and Paul F. Syverson. 2004. Tor: The Second-Generation Onion Router. In *USENIX Security 2004*, Matt Blaze (Ed.). USENIX Association, 303–320.

Dodis and Yampolskiy(2005). Yevgeniy Dodis and Aleksandr Yampolskiy. 2005. A verifiable random function with short proofs and keys. In *International Workshop on Public Key Cryptography*. Springer, 416–431.

Dodis et al.(2006). Yevgeniy Dodis, Aleksandr Yampolskiy, and Moti Yung. 2006. Threshold and Proactive Pseudo-Random Permutations. In *Theory of Cryptography, Third Theory of Cryptography Conference, TCC 2006, New York, NY, USA, March 4-7, 2006, Proceedings (Lecture Notes in Computer Science, Vol. 3876)*. Springer, 542–560. https://doi.org/10.1007/11681878_28

Doerner et al.(2023). Jack Doerner, Yashvanth Kondi, Eysa Lee, Abhi Shelat, and LaKyah Tyner. 2023. Threshold bbs+ signatures for distributed anonymous credential issuance. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 773–789.

ElGamal(1985). Taher ElGamal. 1985. A public key cryptosystem and a signature scheme based on discrete logarithms. *IEEE transactions on information theory* 31, 4 (1985), 469–472.

Escala and Groth(2014). Alex Escala and Jens Groth. 2014. Fine-Tuning Groth-Sahai Proofs. In *PKC 2014 (LNCS, Vol. 8383)*, Hugo Krawczyk (Ed.). Springer, Heidelberg, 630–649. https://doi.org/10.1007/978-3-642-54631-0_36

Even et al.(1985). Shimon Even, Oded Goldreich, and Abraham Lempel. 1985. A randomized protocol for signing contracts. *Commun. ACM* 28, 6 (1985), 637–647.

Fauzi et al.(2019). Prastudy Fauzi, Sarah Meiklejohn, Rebekah Mercer, and Claudio Orlandi. 2019. Quisquis: A new design for anonymous cryptocurrencies. In *Advances in Cryptology–ASIACRYPT 2019: 25th International Conference on the Theory and Application of Cryptology and Information Security, Kobe, Japan, December 8–12, 2019, Proceedings, Part I 25*. Springer, 649–678.

Fedimint(2024). Fedimint. 2024. DKG Implementation. Available at: Link..

Fett et al.(2016). Daniel Fett, Ralf Küsters, and Guido Schmitz. 2016. A Comprehensive Formal Security Analysis of OAuth 2.0. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, USA, 1204–1215. https://doi.org/10.1145/2976749.2978385

Fiat and Shamir(1987). Amos Fiat and Adi Shamir. 1987. How to Prove Yourself: Practical Solutions to Identification and Signature Problems. In *CRYPTO'86 (LNCS, Vol. 263)*, Andrew M. Odlyzko (Ed.). Springer, Heidelberg, 186–194. https://doi.org/10.1007/3-540-47721-7_12

Foundation(2023). Decentralized Identity Foundation. 2023. https://identity.foundation/.

Fujisaki and Suzuki(2007). Eiichiro Fujisaki and Koutarou Suzuki. 2007. Traceable Ring Signature. In *PKC 2007 (LNCS, Vol. 4450)*, Tatsuaki Okamoto and Xiaoyun Wang (Eds.). Springer, Heidelberg, 181–200. https://doi.org/10.1007/978-3-540-71677-8_13

Garay et al.(2011). Juan A Garay, Jonathan Katz, Ranjit Kumaresan, and Hong-Sheng Zhou. 2011. Adaptively secure broadcast, revisited. In *Proceedings of the 30th annual ACM SIGACT-SIGOPS symposium on Principles of distributed computing*. 179–186.

Gennaro et al.(2007). Rosario Gennaro, Stanislaw Jarecki, Hugo Krawczyk, and Tal Rabin. 2007. Secure Distributed Key Generation for Discrete-Log Based Cryptosystems. *Journal of Cryptology* 20, 1 (Jan. 2007), 51–83. https://doi.org/10.1007/s00145-006-0347-3

Groth(2010). Jens Groth. 2010. Short Pairing-Based Non-interactive Zero-Knowledge Arguments. In *ASIACRYPT 2010 (LNCS, Vol. 6477)*, Masayuki Abe (Ed.). Springer, Heidelberg, 321–340. https://doi.org/10.1007/978-3-642-17373-8_19

Groth et al.(2006). Jens Groth, Rafail Ostrovsky, and Amit Sahai. 2006. Perfect non-interactive zero knowledge for NP. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 339–358.

Guillou and Quisquater(1988). Louis C. Guillou and Jean-Jacques Quisquater. 1988. A Practical Zero-Knowledge Protocol Fitted to Security Microprocessor Minimizing Both Trasmission and Memory. In *EUROCRYPT'88 (LNCS, Vol. 330)*, C. G. Günther (Ed.). Springer, Heidelberg, 123–128. https://doi.org/10.1007/3-540-45961-8_11

Gurkan et al.(2021). Kobi Gurkan, Philipp Jovanovic, Mary Maller, Sarah Meiklejohn, Gilad Stern, and Alin Tomescu. 2021. Aggregatable Distributed Key Generation. In *EUROCRYPT 2021, Part I (LNCS, Vol. 12696)*, Anne Canteaut and François-Xavier Standaert (Eds.). Springer, Heidelberg, 147–176. https://doi.org/10.1007/978-3-030-77870-5_6

Haitner et al.(2022). Iftach Haitner, Nikolaos Makriyannis, Samuel Ranellucci, and Eliad Tsfadia. 2022. Highly efficient ot-based multiplication protocols. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 180–209.

Herranz(2006). Javier Herranz. 2006. Deterministic Identity-Based Signatures for Partial Aggregation. *Comput. J.* 49, 3 (2006), 322–330. https://doi.org/10.1093/comjnl/bxh153

Hess(2003). Florian Hess. 2003. Efficient Identity Based Signature Schemes Based on Pairings. In *SAC 2002 (LNCS, Vol. 2595)*, Kaisa Nyberg and Howard M. Heys (Eds.). Springer, Heidelberg, 310–324. https://doi.org/10.1007/3-540-36492-7_20

JAP(2009). JAP. 2009. JAP Anonymity & Privacy. http://anon.inf.tu-dresden.de/.

Katz(2024). Jonathan Katz. 2024. Round-optimal, fully secure distributed key generation. In *Annual International Cryptology Conference*. Springer, 285–316.

Kiayias et al.(2022). Aggelos Kiayias, Markulf Kohlweiss, and Amirreza Sarencheh. 2022. PEReDi: Privacy-enhanced, regulated and distributed central bank digital currencies. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 1739–1752.

Kiayias et al.(2004). Aggelos Kiayias, Yiannis Tsiounis, and Moti Yung. 2004. Traceable Signatures. In *EUROCRYPT 2004 (LNCS, Vol. 3027)*, Christian Cachin and Jan Camenisch (Eds.). Springer, Heidelberg, 571–589. https://doi.org/10.1007/978-3-540-24676-3_34

Kiayias and Zhou(2007). Aggelos Kiayias and Hong-Sheng Zhou. 2007. Hidden Identity-Based Signatures. In *FC 2007 (LNCS, Vol. 4886)*, Sven Dietrich and Rachna Dhamija (Eds.). Springer, Heidelberg, 134–147.

Kluczniak et al.(2016). Kamil Kluczniak, Lucjan Hanzlik, and Mirosław Kutyłowski. 2016. A formal concept of domain pseudonymous signatures. In *Information Security Practice and Experience: 12th International Conference, ISPEC 2016, Zhangjiajie, China, November 16-18, 2016, Proceedings 12*. Springer, 238–254.

Kondi and abhi shelat(2022). Yashvanth Kondi and abhi shelat. 2022. Improved Straight-Line Extraction in the Random Oracle Model With Applications to Signature Aggregation. Cryptology ePrint Archive. https://eprint.iacr.org/2022/393

Kutyłowski et al.(2016). Mirosław Kutyłowski, Lucjan Hanzlik, and Kamil Kluczniak. 2016. Pseudonymous signature on eIDAS token–implementation based privacy threats. In *Australasian Conference on Information Security and Privacy*. Springer, 467–477.

Labs(2024). Bolt Labs. 2024. DKG Implementation. Available at: Link..

Liu et al.(2004). Joseph K. Liu, Victor K. Wei, and Duncan S. Wong. 2004. Linkable Spontaneous Anonymous Group Signature for Ad Hoc Groups (Extended Abstract). In *ACISP 04 (LNCS, Vol. 3108)*, Huaxiong Wang, Josef Pieprzyk, and Vijay Varadharajan (Eds.). Springer, Heidelberg, 325–335. https://doi.org/10.1007/978-3-540-27800-9_28

Lysyanskaya and Rosenbloom(2022). Anna Lysyanskaya and Leah Namisa Rosenbloom. 2022. Universally Composable Sigma-protocols in the Global Random-Oracle Model. Cryptology ePrint Archive. https://eprint.iacr.org/2022/290

Maram et al.(2021). Deepak Maram, Harjasleen Malvai, Fan Zhang, Nerla Jean-Louis, Alexander Frolov, Tyler Kell, Tyrone Lobban, Christine Moy, Ari Juels, and Andrew Miller. 2021. Candid: Can-do decentralized identity with legacy compatibility, sybil-resistance, and accountability. In *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1348–1366.

Metere and Dong(2017). Roberto Metere and Changyu Dong. 2017. Automated cryptographic analysis of the pedersen commitment scheme. In *Computer Network Security: 7th International Conference on Mathematical Methods, Models, and Architectures for Computer Network Security, MMM-ACNS 2017, Warsaw, Poland, August 28-30, 2017, Proceedings 7*. Springer, 275–287.

Miao et al.(2020). Peihan Miao, Sarvar Patel, Mariana Raykova, Karn Seth, and Moti Yung. 2020. Two-Sided Malicious Security for Private Intersection-Sum with Cardinality. In *CRYPTO 2020, Part III (LNCS, Vol. 12172)*, Daniele Micciancio and Thomas Ristenpart (Eds.). Springer, Heidelberg, 3–33. https://doi.org/10.1007/978-3-030-56877-1_1

Micali et al.(2003). Silvio Micali, Michael Rabin, and Joe Kilian. 2003. Zero-knowledge sets. In *44th Annual IEEE Symposium on Foundations of Computer Science, 2003. Proceedings.* IEEE, 80–91.

Park et al.(2023). Sanghyeon Park, Jeong Hyuk Lee, Seunghwa Lee, Jung Hyun Chun, Hyeonmyeong Cho, MinGi Kim, Hyun Ki Cho, and Soo-Mook Moon. 2023. Beyond the Blockchain Address: Zero-Knowledge Address Abstraction. *IACR Cryptol. ePrint Arch.* (2023), 191. https://eprint.iacr.org/2023/191

Pedersen(1991). Torben Pryds Pedersen. 1991. Non-interactive and information-theoretic secure verifiable secret sharing. In *Annual international cryptology conference*. Springer, 129–140.

Rial and Piotrowska(2022). Alfredo Rial and Ania M Piotrowska. 2022. Security Analysis of Coconut, an Attribute-Based Credential Scheme with Threshold Issuance. *Cryptology ePrint Archive* (2022).

Rivest et al.(2001). Ronald L. Rivest, Adi Shamir, and Yael Tauman. 2001. How to Leak a Secret. In *ASIACRYPT 2001 (LNCS, Vol. 2248)*, Colin Boyd (Ed.). Springer, Heidelberg, 552–565. https://doi.org/10.1007/3-540-45682-1_32

Sakimura et al.(2014). Nat Sakimura, John Bradley, Mike Jones, Breno De Medeiros, and Chuck Mortimore. 2014. OpenID Connect Core 1.0 incorporating errata set 1. *The OpenID Foundation, specification* 335 (2014).

Sarencheh et al.(2025a). Amirreza Sarencheh, Alireza Kavousi, Hamidreza Khoshakhlagh, and Aggelos Kiayias. 2025a. DART: Decentralized, Anonymous, and Regulation-friendly Tokenization. *Cryptology ePrint Archive* (2025).

Sarencheh et al.(2025b). Amirreza Sarencheh, Aggelos Kiayias, and Markulf Kohlweiss. 2025b. PARScoin: A Privacy-preserving, Auditable, and Regulation-friendly Stablecoin. In *Cryptology and Network Security*. Springer Nature Singapore, Singapore, 289–313.

Sasson et al.(2014). Eli Ben Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. 2014. Zerocash: Decentralized anonymous payments from bitcoin. In *2014 IEEE symposium on security and privacy*. IEEE, 459–474.

Scafuro and Zhang(2021). Alessandra Scafuro and Bihan Zhang. 2021. One-Time Traceable Ring Signatures. In *ESORICS 2021, Part II (LNCS, Vol. 12973)*, Elisa Bertino, Haya Shulman, and Michael Waidner (Eds.). Springer, Heidelberg, 481–500. https://doi.org/10.1007/978-3-030-88428-4_24

Schnorr(1991). Claus-Peter Schnorr. 1991. Efficient Signature Generation by Smart Cards. *Journal of Cryptology* 4, 3 (Jan. 1991), 161–174. https://doi.org/10.1007/BF00196725

Shamir(1979). Adi Shamir. 1979. How to share a secret. *Commun. ACM* 22, 11 (1979), 612–613.

Shamir(1984). Adi Shamir. 1984. Identity-Based Cryptosystems and Signature Schemes. In *CRYPTO'84 (LNCS, Vol. 196)*, G. R. Blakley and David Chaum (Eds.). Springer, Heidelberg, 47–53.

Sonnino et al.(2019). Alberto Sonnino, Mustafa Al-Bassam, Shehar Bano, Sarah Meiklejohn, and George Danezis. 2019. Coconut: Threshold Issuance Selective Disclosure Credentials with Applications to Distributed Ledgers. In *NDSS 2019*. The Internet Society.

Tomescu et al.(2022). Alin Tomescu, Adithya Bhat, Benny Applebaum, Ittai Abraham, Guy Gueta, Benny Pinkas, and Avishay Yanai. 2022. UTT: Decentralized Ecash with Accountable Privacy. *Cryptology ePrint Archive* (2022).

Volkamer and Grimm(2006). Melanie Volkamer and Rüdiger Grimm. 2006. Multiple Casts in Online Voting: Analyzing Chances. In *Electronic Voting 2006: 2nd International Workshop, Co-organized by Council of Europe, ESF TED, IFIP WG 8.6 and E-Voting.CC, August, 2nd - 4th, 2006 in Castle Hofen, Bregenz, Austria (LNI, Vol. P-86)*. GI, 97–106. https://dl.gi.de/handle/20.500.12116/29174

Wang et al.(2012). Rui Wang, Shuo Chen, and XiaoFeng Wang. 2012. Signing Me onto Your Accounts through Facebook and Google: A Traffic-Guided Security Study of Commercially Deployed Single-Sign-On Web Services. In *2012 IEEE Symposium on Security and Privacy*. 365–379. https://doi.org/10.1109/SP.2012.30

Worldcoin(2023). Worldcoin. 2023. https://whitepaper.worldcoin.org/.

X-explore(2023). X-explore. 2023. Advanced Analysis For Arbitrum Airdrop. https://mirror.xyz/x-explore.eth/AFroG11e24I6S1oDvTitNdQSDh8lN5bz9VZAink8lZ4.

Yang et al.(2021). Kang Yang, Liqun Chen, Zhenfeng Zhang, Christopher JP Newton, Bo Yang, and Li Xi. 2021. Direct anonymous attestation with optimal tpm signing efficiency. *IEEE transactions on information forensics and security* 16 (2021), 2260–2275.

Zhang et al.(2016). Fan Zhang, Ethan Cecchetti, Kyle Croman, Ari Juels, and Elaine Shi. 2016. Town Crier: An Authenticated Data Feed for Smart Contracts. In *ACM CCS 2016*, Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi (Eds.). ACM Press, 270–282. https://doi.org/10.1145/2976749.2978326

Zhang et al.(2020). Fan Zhang, Deepak Maram, Harjasleen Malvai, Steven Goldfeder, and Ari Juels. 2020. DECO: Liberating Web Data Using Decentralized Oracles for TLS. In *ACM CCS 2020*, Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna (Eds.). ACM Press, 1919–1938. https://doi.org/10.1145/3372297.3417239

A Preliminaries (continued)

A.1 Cryptographic assumptions

Assumption 1 (Decisional Diffie-Hellman (DDH)) *The decisional Diffie-Hellman assumption holds in $\mathbb{G}$ for* GrGen *if for all PPT adversaries $\mathcal{A}$, there exists a negligible function* negl *such that:*

$$|\Pr[\mathcal{A}(\mathsf{bp}, g^x, g^y, g^r) = 1] - \Pr[\mathcal{A}(\mathsf{bp}, g^x, g^y, g^{xy}) = 1]| \leq \mathsf{negl}(\lambda)$$

where in each case the probabilities are taken over the experiment in which $\mathsf{GrGen}(1^\lambda)$ *outputs* $\mathsf{bp} = (p, \mathbb{G}, \hat{\mathbb{G}}, \mathbb{G}_T, e, g, \hat{g})$*, and then random $x, y, r \in \mathbb{Z}_p$ are chosen.*

The DDH assumption may be defined similarly for $\hat{\mathbb{G}}$ and $\mathbb{G}_T$.

Assumption 2 (q-Decisional Bilinear Diffie-Hellman Inversion (q-DBDHI)) [Dodis and Yampolskiy(2005)] *Let* GrGen *be a group generator that outputs* $\mathsf{bp} = (\mathbb{G}, \hat{\mathbb{G}}, \mathbb{G}_T, p, e, g, \hat{g})$*. The q-Decisional Bilinear Diffie-Hellman Inversion assumption holds for* GrGen *if for all PPT adversaries $\mathcal{A}$, there exists a negligible function* negl *such that:*

$$\begin{aligned}&\big|\Pr[\mathcal{A}(\mathsf{bp}, g^x, \hat{g}^x, \ldots, g^{x^q}, \hat{g}^{x^q}; e(g, \hat{g})^{1/x})]\\ &-\Pr[\mathcal{A}(\mathsf{bp}, g^x, \hat{g}^x, \ldots, g^{x^q}, \hat{g}^{x^q}; \Gamma)]\big| \leq \mathsf{negl}(\lambda)\end{aligned}$$

where the probability is taken over the internal coin tosses of $\mathcal{A}$ and choices of $x \in \mathbb{Z}_p^*$ and $\Gamma \in \mathbb{G}_T$.

A.2 Shamir secret sharing

In Shamir secret sharing [Shamir(1979)], a secret value s is divided into n parts such that any t shares together can be used to reconstruct the secret s. However, fewer than t shares provide no information about the secret.

Definition 6 (Shamir secret sharing (SSH)). *A (n, t)-secret sharing scheme* $\mathsf{SSH} = (\mathsf{SSH.Share}, \mathsf{SSH.Agg})$ *consists of the following algorithms:*

- $\{s_i\}_{i=1}^n \xleftarrow{\$} \mathsf{SSH.Share}^{n,t}(s)$*: Given s as input, this algorithm outputs n secret shares, where s_i denotes the i-th share of s. To share a secret s into t shares, choose $t-1$ random scalars $(a_1, \ldots, a_{t-1})$ and construct a polynomial $P(x) = s + a_1 x + \ldots + a_{t-1}x^{t-1}$. Each share i is represented by the pair $(x_i, P(x_i))$, where the x_i values are distinct and non-zero.*

– $s^* \leftarrow \mathsf{SSH.Agg}^{n,t}\{s_i\}_{i\in\mathbf{I}}$*: Given* t *shares* ($|\mathbf{I}| = t$) *as input, this algorithm reconstructs the secret* s^**. This is achieved via Lagrange interpolation as follows:* $s = \sum_{i\in\mathbf{I}} y_i \cdot \prod_{j\neq i, j\in\mathbf{I}} \frac{x_j}{x_j - x_i}$*, where* $\mathbf{I}$ *is the set of indices of the shares used for reconstruction.*

Definition 7 (SSH correctness). $\Pr[s^* = s] = 1$.

Definition 8 (SSH privacy). *Requires that, given* $t-1$ *or fewer shares of either* s_1 *or* s_0*, no PPT adversary* $\mathcal{A}$ *can distinguish which message was shared.*

A.3 Public key encryption (PKE)

Definition 9 (Public key encryption (PKE) scheme). *Let* $\mathsf{PKE} = (\mathsf{PKE.KeyGen}, \mathsf{PKE.Enc}, \mathsf{PKE.Dec})$ *be a public key encryption (PKE) scheme. A PKE scheme is defined as follows:*

– $\mathsf{PKE.KeyGen}$ ***(Key generation):*** *This algorithm takes a security parameter* λ *as input and outputs a pair* $(\mathsf{pk}, \mathsf{sk})$*, where* pk *is the public key and* sk *is the secret (private) key. The public key* pk *also (implicitly) defines a corresponding message space, denoted as* $\mathsf{MessageSpace}(\mathsf{pk})$*, which specifies the set of valid plaintexts for encryption.*
– $\mathsf{PKE.Enc}$ ***(Encryption):*** *This algorithm takes the public key* pk *and a plaintext message* m *(where* $m \in \mathsf{MessageSpace}(\mathsf{pk})$*) as input and produces a ciphertext* C*.*
– $\mathsf{PKE.Dec}$ ***(Decryption):*** *The decryption algorithm takes the private key* sk *and a ciphertext* C *as input and outputs either the original plaintext message* m *or* $\perp$ *if decryption fails.*

Definition 10 (PKE correctness). *A public key encryption (PKE) scheme* $\mathsf{PKE} = (\mathsf{PKE.KeyGen}, \mathsf{PKE.Enc}, \mathsf{PKE.Dec})$ *is said to be correct if, for all* $\lambda \in \mathbb{N}$*, for all key pairs* $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{PKE.KeyGen}(1^\lambda)$*, and for all messages* $m \in \mathsf{MessageSpace}(\mathsf{pk})$*,*
$\mathsf{PKE.Dec}(\mathsf{sk}, \mathsf{PKE.Enc}(\mathsf{pk}, m)) = m$ *holds with overwhelming probability.*

Definition 11 (PKE CPA security). *A public key encryption (PKE) scheme* $\mathsf{PKE} = (\mathsf{PKE.KeyGen}, \mathsf{PKE.Enc}, \mathsf{PKE.Dec})$ *is* $\mathsf{IND\text{-}CPA}$*-secure if, for all PPT adversaries* $\mathcal{A}$*, the advantage* $\mathsf{Adv}^{\mathsf{IND\text{-}CPA}}_{\mathcal{A}}(\lambda)$ *in the following experiment is negligible in* λ*:* $\mathsf{Adv}^{\mathsf{IND\text{-}CPA}}_{\mathcal{A}}(\lambda) \leq \mathsf{negl}(\lambda)$.
The experiment, denoted $\mathsf{IND\text{-}CPA}^b_{\mathsf{PKE}}(\mathcal{A}, \lambda)$*, is defined between a probabilistic polynomial-time (PPT) adversary* $\mathcal{A}$ *and a challenger, with a security parameter* λ *and a bit* $b \in \{0, 1\}$*:*

1. *The challenger generates a key pair* $(\mathsf{pk}, \mathsf{sk})$ *by running* $\mathsf{PKE.KeyGen}(1^\lambda)$ *and sends the public key* pk *to the adversary* $\mathcal{A}$*.*
2. *The adversary* $\mathcal{A}$ *submits two plaintext messages* (m_0, m_1)*, ensuring that* $|m_0| = |m_1|$ *and* $m_0, m_1 \in \mathsf{MessageSpace}(\mathsf{pk})$*.*
3. *The challenger selects a random bit* $b \xleftarrow{\$} \{0, 1\}$*, chooses one of the two plaintexts based on* b*, and computes the ciphertext* $C_b = \mathsf{PKE.Enc}(\mathsf{pk}, m_b)$*.*

4. *The challenger sends the ciphertext C_b to the adversary $\mathcal{A}$.*
5. *The adversary $\mathcal{A}$ outputs a bit b' as its guess for b. If $\mathcal{A}$ does not output any value, the guess b' is set to 0 by default.*

The adversary's advantage $\mathsf{Adv}^{\mathsf{IND\text{-}CPA}}_{\mathcal{A}}(\lambda)$ *in this experiment is defined as:*

$$\mathsf{Adv}^{\mathsf{IND\text{-}CPA}}_{\mathcal{A}}(\lambda) = \big|\Pr[\mathsf{IND\text{-}CPA}^{1}_{\mathsf{PKE}}(\mathcal{A},\lambda)=1] - \Pr[\mathsf{IND\text{-}CPA}^{0}_{\mathsf{PKE}}(\mathcal{A},\lambda)=1]\big|$$

Definition 12 (ElGamal encryption scheme). *The ElGamal encryption scheme [ElGamal(1985)], which we denote by* EG*, is a public key encryption (PKE) scheme that operates over a prime-order cyclic group* $\mathbb{G}_q$ *with generator* g*. The scheme consists of three main algorithms* $\mathsf{EG} = (\mathsf{EG.KeyGen}, \mathsf{EG.Enc}, \mathsf{EG.Dec})$ *defined as follows:*

- $\mathsf{EG.KeyGen}$ ***(Key Generation):*** *Let $\mathcal{G}$ be a prime-order group generator that outputs (q, g), where q is a prime and g is a generator of the cyclic group $\mathbb{G}_q$. The key generation algorithm proceeds as follows:*
  - *Take the security parameter 1^λ as input.*
  - *Sample the secret key $\mathsf{sk} = x \xleftarrow{\$} \mathbb{Z}_q$ uniformly at random.*
  - *Compute the public key component $\mathsf{P} = g^x$.*
  - *The public key is $\mathsf{pk} = (q, g, \mathsf{P})$, and the secret key is $\mathsf{sk} = x$.*
- $\mathsf{EG.Enc}$ ***(Encryption):*** *To encrypt a message $M \in \mathbb{G}_q$:*
  - *Sample a random value $r \xleftarrow{\$} \mathbb{Z}_q$.*
  - *Compute: $\mathsf{C}_1 = g^r, \quad \mathsf{B} = \mathsf{P}^r$.*
  - *Compute the ciphertext components: $\psi = (\mathsf{C}_1, \mathsf{C}_2), \quad$ where $\mathsf{C}_2 = \mathsf{B} \cdot M$.*
- $\mathsf{EG.Dec}$ ***(Decryption):*** *Given a ciphertext $\psi = (\mathsf{C}_1, \mathsf{C}_2)$, the decryption process is as follows:*
  - *Compute: $\mathsf{B} = \mathsf{C}_1^x$*
  - *Recover the original message M by computing: $M = \mathsf{C}_2 \cdot \mathsf{B}^{-1}$.*

The security of the ElGamal encryption scheme under chosen-plaintext attack relies on the hardness of the Decisional Diffie-Hellman (DDH) problem in the group $\mathbb{G}_q$*.*

A.4 Commitment

Definition 13 (Commitment scheme). *A* commitment scheme $\mathsf{COM} = (\mathsf{KeyGen}, \mathsf{Commit}, \mathsf{Verify})$ *is defined by the following three algorithms:*

- $\mathsf{KeyGen}(1^\lambda)$*: A probabilistic algorithm that takes as input a security parameter λ, and outputs a pair $(\mathsf{ck}, \mathsf{vk})$, where:*
  - ck*: A commitment key used by the party to produce commitments.*
  - vk*: A verification key used to check the validity of commitments.*

  In schemes with public verifiability, we assume $\mathsf{ck} = \mathsf{vk}$.

- $\mathsf{Commit}(\mathsf{ck}, m)$*: A randomized algorithm that takes a commitment key* ck *and a message* $m \in \mathsf{MessageSpace}(\mathsf{ck})$*, and outputs a commitment* c *along with an opening value* d*:*

$$(c, d) \leftarrow \mathsf{Commit}(\mathsf{ck}, m).$$

  The tuple (c, d) *allows the party to later prove the committed message was* m*.*
- $\mathsf{Verify}(\mathsf{vk}, c, m, d)$*: A deterministic procedure that checks whether a commitment* c*, when opened with value* d*, corresponds to the message* m *using the verification key* vk*. The output is either* accept *or* reject*:*

$$\mathsf{Verify}(\mathsf{vk}, c, m, d) = \begin{cases} \mathsf{accept} & \text{if } (c, d) \text{ correctly opens to } m, \\ \mathsf{reject} & \text{otherwise.} \end{cases}$$

Definition 14 (Correctness). *A commitment scheme* $\mathsf{COM} = (\mathsf{KeyGen}, \mathsf{Commit}, \mathsf{Verify})$ *satisfies correctness if, for every security parameter* λ*, every message* $m \in \mathsf{MessageSpace}(\mathsf{ck})$*, and every key pair* $(\mathsf{ck}, \mathsf{vk}) \leftarrow \mathsf{KeyGen}(1^\lambda)$*, it holds that:*

$$\mathsf{Verify}(\mathsf{vk}, c, m, d) = \mathsf{accept}$$

with probability 1 over the randomness of Commit*, where* $(c, d) \leftarrow \mathsf{Commit}(\mathsf{ck}, m)$*. This property ensures that honest executions by both parties always lead to successful verification.*

Definition 15 (Hiding). *Consider a publicly verifiable commitment scheme* $\mathsf{COM} = (\mathsf{KeyGen}, \mathsf{Commit}, \mathsf{Verify})$*. The hiding property states that no PPT adversary* $\mathcal{A}$ *can distinguish the commitments of any two equal-length messages with significant advantage. The hiding experiment* $\mathsf{Hide}^b_{\mathsf{COM}}(\mathcal{A}, \lambda)$*, parameterized by a bit* $b \in \{0, 1\}$*, proceeds as follows:*

1. *The challenger runs* $\mathsf{KeyGen}(1^\lambda)$ *to generate* $\mathsf{ck} = \mathsf{vk}$ *and gives* ck *to* $\mathcal{A}$*.*
2. $\mathcal{A}$ *returns two messages* (m_0, m_1) *of equal length from* $\mathsf{MessageSpace}(\mathsf{ck})$*.*
3. *A random bit* $b \xleftarrow{\$} \{0, 1\}$ *is sampled, and* $(c, d) \leftarrow \mathsf{Commit}(\mathsf{ck}, m_b)$ *is computed. The challenger sends* c *to* $\mathcal{A}$*.*
4. $\mathcal{A}$ *outputs a guess* b'*.*

The adversary's distinguishing advantage is defined by:

$$\mathsf{Adv}^{\mathsf{Hide}}_{\mathcal{A}}(\lambda) = \left| \Pr[\mathsf{Hide}^1_{\mathsf{COM}}(\mathcal{A}, \lambda) = 1] - \Pr[\mathsf{Hide}^0_{\mathsf{COM}}(\mathcal{A}, \lambda) = 1] \right|.$$

The hiding property is satisfied if:

- *Perfect hiding:* $\mathsf{Adv}^{\mathsf{Hide}}_{\mathcal{A}}(\lambda) = 0$ *for all* $\mathcal{A}$*.*
- *Computational hiding:* $\mathsf{Adv}^{\mathsf{Hide}}_{\mathcal{A}}(\lambda) \leq \mathsf{negl}(\lambda)$ *for all PPT adversaries, where* $\mathsf{negl}(\cdot)$ *denotes a negligible function.*

This ensures that the commitment leaks no useful information about the committed message.

Definition 16 (Binding). *In a (publicly verifiable) commitment scheme* $\mathsf{COM} = (\mathsf{KeyGen}, \mathsf{Commit}, \mathsf{Verify})$*, the binding property asserts that it is infeasible for a PPT adversary* $\mathcal{A}$ *to find two distinct openings for the same commitment. The binding experiment* $\mathsf{Bind}_{\mathsf{COM}}(\mathcal{A}, \lambda)$ *runs as follows:*

1. *Generate keys* $(\mathsf{ck}, \mathsf{vk}) \leftarrow \mathsf{KeyGen}(1^\lambda)$*, and provide* ck *to* $\mathcal{A}$*.*
2. $\mathcal{A}$ *returns* (c, m, d, m', d') *with* $m, m' \in \mathsf{MessageSpace}(\mathsf{ck})$*.*
3. *The challenger checks if:*

$$\mathsf{Verify}(\mathsf{vk}, c, m, d) = \mathsf{accept} \quad \textit{and} \quad \mathsf{Verify}(\mathsf{vk}, c, m', d') = \mathsf{accept},$$

and whether $m \neq m'$*.*

The adversary's success probability is:

$$\mathsf{Adv}^{\mathsf{Bind}}_{\mathcal{A}}(\lambda) = \Pr[\mathsf{Bind}_{\mathsf{COM}}(\mathcal{A}, \lambda) = 1].$$

A scheme satisfies binding if:

- *Perfect binding:* $\mathsf{Adv}^{\mathsf{Bind}}_{\mathcal{A}}(\lambda) = 0$ *for all adversaries.*
- *Computational binding:* $\mathsf{Adv}^{\mathsf{Bind}}_{\mathcal{A}}(\lambda) \leq \mathsf{negl}(\lambda)$ *for all PPT adversaries.*

This ensures that a commitment uniquely determines the committed message.

Definition 17 (Pedersen commitment scheme). *The Pedersen commitment scheme [Pedersen(1991)], denoted* $\mathsf{PCOM} = (\mathsf{KeyGen}, \mathsf{Commit}, \mathsf{Verify})$*, is constructed as follows:*

- $\mathsf{KeyGen}(1^\lambda)$*: On input* λ*, select:*
  - G*: A cyclic group of prime order* q*,*
  - $g \in G$*: A generator,*
  - $h \xleftarrow{\$} G$*: An element sampled uniformly at random, independent of* g*.*

  The commitment and verification keys are identical:

$$\mathsf{ck} = \mathsf{vk} = (G, q, g, h).$$

- $\mathsf{Commit}(\mathsf{ck}, m)$*: For a message* $m \in \mathbb{Z}_q$*, choose* $d \xleftarrow{\$} \mathbb{Z}_q$*, and compute:*

$$c = g^d \cdot h^m.$$

  The output is the pair (c, d)*.*
- $\mathsf{Verify}(\mathsf{vk}, c, m, d)$*: Accept if:*

$$g^d \cdot h^m = c.$$

  Otherwise, reject.

This scheme enjoys the following security guarantees [Micali et al.(2003), Metere and Dong(2017)]:

- *Perfect hiding: The commitment value* c *leaks no information about* m *due to the randomness* g^d*.*
- *Computational binding: It is computationally hard to find two distinct openings* (m, d) *and* (m', d') *such that:*

$$g^d \cdot h^m = g^{d'} \cdot h^{m'}.$$

B Ideal functionalities

B.1 Non-interactive zero-knowledge (NIZK) functionality $\mathcal{F}_{\mathsf{NIZK}}$

Groth et al. [Groth et al.(2006)] provided a formal definition of non-interactive zero-knowledge (NIZK) in the form of an ideal functionality $\mathcal{F}_{\mathsf{NIZK}}$. The ideal functionality $\mathcal{F}_{\mathsf{NIZK}}$ is parameterized by a relation $\mathcal{R}$ and consists of two phases: *proof generation* and *proof verification*. Upon receiving a request $(\mathtt{Prove}, \mathsf{sid}, x, w)$ from a party P, $\mathcal{F}_{\mathsf{NIZK}}$ checks whether $(x, w) \in \mathcal{R}$. If this condition does not hold, the request is ignored. Otherwise, $\mathcal{F}_{\mathsf{NIZK}}$ sends $(\mathtt{Prove}, \mathsf{sid}, x)$ to the adversary $\mathcal{A}$. Upon receiving a response $(\mathtt{Proof}, \mathsf{sid}, \pi)$ from $\mathcal{A}$, the functionality stores the tuple (x, π) and sends $(\mathtt{Proof}, \mathsf{sid}, \pi)$ back to P.
In our construction, we aim to ensure that the statement x is not revealed to the adversary. Recall that during the issuance protocol, the user generates t pairs (x_i, π_i) for $i = 1, \ldots, t$, where each pair corresponds to the i-th issuer Iss_i. The user's statement includes secret sharings of the user's real-world identifier s ($s_i \in x_i$, see Figure 6), which must remain confidential. Specifically, the statement x_i is sent by the user to the i-th issuer Iss_i (via a secure channel), where there may be up to $t-1$ malicious issuers among the t participating issuers in the issuance process. Consequently, the adversary gains access to all t statements, including those provided securely to the honest issuers. This implies that the adversary also learns the shares s_i corresponding to *honest issuers*, and thus, having all t shares would enable the reconstruction of s. Note that this issue arises due to the nature of the UC modeling. In real-world implementations, the proof generation is performed locally by the (honest) user, ensuring that the adversary does not gain access to the statement x.
A concept closely related to NIZK is the notion of *signatures of knowledge* (SoK). SoK combines digital signatures with zero-knowledge proofs and allows a signer to authenticate a message not only by proving possession of a secret key but also by demonstrating knowledge of a witness w for an NP statement $x \in L$. Such a signature ensures that, upon verification, it is confirmed that the signer possesses a valid witness for the truth of the statement. The concept of SoK was formally defined by Chase et al. [Chase and Lysyanskaya(2006)] via an ideal functionality $\mathcal{F}_{\mathsf{SoK}}$. Similar to $\mathcal{F}_{\mathsf{NIZK}}$, $\mathcal{F}_{\mathsf{SoK}}$ is for $(x, w) \in \mathcal{R}$. It consists of three primary phases: *setup*, *signature generation*, and *signature verification*. Unlike $\mathcal{F}_{\mathsf{NIZK}}$, $\mathcal{F}_{\mathsf{SoK}}$ receives $(\mathtt{Verify}, \mathtt{Sign}, \mathsf{SimSign}, \mathsf{Extract})$ from the adversary $\mathcal{A}$. Upon receiving a request $(\mathtt{Sign}, \mathsf{sid}, m, x, w)$ from a party P, $\mathcal{F}_{\mathsf{SoK}}$ checks whether $(x, w) \in \mathcal{R}$. If the check passes, $\mathcal{F}_{\mathsf{SoK}}$ computes $\sigma \leftarrow \mathsf{SimSign}(m, x)$. Unlike $\mathcal{F}_{\mathsf{NIZK}}$, $\mathcal{F}_{\mathsf{SoK}}$ does not leak the statement x to the adversary.
To avoid leaking the statement to the adversary in the proof generation interface, we utilize $\mathcal{F}_{\mathsf{SoK}}$, which inherently prevents such leakage. Although $\mathcal{F}_{\mathsf{SoK}}$ introduces a message m as part of the signature, this component is unnecessary in our setting. Therefore, we simplify $\mathcal{F}_{\mathsf{SoK}}$ by fixing m to a predefined dummy message for all sessions. This transforms $\mathcal{F}_{\mathsf{SoK}}$ into a form that functions as $\mathcal{F}_{\mathsf{NIZK}}$ without the undesired statement leakage. Therefore, in the following we present the formal definition of $\mathcal{F}_{\mathsf{SoK}}$ from [Chase and Lysyanskaya(2006)] un-

der a simplifying assumption: the message m in all interfaces and algorithms is a predefined (dummy) message, hence, we avoid addressing it. Without loss of generality, we denote the following functionality as $\mathcal{F}_{\mathsf{NIZK}}$.

Functionality $\mathcal{F}_{\mathsf{NIZK}}$

The functionality is parameterized by a relation $\mathcal{R}$. Prove, SimProve, and Extract are descriptions of PPT TMs, and Verify is a description of a deterministic polytime TM.

1. **Setup.** Upon receiving $(\texttt{Setup}, \mathsf{sid})$ from some party P if this is the first time that $(\texttt{Setup}, \mathsf{sid})$ is received, send $(\texttt{Setup}, \mathsf{sid})$ to $\mathcal{A}$; upon receiving $(\texttt{Algorithms}, \mathsf{sid}, \mathsf{Prove}, \mathsf{Verify}, \mathsf{SimProve}, \mathsf{Extract})$ from $\mathcal{A}$, store these algorithms. Output $(\texttt{Algorithms}, \mathsf{sid}, \mathsf{Prove}, \mathsf{Verify})$ to P.
2. **Proof generation.** Upon receiving $(\texttt{Prove}, \mathsf{sid}, x, w)$ from P, if $(x, w) \notin \mathcal{R}$, ignore. Else, compute $\pi \leftarrow \mathsf{SimProve}(x)$, and check that $\mathsf{Verify}(x, \pi) = 1$. If so, then output $(\texttt{Proof}, \mathsf{sid}, \pi)$ to P and record the entry (x, π). Else, output an error message (*Completeness error*) to P and halt.
3. **Proof verification.** Upon receiving $(\texttt{Verify}, \mathsf{sid}, x, \pi)$ from P: if (x, π) is stored, then output $(\texttt{Verification}, \mathsf{sid}, 1)$ to P. Else, let $w \leftarrow \mathsf{Extract}(x, \pi)$; if $(x, w) \in \mathcal{R}$, output $(\texttt{Verification}, \mathsf{sid}, 1)$ to P. Else if $\mathsf{Verify}(x, \pi) = 0$, output $(\texttt{Verification}, \mathsf{sid}, 0)$ to P. Else output an error message (*Verification error*) to P and halt.

B.2 Communication channel functionality $\mathcal{F}_{\mathsf{Ch}}^{\mathsf{sas}}$

Our functionality $\mathcal{F}_{\mathsf{SyRA}}$ does not reveal the network identifiers of users to adversary. To realize this functionality, our protocol Π_{SASSI} employs sender-anonymous and secure communication channel $\mathcal{F}_{\mathsf{Ch}}^{\mathsf{sas}}$ to deliver messages and to fulfill network-level anonymity. As discussed in Section 4, regardless of the robustness of cryptographic safeguards at the protocol level, "network leakage" could potentially expose the parties involved in a communication. Hence, in our formal UC modelling, a degree of sender anonymity is essential for maintaining (UC) anonymity.

The purpose of using $\mathcal{F}_{\mathsf{Ch}}^{\mathsf{sas}}$ in our description is to highlight that there is network leakage and that it can compromise privacy with respect to the functionality. This is not specific to Π_{SASSI} but a common issue whenever one formalizes any privacy-preserving primitive in the UC setting. (For example consider formalizing Zerocash [Ben-Sasson et al.(2014)] in the UC setting; without sender anonymous channel it will offer no privacy whatsoever. SASSI is no different in this respect.) The alternative approach would have been to let such leakage come through the functionality, but this would complicate the program of the

functionality much more – we opted to avoid this. We expect that in practice, deployments of SASSI will be used via the usual and customary compromises people make in these settings to minimize network leakage (and this is exactly as in Zerocash and other similar schemes), e.g., running the protocol via a VPN, Tor [Dingledine et al.(2004)], a short-lived network identity (e.g., a computer terminal in a library), or JAP [JAP(2009)].

Functionality $\mathcal{F}_{\mathsf{Ch}}^{\mathsf{sas}}$

Let S be the sender and R the receiver of a message m.

1. Upon input $(\mathtt{Send}, \mathsf{sid}, \mathsf{R}, m)$ from S, record a mapping $\mathsf{P}(\mathsf{mid}) \leftarrow (\mathsf{S}, \mathsf{R}, m)$ where mid is chosen at random. Output $(\mathtt{Send}, \mathsf{sid}, \mathsf{R}, \mathsf{mid}, |m|)$ to $\mathcal{A}$.
2. Upon receiving $(\mathtt{Ok}, \mathsf{sid}, \mathsf{mid})$ from $\mathcal{A}$, retrieve $(\mathsf{S}, \mathsf{R}, m) \leftarrow \mathsf{P}(\mathsf{mid})$ and send $(\mathsf{Received}, \mathsf{sid}, \mathsf{mid}, m)$ to R.
3. Upon receiving $(\mathtt{SendBack}, \mathsf{sid}, \mathsf{mid}, m')$ from R, record (mid, m'). Output $(\mathtt{SendBack}, \mathsf{sid}, \mathsf{R}, \mathsf{mid}, |m'|)$ to $\mathcal{A}$.
4. Upon receiving $(\mathtt{SendBackOk}, \mathsf{sid}, \mathsf{mid})$ from $\mathcal{A}$, retrieve $(\mathsf{S}, \mathsf{R}, m) \leftarrow \mathsf{P}(\mathsf{mid})$, and (mid, m'). Send $(\mathtt{ReceivedBack}, \mathsf{sid}, \mathsf{R}, m')$ to S.

B.3 Broadcast functionality $\mathcal{F}_{\mathsf{B}}$

The broadcast functionality introduced in [Garay et al.(2011)] does not provide secrecy guarantees for the broadcasted message m. Furthermore, it reveals the identity of the sender to the recipients. In our work, we require *UC-sender anonymity* (see Appendix B.2 for details), which necessitates hiding the sender's identity from the receiving entities. We refer to this as *sender-anonymous broadcast*. One possible approach to realize this involves using sender-anonymous point-to-point communication channels between the sender and the receiving servers. Subsequently, the servers can run a Byzantine agreement protocol (such as the one proposed in [Boyle et al.(2021)]) to agree on the message content as described in [Sarencheh et al.(2025b)].

Functionality $\mathcal{F}_{\mathsf{B}}$

The broadcast functionality $\mathcal{F}_{\mathsf{B}}$ is parameterized by a set of parties $\mathbf{I} = \{\mathsf{Iss}_1, \cdots, \mathsf{Iss}_t\}$ and proceeds as follows.

1. Upon receiving $(\mathtt{Broadcast}, \mathsf{sid}, m)$ from a party P, record $(\mathsf{mid}, \mathsf{P})$, where mid is freshly sampled. Send $(\mathtt{Broadcasted}, \mathsf{sid}, m, \mathsf{mid})$ to all parties in $\mathbf{I}$ and to $\mathcal{A}$.

2. Upon receiving $(\texttt{Send.Back}, \mathsf{sid}, m', \mathsf{mid})$ from some $\mathsf{Iss}_i \in \mathbf{I}$, retrieve $(\mathsf{mid}, \mathsf{P})$, and record $(\mathsf{Iss}_i, m', \mathsf{mid}, \mathsf{P})$. Output $(\texttt{Send}, \mathsf{sid}, \mathsf{Iss}_i, \mathsf{mid})$ to $\mathcal{A}$.
3. Upon receiving $(\texttt{Send.Back.Ok}, \mathsf{sid}, \mathsf{Iss}_i, \mathsf{mid})$ from $\mathcal{A}$, retrieve $(\mathsf{Iss}_i, m', \mathsf{mid}, \mathsf{P})$, and send $(\texttt{Received}, \mathsf{sid}, \mathsf{Iss}_i, m')$ to P.

B.4 Distributed key generation (DKG) functionality $\mathcal{F}_{\mathsf{DKG}}^{n,t}$

We model distributed key generation via the functionality $\mathcal{F}_{\mathsf{DKG}}^{n,t}$ in [Doerner et al.(2023), Katz(2024)]. (We adapt [Doerner et al.(2023)] to output public key shares explicitly so that it is equivalent to [Katz(2024)].) The functionality $\mathcal{F}_{\mathsf{DKG}}^{n,t}$ assumes that the adversary $\mathcal{A}$ can corrupt up to $t-1$ parties and that $\mathcal{A}$ learns the public key shares $\{A_i\}_{i=1}^n$ of all parties. The adversary determines the secret key shares for corrupted parties, while the remaining secret key shares are generated uniformly at random. For concreteness, $\mathcal{F}_{\mathsf{DKG}}^{n,t}$ assumes Shamir secret sharing of the secret key α, but it can be adapted to other secret-sharing schemes. For example, an n-out-of-n additive sharing of α can be used when $t = n - 1$.

Functionality $\mathcal{F}_{\mathsf{DKG}}^{n,t}$

The functionality is parameterized by the number of parties n, the threshold t, and the group $\mathbb{G}$ with order p. The index set $\mathbf{C}$ denotes the parties corrupted by the adversary $\mathcal{A}$ where $\mathsf{max}\ |\mathbf{C}| = t - 1$.

1. Upon receiving $(\texttt{DKeyGen}, \mathsf{sid})$ from some party Iss_i, send $(\texttt{DKeyGenReq}, \mathsf{sid}, \mathsf{Iss}_i)$ to $\mathcal{A}$.
2. Upon receiving $(\texttt{CPoints}, \mathsf{sid}, \{\alpha_i\}_{i\in\mathbf{C}})$ from $\mathcal{A}$, and when $(\texttt{DKeyGen}, \mathsf{sid})$ is received from all parties, proceed as follows.
3. Sample $\alpha \leftarrow \mathbb{Z}_p$, and set $A \leftarrow g^\alpha$.
4. Sample a random polynomial f of degree $t-1$ over $\mathbb{Z}_p$ where $\{\mathsf{f}(i) = \alpha_i\}_{i\in\mathbf{C}}$ and $\mathsf{f}(0) = \alpha$.
5. For each $i \in [n]$, compute $A_i \leftarrow g^{\mathsf{f}(i)}$.
6. Send $(\texttt{PublicKeys}, \mathsf{sid}, A, \{A_i\}_{i=1}^n)$ to $\mathcal{A}$, and $(\texttt{Point}, \mathsf{sid}, A, \mathsf{f}(i), \{A_i\}_{i=1}^n)$ to each party Iss_i for $i \in [n]$ via private-delayed output.

B.5 Multiplication functionality $\mathcal{F}_{\mathsf{mul}}$

The multiplication ideal functionality $\mathcal{F}_{\mathsf{mul}}$ involves t parties $\mathbf{I} = (\mathsf{Iss}_1, \ldots, \mathsf{Iss}_t)$. It enables parties to collaboratively compute a multiplication result in the exponent without revealing their individual inputs. It is parameterized by a secret sharing aggregation algorithm $\mathsf{SSH.Agg}$, to reconstruct the secret given t shares.

Operations are performed over a finite field defined by a prime q. The functionality proceeds by first handling inputs from the parties. Each party $\mathsf{lss}_i \in \mathbf{I}$ submits a message $(a_i, \vec{b}_i, \mathsf{aux}_i)$ where aux_i is used to check the well-formedness of $\vec{b}_i$. Non well-formed inputs or those from entities outside the set $\mathbf{I}$ are ignored. Once all t parties provide their inputs, the functionality employs $\mathsf{SSH.Agg}$ to reconstruct the values a and b from their respective t shares. Subsequently, the functionality performs the multiplication $\phi = a \cdot b \mod q$. The group elements $g^{\phi^{-1}}$, and $\hat{g}^{\phi^{-1}}$ are then sent to all parties where g and $\hat{g}$ are source group generators.
See Section 5 for implementation details of $\mathcal{F}_{\mathsf{mul}}$, based on [Doerner et al.(2023)]. Below, we summarize the high-level structure of the protocol introduced in [Doerner et al.(2023)]. The protocol begins with the user providing a message $\mathbf{m}$ to be signed and a subset $\mathbf{I} \subseteq [n]$ representing the indices of the selected signing servers. Assume that these t servers collectively possess a secret sharing $\mathbf{r}$ of a random scalar r, along with common knowledge of uniformly chosen values s and e. Furthermore, suppose the servers hold a secret sharing $\mathbf{u}$ of the product $u = r \cdot (x + e)$. Each server lss_i can locally compute:

$$R_i := \left(g_1 \cdot h_1^s \cdot \prod_{k \in [\ell]} h_{k+1}^{m_k} \right)^{r_i}$$

If each participant lss_i then transmits the tuple (s, e, R_i, u_i) to the user, the user is able to reconstruct:

$$A := \left(\prod_{i \in \mathbf{I}} R_i \right)^{\left(\prod_{i \in \mathbf{I}} u_i\right)^{-1}} = \left(g_1 \cdot h_1^s \cdot \prod_{k \in [\ell]} h_{k+1}^{m_k} \right)^{r \cdot (r \cdot (x+e))^{-1}}$$

which constitutes a BBS+ signature on the message $\mathbf{m}$. Both s and e can be generated using a commit-and-open coin tossing mechanism. The shares $\mathbf{r}$ are locally sampled by the servers, and a two-round secure multiplication protocol is employed to compute shares of the pairwise products $r_i \cdot \text{lagrange}(\mathbf{I}, j, 0) \cdot p(j)$ for each pair $(i, j) \in \mathbf{I} \times \mathbf{I}$. Here, $\text{lagrange}(\mathbf{I}, j, 0)$ refers to the Lagrange coefficient used by server lss_j for reconstructing the secret at the origin using Shamir's secret sharing scheme over the set $\mathbf{I}$, and p denotes a Shamir secret sharing of the secret value x. Each server lss_i computes its share u_i by adding its local shares of these pairwise products with $r_i \cdot e$. To protect against incorrect inputs from corrupted servers, the protocol rerandomizes the multiplier inputs by adding secret sharings of zero. This design ensures that any bias introduced into the outputs of honest parties, as a result of adversarial misbehavior, remains independent of their secret inputs. The generation of sharings of 0 is done non-interactively using established techniques.

Functionality $\mathcal{F}_{\mathsf{mul}}$

The functionality $\mathcal{F}_{\mathsf{mul}}$ is parameterized by t parties $\mathbf{I} = (\mathsf{lss}_1, \ldots, \mathsf{lss}_t)$. Let $\mathsf{SSH.Agg}$ be the secret sharing aggregation algorithm.

1. **Input.** Upon receiving $(\mathtt{Compute}, \mathsf{sid} \,||\, \mathsf{id} \,||\, \mathbf{I}, a_i, \vec{b}_i, \mathsf{aux}_i)$ from lss_i, ignore if $\mathsf{lss}_i \notin \mathbf{I}$ where $\mathsf{sid} = (\mathsf{sid}', \mathbf{I})$. Else, parse $\vec{b}_i = (b_i^{(1)}, b_i^{(2)})$, and $\mathsf{aux}_i = (z_i, \{\hat{\mathsf{ivk}}_k\}_{k=1}^t, \mathsf{com}_{\mathbb{I}} = \{\mathsf{com}_k\}_{k=1}^t)$. Ignore if $\hat{\mathsf{ivk}}_i \neq \hat{g}^{b_i^{(1)}}$ or if $\mathsf{com}_i \neq g^{b_i^{(2)}} \cdot h^{z_i}$. Else, record $(a_i, \vec{b}_i, \mathsf{aux}_i, \mathsf{lss}_i, \mathbf{I}, \mathsf{id})$. Upon receiving messages from all t parties in $\mathbf{I}$ with id, ignore if not all $(\{\hat{\mathsf{ivk}}_k\}_{k=1}^t, \mathsf{com}_{\mathbb{I}})$ are the same in t inputs received. Else, call $\psi \leftarrow \mathsf{SSH.Agg}\,\{\psi_i\}_{\forall i \in \mathrm{index}(\mathbf{I})}$ for $\psi_i = a_i$ and $(b_i^{(1)} + b_i^{(2)})$ generating a, and b. Sample a fresh α, and set $f(\alpha) \leftarrow (a, b, \mathbf{I})$. Leak $(\mathtt{Compute}, \mathsf{sid} \,||\, \mathsf{id} \,||\, \mathbf{I}, \alpha)$ to the adversary $\mathcal{A}$.
2. **Output.** Upon receiving $(\mathtt{Compute.Ok}, \mathsf{sid} \,||\, \mathsf{id} \,||\, \mathbf{I}, \alpha)$ from the adversary $\mathcal{A}$, ignore if $f(\alpha) = \perp$. Else, retrieve $f(\alpha) = (a, b, \mathbf{I})$. Compute $\phi \leftarrow a \cdot b \bmod q$, $K \leftarrow g^{\phi^{-1}}$, and $\hat{K} \leftarrow \hat{g}^{\phi^{-1}}$. Output $(\mathtt{Computed}, \mathsf{sid} \,||\, \mathsf{id} \,||\, \mathbf{I}, K, \hat{K})$ to each party $\mathsf{lss}_i \in \mathbf{I}$.

B.6 Random oracle functionality $\mathcal{F}_{\mathsf{RO}}$

$\mathcal{F}_{\mathsf{RO}}$ models an idealized hash function [Camenisch et al.(2018)] and is defined as follows. The third check is included to avoid output collisions for simplicity, though it is removed in realizations. We follow this approach to streamline the formal analysis, e.g., similar to [Rial and Piotrowska(2022)].

Functionality $\mathcal{F}_{\mathsf{RO}}$

The functionality is parameterized by a message space M and output space H. $\mathcal{F}_{\mathsf{RO}}$ functions as follows:
Upon receiving $(\mathtt{Query}, \mathsf{sid}, m)$ from a party P:

1. Ignore if $m \notin M$.
2. If there exists a tuple (sid, m', h') where $m' = m$, set $h \leftarrow h'$.
3. Else, select $\bar{h} \xleftarrow{\$} H$ such that there is no stored tuple (sid, m^*, h') where $h' = \bar{h}$. Set $h \leftarrow \bar{h}$.
4. Store (sid, m, h).
5. Output $(\mathtt{Query.Re}, \mathsf{sid}, h)$ to party P.